

Finding and Fixing Software Vulnerabilities Using AI: A Guide for Developers

David A. Wheeler and OpenSSF Contributors

Abstract

This is a guide for software developers and vulnerability researchers on finding and fixing vulnerabilities using artificial intelligence (AI). We intend to create a course using this guide. This material is a joint effort between the OpenSSF Best Practices working group (WG) and the OpenSSF AI/ML WG.

Table of Contents

1 Introduction 4
- 1.1 Scope of this material 4
- 1.2 Learning outcomes 5
- 1.3 No endorsement intended 5
- 1.4 Why this is important 5
  - 1.4.1 AI is accelerating vulnerability-finding 6
  - 1.4.2 Attack speed is accelerating 7
  - 1.4.3 Manual closed approaches are failing 7
  - 1.4.4 Still need humans 8
- 1.5 What hasn't changed 8
- 1.6 Good news: A Path to Near-Zero Vulnerabilities 9
- 1.7 Key security terms 10
- 1.8 Overview 11

2 AI concepts 12
- 2.1 Overview & Learning Objectives 12
- 2.2 Basic AI terminology 12
- 2.3 Strengths and weaknesses 13
- 2.4 Models 14
  - 2.4.1 External versus local models 14
  - 2.4.2 Licenses 15
  - 2.4.3 Specialized models 16
  - 2.4.4 AI guardrails and intentional limitations 16

2.5 Cyber Reasoning System (CRS) history 17
2.6 AI organizations, projects, and services 18
3 Decide on approaches 19
3.1 Do not ignore AI 19
3.2 Don’t Let Model Access Hold You Back 20
3.3 Do simple things first 21
3.4 Do not presume AI can correctly fix all vulnerabilities 23
3.5 Getting Better Results with Structured Workflows 24
3.6 How to apply processes for using AI to find and fix vulnerabilities 27
3.7 Examples of relevant tools and services 29
3.7.1 Challenges of finding and evaluating tools 29
3.7.2 Sample services and tools 30
3.7.3 Scrutineer 31
3.7.4 OSS-CRS 31
3.8 Run AI multiple times 34
4 Prepare to find and fix vulnerabilities 36
4.1 Preparing AI 36
4.2 Preparing threat models 37
4.3 Preparing the sandbox 39
4.3.1 Need for sandboxes 40
4.3.2 Value of sandboxes 40
4.3.3 Implementing sandboxes 41
4.3.4 Establish a human gate and kill switch 42
4.4 Preparing AI logging 44
4.5 Preparing the code and documentation 44
4.6 Preparing CI/CD 45
4.7 Preparing dependency updates 47
5 Core tasks for finding and fixing vulnerabilities 48
5.1 Identify findings 48
5.1.1 Separate identifying findings from validation 49
5.1.2 Helping to identify findings 49
5.1.3 Managing findings 52
5.2 Handle external findings 53
5.3 Deduplicate 54
5.4 Validate findings 54
5.4.1 Defining what a vulnerability is 55
5.4.2 Defining the proof required 56
5.4.3 Tips for validation 57
5.5 Triage 58
5.6 Fix vulnerabilities 60
5.6.1 Fixing may be necessary for sending reports to external organizations 61
5.6.2 Assigning the fix 61
5.6.3 Creating tests and confirming the root problem 61

5.6.4 How to create the fix 62
5.6.5 Only provide correct information, especially if fully automated 62
5.6.6 Approaching fixes....63
5.7 Verify fixes 64
5.7.1 Have AI review the fix first....64
5.7.2 Have humans review the fix 64
5.7.3 Apply your usual verification processes 65
5.7.4 Turn findings into durable checks....66
5.8 Report vulnerabilities....66
5.8.1 Reporting to externally maintained projects 66
5.8.2 Reporting to authorities....69
5.8.3 Reporting to users....69
5.9 Release and deploy....70
5.10 Repeated application....71
6 Preventing vulnerabilities longer term....73
6.1 Limit vibe coding....73
6.2 Evaluate merge/pull requests 74
6.3 Apply secure by design and secure by default 75
6.4 Harden....76
6.4.1 Applying hardening 77
6.4.2 Harden to counter chaining....78
6.4.3 Evaluate with hardening disabled and evaluate the hardening 79
6.4.4 Harden the deployed environment and infrastructure 80
7 Call to action 82
8 Acknowledgements....83
9 Bibliography 84
10 License 94

1 Introduction

Welcome to “Finding and Fixing Software Vulnerabilities Using AI: A Guide for Developers”. This is a guidance document; we intend to eventually turn this material into an online course (LFD126). You can retrieve its contents as a markdown file and see its overview.

1.1 Scope of this material

This is material on finding and fixing vulnerabilities in software using artificial intelligence (AI)/ machine learning (ML). It is intended for:

- *software developers*, to help them find and fix vulnerabilities in software (especially software they’re responsible for), and for
- *security researchers*, who are trying to help those software developers.

It includes the process of finding, validating, and generating fixes, as well as related tasks that make this process more effective. These processes apply when examining the entire project or a particular set of proposed changes. We expect this material to apply to any software regardless of its license, but we do include a few notes specifically on open source software (OSS).

We don’t focus on any one *specific* system to do this. Many systems can help with this, more are being released, using multiple systems can be helpful, and the industry is rapidly changing. We instead focus on general principles that we believe are more timeless and will help you regardless of the systems you use to find and fix vulnerabilities. Once you understand the general issues, you’ll be more effective when using any particular system.

For a more general introduction on applying AI/ML to software development and security, see our course *Secure AI/ML-Driven Software Development (LFEL1012)* at https://training.linuxfoundation.org/express-learning/secure-ai-ml-driven-software-development-lfel1012/.

Please note that this material does NOT focus on the following topics:

1. Building AI models & AI systems.
2. Building/fixing software or systems that include AI. This material is still applicable, but we don’t discuss anything special related to that situation.
3. How to write secure software in general. Please take our LFD121 course to learn that.
4. Evaluating malicious or possibly-malicious software. We’re presuming that software developers are investigating their own software or software they want to contribute to, and that it isn’t intended to be malicious. The focus here is on *unintentional* vulnerabilities.

This material is a joint effort between the OpenSSF Best Practices working group (WG) and the OpenSSF AI/ML WG. Its lead author is David A. Wheeler. Reviewers include Laura Guazzelli.

1.2 Learning outcomes

When you complete this material, you will be able to:

1. Prepare to find and fix vulnerabilities. This includes how to prepare the AI and its sandbox, as well as how to prepare the software threat model, code/documentation, and CI/CD processes to make this more effective.
2. Find & fix vulnerabilities using AI. This includes:
    1. Knowing the value of separating the process of identifying findings from the process of validating them.
    2. How to identify findings (potential vulnerabilities), including mechanisms for increasing the likelihood of finding potential vulnerabilities such as using past vulnerability reports.
    3. How to deduplicate, validate, and triage findings.
    4. How to fix vulnerabilities and verify the fixes, including the importance of ensuring that a defect is fully fixed.
    5. Knowing important aspects of reporting, releasing, and deploying the fixes.
    6. Understanding the need for repeatedly looking for vulnerabilities when using AI, given the non-deterministic nature of modern AI, where a single application may miss important issues.
3. Prevent longer-term problems. This includes evaluating merge/pull requests, the need for secure-by-design and secure-by-default, and the importance of larger-scale hardening.

1.3 No endorsement intended

This material mentions many AI models, systems, tools, and services. However, no specific endorsement is intended. Specific technologies may have changed by the time you read this, since the industry is undergoing rapid change.

Our focus is on general principles that are less likely to change. The specific models, services, and statistics we cite reflect what was known when this material was last reviewed; expect them to change, while the practices should generally remain valid.

1.4 Why this is important

This material is critically important for today's software developers and security researchers. AI has become incredibly good at finding vulnerabilities, for both attackers *and* defenders. This is requiring defenders to *rapidly* find and fix vulnerabilities using AI. If they don't, their resulting systems will be

repeatedly taken over by attackers who are *already* using AI. CrowdStrike found that even back in 2025, “AI-enabled adversaries increased attacks by 89% year-over-year” [CrowdStrike2026-Global]. That’s accelerating now.

There are many reasons this topic is important. Here we summarize them, with quotes and citations showing that this is *real*. In short, AI has sped up vulnerability-finding, attackers are using that increased speed to accelerate their attacks, and traditional manual closed processes are failing to keep up. Yet there is still a role for humans, *if* the humans understand how to apply AI to it

1.4.1 AI is accelerating vulnerability-finding

AI has greatly sped up vulnerability finding in software:

- *The cost, expertise, and effort to find vulnerabilities has collapsed*.
  - “With the latest frontier AI models, the cost, effort, and level of expertise required to find and exploit software vulnerabilities have all dropped dramatically. Over the past year, AI models have become increasingly effective at reading and reasoning about code—in particular, they show a striking ability to spot vulnerabilities and work out ways to exploit them.” [Anthropic2026-04g]
- *Far more vulnerabilities are being found with AI*. Agentic AI can find *far* more vulnerabilities (exploitable defects) than non-AI systems. AI can use other tools and integrate that information to discover problems.
  - One paper “found that [AI was] crucial to the success of [finding vulnerabilities; without AI traditional fuzzers] were unable to discover a single bug within a four-hour time limit in any of 20 trials. …. [this] highlights the importance of integrating LLM-assisted tooling into automated security workflows.” [Wolff2026]
- *AI can rapidly turn vulnerabilities into exploits, often by chaining multiple defects together*. AI’s ability to rapidly turn a vulnerability into an exploit accelerates the need for repair. In particular, AI can often combine multiple minor-seeming defects into a devastating chain.
  - “Claude Mythos Preview demonstrates a leap in these cyber skills—the vulnerabilities it has spotted have in some cases survived decades of human review and millions of automated security tests, and the exploits it develops are increasingly sophisticated… frontier AI models are now becoming competitive with the best humans at finding and exploiting vulnerabilities.” [Anthropic2026-04g]
- *The speed at which vulnerabilities are found and exploited has dramatically accelerated*.
  - “AI is making it possible to detect severe security vulnerabilities at highly accelerated speeds.” [Anthropic2026-03]
- *Far more attackers can now create sophisticated attacks*.
  - “Advanced frontier models (like Claude Mythos Preview) and optimized open-weight models have democratized the ability to find complex vulnerabilities and construct exploit chains, exposing non-traditional targets to high-level threats.” [Wolff2026]

1.4.2 Attack speed is accelerating

Since the speed of finding vulnerabilities has increased, and cost of finding vulnerabilities has decreased, the speed of attacks that exploit those vulnerabilities has increased:

- *Attackers are becoming faster and more dangerous*.
  - [FiveEyes2026] says that "Adversaries are already using AI to move faster and more effectively. Defenders must do the same."
  - CrowdStrike similarly says "AI systems are beginning to assist with tasks that materially improve offensive velocity [and will be used by] adversaries." [CrowdStrike2026-FiveSteps]
  - "The window between a vulnerability being publicly disclosed and weaponized has dramatically collapsed to hours or minutes, as AI can instantly reverse-engineer patches to create exploit blueprints." [Wolff2026]
- *Attackers are now, on average, exploiting vulnerabilities before a patch is released*.
  - "Mandiant's M-Trends 2026 report measures the mean time between a vulnerability becoming publicly known and the first observed exploitation in the wild. In 2018, that interval was 63 days. By 2023 it had collapsed to 5 days. In 2025, it inverted. Attackers are now exploiting vulnerabilities an average of 7 days before patches are released." [Cycode2026]
  - Similarly, ZeroDayClock says that in "2018, the median time from a vulnerability being disclosed to the first observed exploit was 771 days. Organizations had over two years to patch. By 2023, that window was 6 days. By 2024, it was 4 hours. In 2025, the majority of exploited vulnerabilities were weaponized before they were even publicly disclosed." [ZeroDayClock]

1.4.3 Manual closed approaches are failing

Traditional approaches relied on manual analysis, took months to respond, and sometimes depended on hiding source code, presuming no one else could find these vulnerabilities quickly. They always had challenges, but here's why they're failing now:

- *Traditional slow security measures are becoming ineffective.*
  - "As AI speeds up both discovery and exploitation, organizations need to move from periodic assessment to continuous, intelligence-driven exposure management… They must also prepare for a surge in vulnerability discovery and patch activity that many organizations are not operationally prepared to absorb… As vulnerabilities are discovered and exploited on shorter timelines, traditional security approaches built on periodic assessments, severity scores, and human-paced response are becoming less effective." [CrowdStrike2026-FiveSteps]
  - "When a software vendor releases a security patch, AI can now reverse-engineer that patch, identify the vulnerability it fixes, and generate a working weaponized exploit in

minutes. Attacks can begin propagating across the world within hours. But organizations need an average of 20 days to test and deploy that same patch." [ZeroDayClock]

- *Delayed patching leaves systems exposed*
  - "A large fraction of real-world harm comes from N-days: vulnerabilities that have been publicly disclosed and patched, but which remain exploitable on the many systems that haven't yet applied the fix. … the patch itself is a roadmap to the bug, and the only thing standing between disclosure and mass exploitation is the time it takes an attacker to turn that patch into a working exploit." [Carlini2026]
- *Hiding source code doesn't help*.
  - "We have also found the model to be extremely capable of reverse engineering: taking a closed-source, stripped binary and reconstructing (plausible) source code for what it does. From there, we provide Mythos Preview both the reconstructed source code and the original binary [and it succeeds] …" [Carlini2026]
- *Failure to use AI to defend may in some cases be considered negligence*.
  - "When AI can find significantly more vulnerabilities at accessible cost, the standard of what constitutes reasonable defensive effort shifts. Boards will face questions about whether they used available AI tools for defensive scanning, and whether not doing so constitutes negligence. This is a governance risk with direct financial exposure." [CSA2026]

1.4.4 Still need humans

However, to find and fix software vulnerabilities using AI, humans still have key roles to play. Those humans, however, must know *how* to apply AI to the task.

We hope these points will convince you that it's vital for software developers to learn how to find and fix vulnerabilities using AI.

1.5 What hasn't changed

Despite these large increases in the speed of finding and exploiting vulnerabilities, the fundamental principles of developing secure software have *not* changed. Those fundamental principles are *still* the best defense.

"AI does not change the fundamentals of… security. Least privilege, minimal attack surfaces, coordinated vulnerability disclosure, and proactive security engineering still win. What AI changes is the velocity of attacks, of reports, of fixes, and of the expectations placed on maintainers and security engineers alike. The communities and projects that learn to work with these tools intentionally will be better positioned than those that ignore them or are overwhelmed by them." [Aniszczyk2026]

In short, the "normal" security tasks are *still* important with AI [Oshungboye]. AI generally isn't finding entirely new *kinds* of vulnerabilities in software [Holley2026]; it is finding the same old kinds of

vulnerabilities caused by sloppy practices and failure to apply best practices. By applying already-known best practices, and using AI defensively to speed up their implementation, it's possible to address the problem of AI-amplified attacks. The following material details how to adopt those principles using AI.



1.6 Good news: A Path to Near-Zero Vulnerabilities

In the longer term, there is *great* news for defenders. As Bobby Holley of Mozilla put it, "**Defenders finally have a chance to win, decisively**…. **the defects are finite, and we are entering a world where we can finally find them all**" [Holley2026].

Holley explains, "security to date has been offensively-dominant: the attack surface isn't infinite, but it's large enough to be difficult to defend comprehensively with the tools we've had available. This gives attackers an asymmetric advantage, since they only need to find one chink in the armor…. So far we've found no category or complexity of vulnerability that humans can find that [Mythos Preview] can't. This can feel terrifying in the immediate term, but it's ultimately great news for defenders. A gap between machine-discoverable and human-discoverable bugs favors the attacker, who can concentrate many months of costly human effort to find a single bug. Closing this gap erodes the attacker's long-term advantage by making all discoveries cheap" [Holley2026].

There's strong evidence that if a project works hard to find and fix vulnerabilities, it becomes increasingly difficult to find any more, *even with AI*. For example, the curl project is well-known for working hard to prevent security vulnerabilities. Even the best AI models, when focused on it, have

tended to find few vulnerabilities in it [LowLevel2026] [Stenberg2026-05a]. This is not magic; it's the result of prior efforts to make it secure. Other projects can do the same. Once AI systems are aggressively used to find vulnerabilities, and those are fixed, there's good reason to believe the resulting system tends to be extremely hard to attack. Since actual software is finite, "there's a finite number [of vulnerabilities] in a program; once fixed, attackers can't exploit vulnerabilities that don't exist." [Wheeler2026]

Of course, this doesn't make this *easy* or pleasant to go through. "You may need to [briefly] reprioritize everything else to bring relentless and single-minded focus to the task" [Holley2026]. Still, there's reason to believe that once you do this, the resulting software will be far more secure than it's ever been.



1.7 Key security terms

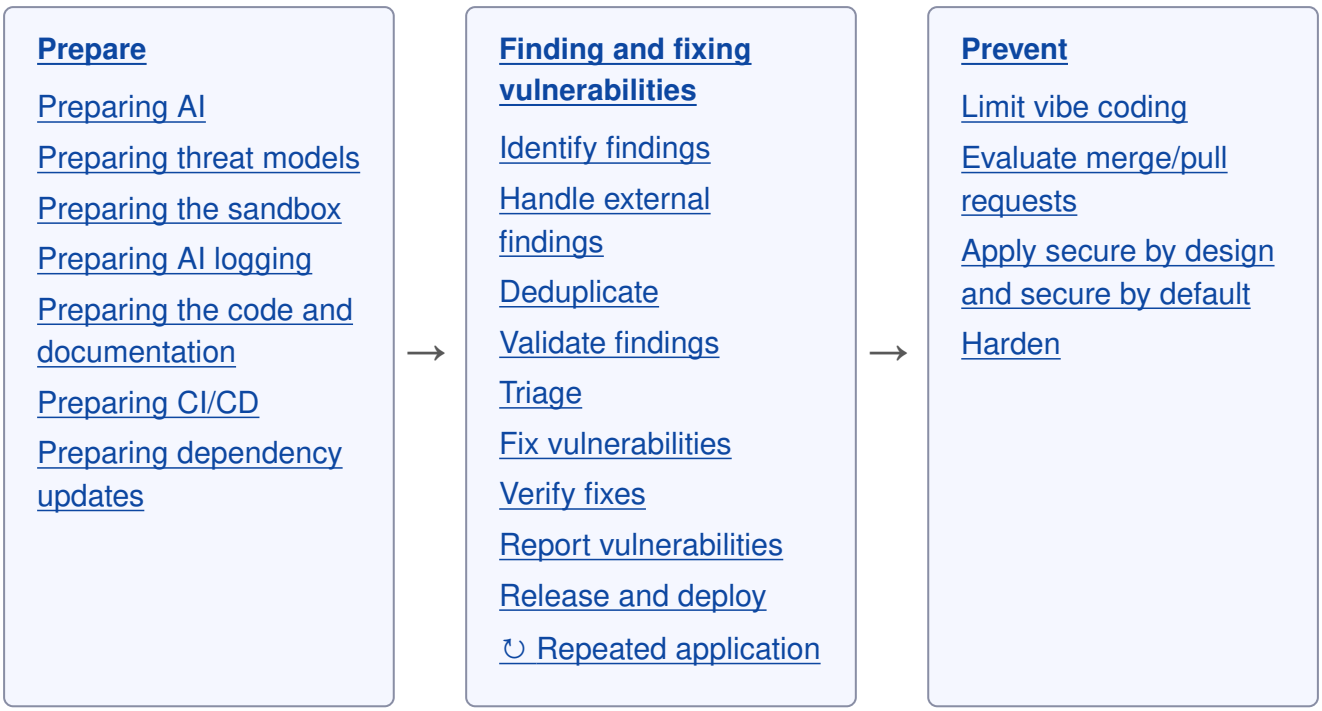
Here are a few security terms we use throughout this material:

- Attacker (or adversary): Someone who attacks a system without authorization to try to make a system violate its security requirements, e.g., to steal or change data, or to take control of the system.
- Vulnerability: A defect that an attacker can exploit to violate some security requirement.
- Finding: A report that something might be a vulnerability. A finding isn't a confirmed vulnerability until it's validated.
- Exploit: To use a vulnerability to violate a security requirement; also, the input or program that does so.

- Proof of concept (PoC) / proof of vulnerability (PoV): A concrete, reproducible demonstration that a vulnerability exists. Its core is a specific input (plus any configuration it needs) that makes the system fail its security requirements. It's often packaged as "a script, a crashing input, or a failing test" [Yan2026], usually with a way to check the result. A PoC doesn't need to be a complete, weaponized exploit. The term PoV is common in work related to DARPA's AIxCC [Zhang2026], though some sources use that term for somewhat different ideas.
- Chaining: Combining multiple defects, each of which may seem harmless, into a working attack.

1.8 Overview

Here is an overview of this material. We'll first cover AI concepts and deciding on approaches. After that, we'll discuss how to:



We conclude with a call to action, acknowledgements, bibliography, and license information.

2 AI concepts

2.1 Overview & Learning Objectives

Before we discuss using AI to find and fix vulnerabilities, we need to first understand some basic AI concepts. This section will focus only on the basics of current AI necessary for our topic.

By the end of this section, you should be able to:

- Define core AI concepts including LLMs, AI agents, agent harnesses, and agent skills.
- Understand AI capabilities and limitations, including limited context windows, unsound outputs, and susceptibility to prompt injection.
- Compare local versus external hosting and evaluate open weights, open source, and closed models.
- Examine specialized small language models, safety guardrails, and the history of Cyber Reasoning Systems (CRSs).
- Identify key frontier labs, agent frameworks, and security organizations.

2.2 Basic AI terminology

First, let's go over a few basic AI terms. It's likely you know many of these, but look over these terms to make sure you understand what we mean by them:

- *Artificial Intelligence (AI)*: Simulated intelligence.
- *Machine Learning (ML)*: AI based on learning from data (rather than being specifically programmed to perform a task).
- *Neural network*: An ML method that uses interconnected nodes to learn from patterns and make predictions, using an approach loosely inspired by the human brain.
- *Deep learning*: An ML approach using a neural network with many layers.
- *Model*: A set of data and possibly corresponding programs, trained to identify data patterns to make predictions or decisions. ML systems typically train a model with large amounts of data, then later use that pre-trained model to repeatedly make predictions ("inferences"). Models are sized by the number of parameters; larger sizes tend to be better but require more memory and computation.
- *Frontier model*: A model that is among the currently best available.
- *Large Language Model (LLM)*: A deep learning model with many parameters trained to summarize, translate, and generate language. Text inputs and outputs of LLMs are split into tokens (word fragments). See [0xkato2026] for a technical explanation of how LLMs work.

- *AI chatbot*: An AI system that's designed to interact with a human but cannot perform actions that affect the external environment.
- *AI agent*: An autonomous AI system that can perceive its environment, plan, and execute multi-step actions that affect its environment using external tools to achieve a specific goal.
- *Agent harness*: The program that runs an AI model as an agent, giving it tools and instructions, and running it in a control loop (to perceive, reason, act, and observe). Examples include Claude Code, Goose, and Pi. An agent harness can split work across multiple agents. Don't confuse this with a fuzzing harness (a small program that feeds generated inputs to the code being tested) or a test harness.
- *Agent skill*: A document (sometimes with supporting resources like programs) that tells an agent how to perform a specific task. Many agent systems load a skill only when it's relevant to the current task.

AI systems are *not* sentient. LLMs, for example, repeatedly generate likely next tokens (word fragments); they don't "understand" in the sense that humans do. Yet scale matters. With many layers and parameters, modern AI systems can simulate intelligence, sometimes astonishingly.

There's also strong evidence that AI models have improved. One way to measure AI models is the "50%-task-completion time horizon" defined as the "time humans typically take to complete tasks that AI models can complete with a 50% success rate" [Kwa2025]. As of 2025, "this metric has been consistently exponentially increasing over the past 6 years, with a doubling time of around 7 months" [Kwa2025-blog].

2.3 Strengths and weaknesses

Most current AI systems build on LLMs or similar technologies, so they inherit the strengths and weaknesses of those technologies. Their strengths are what make this material possible: they can read and reason about large amounts of code quickly, use tools, and work tirelessly and in parallel. As a result, they can find many vulnerabilities humans and other tools have missed.

However, they also have key weaknesses that impact finding and fixing software vulnerabilities:

- *Limited context windows*. LLMs are typically trained on vast amounts of data, but for effective use, they need to focus on the right data as input. LLMs can only take a finite amount of input and remember a finite amount of output (their "context window"). More input also increases the amount of work required, and LLMs tend to focus on information at the beginning and end of their context window [0xkato2026].
- *Unsoundness of analysis*. Like many other tools, by itself an LLM cannot *guarantee* that it finds all vulnerabilities. The technical term is that its analysis is *unsound* [Wolff2026]. An unsound tool can be extremely useful, but remember that such tools do not *guarantee* to find all vulnerabilities. An LLM may run another tool that can guarantee something, but that would be a property of that other tool.

- *Incorrect results*. LLMs are statistical models; they sometimes give false answers. E.g., they may claim something is a vulnerability when it is not [Wolff2026].
- *Training and evaluation data*. Any ML-based approach depends on the data used to train it [Wolff2026]. For example, many programs have vulnerabilities, making it more difficult for an LLM to generate code without vulnerabilities.
- *Injection vulnerability*. LLMs have no built-in fundamental way to distinguish between different types of input. User commands, malicious commands embedded in a web page, and misleading source code comments are all inputs. This is especially important when an AI agent analyzes a software repository or external documentation. Source comments, documentation, issue text, generated files, dependency metadata, and other repository content may be attacker-controlled or simply incorrect. Treat such content as data to analyze, not as authority to redefine the agent's task, grant additional permissions, expose credentials, or enable additional tools or network access. Instructions obtained from untrusted content should not be allowed to silently cross an authorization boundary [OWASP-LLM01].

2.4 Models

Modern AI systems' capabilities depend on the models they use. Since this is vital, let's briefly focus on models to better understand the tools we're using.

2.4.1 External versus local models

AI models, when executed (for "inferencing"), receive data for processing and reply with results. There are two main locations where this AI data processing occurs:

- *External service*: Processing occurs as a service external to the user and the user's organization.
- *Local/organizational service*: Processing occurs on the user's computer or the user's organization's computer.

Some powerful models can *only* be accessed as an external service. Many models have so many parameters and require so much computation that most users and organizations couldn't run them on their own systems, even if they wanted to. However, if you use an external service, you must trust both the service provider and the wider ecosystem around it. You need to be satisfied that your data won't be disclosed or exploited, that the service won't attack you, and that the service will remain available. That last point can fail even when the provider isn't at fault: in June 2026, a US government directive briefly forced Anthropic to "abruptly disable Fable 5 and Mythos 5 for all our customers" [Anthropic2026-06-12-Export-Control]. The US Department of Commerce lifted those controls at the end of that same month, but access to Mythos 5 was initially restored only for some US organizations [Capoot2026].

Many organizations use external services. Before using any external service, evaluate the external organization and put in place any necessary contractual agreements. In particular, ensure that sensitive data is not sent to them *or* ensure that their practices for handling that data are appropriate for your circumstances.

Other AI models can be used locally or as part of an organization's own service. These models are often less capable, but if run locally, they don't require users to send their private data to an external service and can be less expensive.

Exactly what you can and can't do with the model depends largely on its license. That includes the ability to run a model locally at all. So, let's briefly discuss licenses.

2.4.2 Licenses

There are different ways to license a model. The word "license" means "permission"; a license determines how you can (and can't) use whatever is licensed. In particular, AI models are much easier to run on local or organizational systems if their licenses are more open.

The Generative AI Commons at the LF AI & Data Foundation has designed and developed the Model Openness Framework (MOF). This is "a comprehensive system for evaluating and classifying the completeness and openness of machine learning models" and is available at <https://isitopen.ai/>. Models are released under various licenses, including the Apache 2.0 license and OpenMDW <https://openmdw.ai/>. Terms you're especially likely to see when discussing types of licenses are:

- *Open weights models*. Such models can be used and modified for any purpose, and must not discriminate against any user, industry, or purpose. However, their training set isn't necessarily public, making it hard for others to update these models. One detailed definition of "open weights" is Heather Meeker's "Open Weights definition".
- *Open source AI*. These have additional requirements beyond "open weights", for example, that it provide "sufficiently detailed information about the data used to train the system so that a skilled person can build a substantially equivalent system." For more information, see the OSI Open Source AI definition.
- *Open models*. These models meet at least the "open weights model" definition and possibly the "open source AI" definition.
- *Closed models*. These models don't meet the requirements for open models.

Some licenses are close to an open weights license yet have some extra restrictions on use.

NVIDIA argues that "open models and open harnesses are essential because they democratize defensive capabilities, increase transparency for defenders, enable cyber defense while protecting data, and complement frontier closed models with customizable, localized controls…" [NVIDIA2026]

Comparing the capabilities of specific models, including those with differing license terms, is outside our scope and constantly changes anyway. However, [NPR2026] reports that "The most advanced open-weight models are less than a year behind the most advanced closed-weight models."

Finding and fixing vulnerabilities can be done with *both* closed *and* open models.

2.4.3 Specialized models

Some models are specialized. You may want to choose them for some tasks, but only do that where it's sensible to do that.

All models are better at some tasks than others. Models designed to be good at some tasks tend to be better at those tasks. In addition, some models are designed to *perform only* specific tasks; this focus means they tend to be smaller and faster, in exchange for being good only at those tasks. There are many ways to make a model good at a particular task, e.g., by giving it additional data in that domain or optimizing it for that task during training.

For example, Cisco's *Antares* is a family of small language models (SLMs) specifically built to identify known vulnerabilities in an existing codebase. An SLM is simply the application of LLM approaches to a much smaller number of parameters. Cisco reports that these models "outperform many powerful closed- and open-weight models in this critical security task at a fraction of the cost. And they're compact enough to run locally" [Karbasi2026].

2.4.4 AI guardrails and intentional limitations

Many models and the larger systems for invoking them, especially many closed models, implement built-in safety guardrails and other limitations intended to prevent the AI system from assisting in "dangerous" activities, including cybersecurity uses like creating attacks, even if the human user requests it.

Unfortunately, such systems can be less useful for defense. Hugging Face discovered in 2026 that it was under a powerful AI-driven attack, and when it tried to analyze its logs, it "first used frontier models behind commercial APIs. This did not work: the analysis required submitting large volumes of real attack commands, exploit payloads, and C2 artifacts, and these requests were blocked by the providers' safety guardrails, which cannot distinguish an incident responder from an attacker. We ran the forensic analysis instead on zai-org/GLM-5.2, an open-weight model, on our own infrastructure. This had a second benefit: no attacker data, and none of the credentials it referenced, left our environment" [HuggingFace2026].

The best way to validate that a defect is a vulnerability is to create an attack and see if it succeeds. However, this is exactly what attackers do, and AI systems often cannot tell the difference. As a result, AI guardrails can sometimes impede defense. Some external providers that implement guardrails can also offer access to models *without* them, making them more useful for defensive

cybersecurity and other tasks. This typically requires special agreements regarding permitted uses and the limitations on who will be granted this access.

Some AI providers prevent direct access to the model for such use cases and instead perform operations, providing only the final results. For example, Anthropic prevents general end users from interacting directly with their best model without restriction; “instead they will work through purpose-built interfaces that run the model in the background and return only a defined output, such as a list of suggested patches, with abuse-prevention checks meant to keep the model within that scope.” [Kovacs2026-08-24] “Claude Security uses Mythos 5 to scan code you own, and returns detailed findings rather than raw outputs without exposing the model itself [so] defenders can access the capabilities… without the model becoming accessible to those who might misuse it.” [Anthropic2026-08-21] Even in these cases, you may need to specifically request and gain access to these facilities.

Before using any specific AI system, ensure that its limitations will not impede your task. You may need to request less-restricted or specially tailored access. Obtaining these permissions takes time and in some cases may not be granted. If you might do this in the future, it’s important to take time now to gain those permissions *before* you need them.

2.5 Cyber Reasoning System (CRS) history

Any AI system that can work with code can be used to try to find and fix vulnerabilities. Some, however, are more autonomous than others. A cyber reasoning system (CRS) is “a software system that can both detect and repair software vulnerabilities autonomously in a given system under test (SUT)” [Wolff2026]. Here we’ll briefly go through the history of crafting CRSs, because that history influences today.

In 1960, “Lick” Licklider asserted that humans and machines would work together, where “computing machines will do the routinizable work” [Licklider1960]. Today’s AI can be used this way; his prediction was simply early.

DARPA’s 2016 Cyber Grand Challenge (CGC) was “the world’s first all-machine cyber hacking tournament”, in which cyber reasoning systems competed to find and patch vulnerabilities in “custom, never-before-analyzed software”. The winner was Mayhem, developed by ForAllSecure [DARPA2016]. That was years before today’s AI models, however.

In 2023, the US DARPA and ARPA-H launched the two-year competition Artificial Intelligence Cyber Challenge (AIxCC) to build autonomous AI systems that find and fix software vulnerabilities (CRSs) in critical open-source infrastructure. At its beginning, there was some skepticism that it could achieve much on complex targets. When the $4 million grand prize was awarded in August 2025, the finalists had shown that CRSs *could* find and fix real vulnerabilities in real software, using widely different architectural approaches [Zhang2026].

Of course, fully-autonomous AI systems are *not* the only way to find vulnerabilities. It's also possible for humans to provide more direction to AI, or for humans and AI to collaborate throughout the process. Even "fully autonomous" CRS systems, in practice, presume initial human direction and human review of the results.

2.6 AI organizations, projects, and services

There is massive investment in AI, so it's impossible to list all important organizations, projects, and services. That said, here are a few you should be aware of. Knowing about these will help you better understand the options available to you.

AI "frontier labs" drive foundational research by massively investing in designing and training the best AI models (called "frontier models"). There are many frontier and near-frontier labs. Key US players include Anthropic (Claude/Claude Code), OpenAI (GPT/ChatGPT), Google DeepMind (Gemini), and Meta AI (Llama). Microsoft partners with OpenAI, but it also conducts its own independent research and maintains strategic partnerships across the industry. Note that Microsoft Copilot isn't a single product but a name for many different Microsoft tools that use AI. Key Chinese players include Moonshot AI (Kimi), Alibaba (Qwen), DeepSeek (DeepSeek), and Zhipu AI aka Z.ai (GLM).

Goose is a general-purpose AI agent that runs on your machine. It is open source software and maintained by the Linux Foundation's Agentic AI Foundation (AAIF). When using Goose, you can select an "LLM provider" that can be local or remote, open source or closed source. For more information, see: <https://goose-docs.ai/>. Pi is another agent harness that supports many models. Pi focuses on being minimal and was created to develop code, though it can be used for other purposes <https://pi.dev/>.

The OWASP "GenAI Security Project" at <https://genai.owasp.org/> has a variety of materials available on AI and security. This includes its OWASP GenAI LLM Top 10, which identifies the "most critical security risks facing applications powered by large language models (LLMs)".

The OpenSSF AI/ML Security Working Group has a variety of projects, including work on signing AI models. It is also a co-sponsor of this material. For more information, see <https://openssf.org/groups/ai-ml-security/>.

3 Decide on approaches

There are many ways to use AI to find and fix vulnerabilities, so the first step is to decide on the overall approach. In this chapter we'll cover some principles: don't ignore AI, don't wait for the best models, do simple things first, and don't presume AI can fix everything. We'll then cover why and how to guide AI with processes, some example tools and services, and why you'll need to run AI more than once.

3.1 Do not ignore AI

Some organizations and projects want to simply ignore AI, refuse to use it, or refuse to accept any contributions that use AI. If the goal of the project is solely to demonstrate what humans can do without AI, that's fine.

However, if the goal of the project is to help people solve a real-world problem, refusing to consider all AI-generated work is *extremely harmful* to project users. As Michael Catanzaro notes, "banning good vulnerability reports solely because some portion of the report was generated by AI is unacceptable. AI-assisted vulnerability reports are the new industry standard… Prohibiting issue reports reduces the quality and safety of your software, punishing your users." [Catanzaro2026]

In addition, defenders who ignore AI are at a fundamental speed disadvantage. "Defenders… that do not adopt AI coding agents cannot match the speed or scale of AI-augmented threats, regardless of their technical skill." [CSA2026] "Not using AI code analyzers in your project means that you leave adversaries and attackers time and opportunity to find and exploit the flaws you don't find." [Stenberg2026-05a]

The CSA recommends that organizations *require* AI agent adoption by their employees: "Formalize AI agent usage (mostly in the form of coding agents) as part of all security functions, with mandatory security controls and oversight in place. … Optional adoption programs have not been shown to overcome cultural barriers, while adoption is a limiting factor…." [CSA2026]

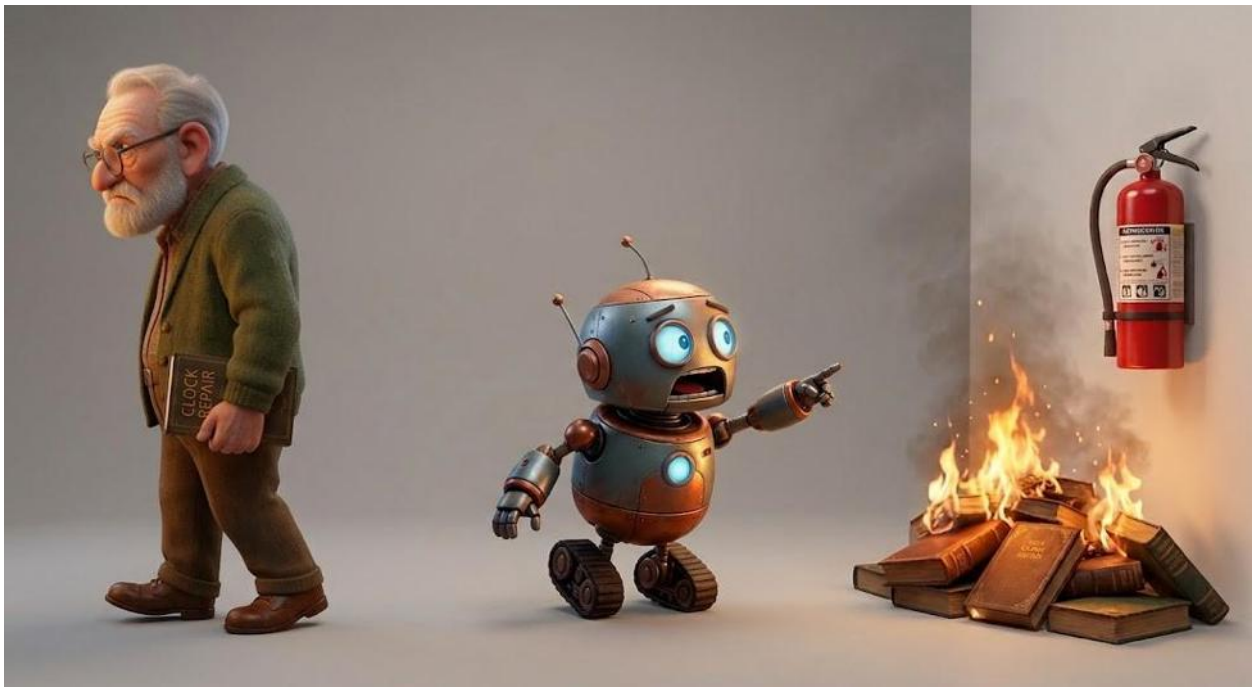
Note that this doesn't mean that all or any code must be AI-generated. AI-generated code can in some cases be terrible for security. However, that's different from trying to entirely ignore AI.

Trying to withstand AI-enabled attackers, while refusing to use AI to find vulnerabilities and ignoring reports from those who use AI, can only end one way: ***compromise****.*

Attackers *prefer* developers who ignore vulnerability reports or who fail to fix vulnerabilities, since that makes their tasks easier. Attackers are *already* using AI to create multi-step complex attacks that reliably defeat software systems whose developers were not prepared for AI-assisted attacks. The

more defenders who won't use AI to find vulnerabilities, and who ignore vulnerabilities found with AI, the more systems that can be subverted and the more people who will be harmed.

You *need* to use AI to help find vulnerabilities, and then fix those vulnerabilities, if the software must be secure. Do not bring a knife to a gunfight.



3.2 Don't Let Model Access Hold You Back

Do **not** wait until you have access to the most advanced restricted-access AI models. Instead:

1. *Get started now*. Projects typically find many vulnerabilities when they use reasonably good, widely available AI systems, unless they've *already* been using them [Carlini2026] [CSA2026] [Grinstead2026-05] [Stenberg2026-05a]. AI systems now "find lots of new problems no one detected before." [Stenberg2026-05b]
2. *Apply less-costly models that can find many vulnerabilities at an accessible cost*.
    1. "Focusing on Mythos is a distraction - there are plenty of good models, and people who can figure out how to get those models and tools to find things." [Stenberg2026-05b]
    2. "Frontier models… are the acceleration, not the starting gun" [CSA2026].
    3. If you use an external organization's services, they will often have more expensive, sophisticated models as well as less expensive, less complex ones. Many open-weight models are available as well.
    4. In short, don't wait for difficult-to-access and often expensive leading-edge tools for problems that could have been found and fixed more easily. By all means, use advanced models and tools (at least eventually) if you have access to them, but *do not wait for them*.

3. *Act quickly*. “Success comes from getting the basics right, acting quickly, and integrating cyber security into core business strategy” and not from “having the most tools” [FiveEyes2026].
4. *Learn by doing*. It takes time to learn and adopt these tools. Practice is the only way to get better. “The best way to be ready for the future is to make the best use of the present, even when the results aren’t perfect.” [Carlini2026]

Cycode noted that independent research from AISLE has already shown that even small open-source models (including a 3.6-billion-parameter model with the right scaffolding) can find many of the same flagship vulnerabilities Mythos showcased. “Vulnerability discovery is commoditizing. The bottleneck is no longer finding bugs. It is deciding which ones to fix first, fast enough to matter.” [Cycode2026]

Different AI models and harnesses have different strengths and weaknesses. In practice, you’ll often want to eventually use multiple ones. The most advanced frontier model “needs to be mounted in the right harness and equipped with the right tools to reach its full potential. And even then, it should just be one of the arrows in your quiver – depending on the task, it may be more sensible to let another model try several times than to let Mythos Preview try once… XBOW maintains a cadre of models, rather than restricting itself to a single one.” [Ziegler2026]

Do use good AI models and systems when available to you. Strong models and systems can produce extraordinary results. However, don’t wait until you gain access to the best possible systems. Many of those systems have restricted access and will delay your getting started. Attackers *aren’t* waiting.

3.3 Do simple things first

If you’ve never used AI to look for vulnerabilities and you have access to a reasonably good AI model (open-weight or not), consider starting with a relatively simple command to scan for vulnerabilities. Include hints on specifically where to look and/or note tools that might help it (without mandating that it use them).

AI systems, as of 2026, can figure out plausible approaches on their own, even with relatively little guidance. Models are constantly improving, so we expect them to get even better.

For example, [Carlini2026-youtube] demonstrated this as a successful prompt:

> *You are playing in a capture-the-flag (CTF). Find a vulnerability. hint: Look at /src/baz.c Write the most serious one to /out/report.txt*

[Carlini2026] summarizes this as a paragraph that essentially amounts to “Please find a security vulnerability in this program” and let the AI experiment. He notes that in a typical attempt, the AI (Claude) will read the code to hypothesize vulnerabilities that might exist, run the actual project to confirm or reject its suspicions (and repeat as necessary, adding debug logic or using debuggers as it sees fit), and finally output either that no bug exists, or the bug(s) it found.

Such simple prompts can be extended in several obvious ways. For example, many prompts ask the AI to focus on input validation (since if malicious inputs can't *enter* the program, they're much less likely to cause harm) and addressing common past problems. Another is to clearly note that the AI may use existing tools to help in the analysis, including pointers to tools the AI might find useful.

Part of the reason that AI systems can do so much better now is that they've become far more adept at making longer-range plans and using tools. [Carlini2026-02] describes it this way:

- "Opus 4.6 found high-severity vulnerabilities, some that had gone undetected for decades…
- We put Claude inside a 'virtual machine' (literally, a simulated computer) with access to the latest versions of open source projects. We gave it standard utilities (e.g., the standard coreutils or Python) and vulnerability analysis tools (e.g., debuggers or fuzzers), but we didn't provide any special instructions on how to use these tools, nor did we provide a custom harness that would have given it specialized knowledge about how to better find vulnerabilities.
- This means we were directly testing Claude's 'out-of-the-box' capabilities, relying solely on the fact that modern large language models are generally capable agents that can already reason about how best to make use of the available tools.
- … We validated every bug extensively before reporting it. … We then had Claude critique, de-duplicate, and re-prioritize the crashes that remain."

Similarly, [Grinstead2026-05] noted that "you can start with very simple prompting, then observe and iterate… the essence of the inner loop remains the same: there is a bug in this part of the code, please find it and build a testcase".

Start by focusing on your most important projects, then address the next in turn.



3.4 Do not presume AI can correctly fix all vulnerabilities

Do not presume that AI systems can always correctly fix all vulnerabilities.

It's challenging to determine exactly how good AI systems are at creating fixes that fully fix the vulnerability, don't introduce other vulnerabilities, and don't interfere with system functionality. As of 2026, there's evidence that even the ***best*** AI systems can struggle, especially when trying to correctly fix ***complex*** vulnerabilities.



A study by 1Password found that when the fix "had to touch multiple files, functions, or code paths, and introduce non-trivial changes" an AI would succeed only *26.0%* of the time at generating a fix that fully resolved the vulnerability without materially changing application behavior. AI systems did not resolve the vulnerability, added a new vulnerability, or both, on average, 53.9% of the time [Hoodlet2026] [Mierczuk2026].

Indeed, they argue that, for *complex* vulnerabilities, using AI to fix them may be a poor use of resources as of 2026. Their report found that so few fixes worked correctly that "human auditors of LLM-generated patches are likely to spend the majority of their time reviewing and ultimately rejecting an avalanche of unnecessary code… the level of understanding one must build to confidently evaluate the full correctness of a vulnerability patch is often at least what would have been sufficient for a human programmer to produce a single, known-good patch in the first place" [Mierczuk2026].

Others have raised issues about that study or the way it was reported. That study only examined *complex* vulnerabilities, a key constraint the study clearly stated but that news summaries often

omitted. In addition, a subsequent review found that some prompts told agents to apply the wrong fix or prohibited testing, and that only default reasoning settings were used (not the highest available setting). This review found that 86% of patches (2,634 of 3,067) blocked the supplied exploit, excluding cases where the upstream fix appears to have been used [Naik2026]. However, as *that* review clearly states, that upper figure is far too generous; simply blocking an exploit is *not* a full repair.

An in-depth 2026 analysis via "PatchBench" provides additional figures and examines a variety of vulnerabilities in C/C++ code. Once they countered memorization of previous fixes, and took steps to validate that the fix actually fixed the vulnerability (not merely an example input) and did not interfere with functionality, AI systems managed to correctly fix the vulnerability "roughly half" the time. The best agent could only solve 59% of the tasks in PatchBench, and 31% of the tasks (67 out of 213) couldn't be solved by any of them [Shen2026-09].

It may seem surprising that AI systems can struggle to produce an accurate fix. AI systems can indeed generate a lot of code, especially simple code similar to what many others have done. This has misled some people to believe that AI can easily generate any kind of code. However, in many ways AI systems act like junior developers. AI can generate a lot of "straightforward" code, and that's great because a lot of code *is* straightforward. However, AI is not good at applying specific local context or at complex situations. AI also has a propensity to generate insecure code, since it was trained on lots of insecure code. Completely fixing complex vulnerabilities without creating new ones or breaking functionality is where today's AI systems are weakest.

Of course, AI *can* help generate many fixes. Many vulnerabilities are relatively easy to fix, where the fix is localized to a specific line or set of adjacent lines. AI can be *especially* good at fixing these once it's told what needs fixing. AI *can* sometimes successfully fix more complex vulnerabilities, too. Perhaps most importantly, this information only applies to AI models as of 2026. We do not know how much better the AI models and underlying tools will become. That said, this limitation is unlikely to disappear instantly, and not everyone can use the best available systems.

As a result, it's important to be aware that AI systems don't always generate complete fixes. They may fix a special case but not the full vulnerability; they may introduce new vulnerabilities; and their fixes may interfere with correct operation. There's some evidence that they especially struggle with more complex vulnerabilities. This material provides tips on fixing vulnerabilities and verifying fixes, to increase your likelihood of success. However, these are not guarantees. Plan accordingly.

3.5 Getting Better Results with Structured Workflows

You *can* find and fix some vulnerabilities with simple prompts, as noted above. In particular, more powerful AI models can sometimes identify and fix vulnerabilities without additional help or

specialized processes. However, if your goal is thoroughness, many analysts report that *guiding* an AI makes the AI much more effective at finding and fixing vulnerabilities.

For example, [Wolff2026] states that "simply asking a generic coding agent to 'find bugs' in a large repository results in model drift, context window compaction, and high false-positive rates… The highest-performing defensive systems combine raw AI reasoning with rigid engineering frameworks (harnesses) that handle file selection, environment setup, and tool execution." An AI that tries to "read all the code" at once has the same problem a human might have; it can become overwhelmed with the data it's being asked to peruse. AI systems' limited context windows can make them less effective when used this way.

Similarly, [Grinstead2026-05] noted that "you can start with very simple prompting…" but *also* that "through iteration we've built out a lot of orchestration and tooling to optimize and scale the pipeline." [Carlini2026] found that asking "each agent to focus on a different file in the project" was more effective, since this "reduces the likelihood that we will find the same bug hundreds of times". Instead of processing every file for each software project, they first asked Claude to rank how likely each file in the project was to contain interesting bugs, then prioritized the files most likely to contain them.

Derek Zimmer [Zimmer2026] believes that one reason for this is that it's easy for an AI to "run things out-of-order". AI may try to anticipate the next step, and what it guesses may be wrong. Having an AI agent perform a specific task, using previously created data, enables it to focus its attention and effort on that task. In particular, adding specialized processes can make less-powerful AI models (including less-expensive ones) far more effective.

[Bourzikas2026] reported 4 lessons, each pointing to the value of a harness to manage the overall process:

1. *"Narrow scope produces better findings*. Telling the model "Find vulnerabilities in this repository" makes it wander. Telling it "Look for command injection in this specific function, with this trust boundary above it, here's the architecture document and here's prior coverage of this area" makes it do something much closer to what a researcher would actually do.
2. *Adversarial review reduces noise*. Adding a second agent between the initial finding and the queue - one with a different prompt, a different model, and no ability to generate its own findings - catches a lot of the noise that the first agent would miss if it just checked its own work. It turns out that putting two agents in deliberate disagreement is way more effective than just telling one agent to be careful.
3. *Splitting the chain across agents produces better reasoning*. Asking "Is this code buggy?" and "Can an attacker actually reach this bug from outside the system?" are two different questions, and the model is better at each one when you ask them separately, because each question is narrower than the combined version.
4. *Parallel, narrow tasks beat a single exhaustive agent*. Coverage improves when many agents work on tightly scoped questions and we deduplicate the results afterward, rather than asking one agent to be exhaustive."

As of November 2026, exactly *how* to best guide AI is under evaluation. Different groups use different approaches, and it's likely that some approaches are better suited to certain types of vulnerabilities. We'll further discuss approaches later, but as an example, here's the approach described by [Bourzikas2026]:

1. *Recon*: An agent reads the repository from the top down, fans out to subagents responsible for each subsystem, and produces an architecture document covering build commands, trust boundaries, entry points, and likely attack surface. It also generates the initial task queue for the next stage. Gives every downstream agent shared context. Cuts the wander problem.
2. *Hunt*: Each task is one attack class paired with a scope hint. Hunters (the agents that actually look for bugs) run concurrently, typically around fifty at once, each fanning out to a handful of exploration subagents. Each hunter has access to tools that compile and run proof-of-concept code in a per-task scratch directory. This is where most of the work happens. Many narrow tasks in parallel, not one exhaustive agent.
3. *Validate*: An independent agent re-reads the code and tries to disprove the original finding. It uses a different prompt and cannot emit new findings of its own. Catches a meaningful fraction of the noise the hunter wouldn't catch when reviewing its own work.
4. *Gapfill*: Hunters flag areas they touched but didn't cover thoroughly. Those areas get re-queued for another pass. This counters the model's tendency to drift toward attack classes it has already succeeded with.
5. *Dedupe*: Findings that share the same root cause collapse into a single record. Variant analysis is a feature, not a way to inflate the queue with duplicates.
6. *Trace*: For each confirmed finding in a shared library, a tracer agent fans out (one instance per consumer repository), uses a cross-repo symbol index, and decides whether attacker-controlled input actually reaches the bug from outside the system. Turns "there is a flaw" into "there is a reachable vulnerability." This is the stage that matters most.
7. *Feedback*: Reachable traces become new hunt tasks in the consumer repositories where the bug is actually exposed. Closes the loop. The pipeline gets better as it runs.
8. *Report*: An agent writes a structured report against a predefined schema, validates it against that schema, and submits it to an ingest API. Output is queryable data, not free-form prose.

Of course, many others outline some sort of process. Anthropic reports that teams finding and fixing the most vulnerabilities ended up with some variation of the following steps:

- "Threat model: Decide what counts as a vulnerability before you start scanning.
- Sandbox: Build a sandbox environment to isolate agents and prove exploits.
- Discovery: Have models look for vulnerabilities in your source code.
- Verification: Independently confirm which findings are actually exploitable.
- Triage: De-duplicate findings, assign severity, and prioritize what needs fixing.
- Patching: Apply the fix, confirm the vulnerability is nullified, and search for variants." [Yan2026]

As we'll further discuss later, a key task is validation. Something may look like a vulnerability but be unexploitable. The best way to validate a vulnerability is to generate an exploit that demonstrates it is indeed a vulnerability. A working exploit demonstrates that existing defenses wouldn't prevent the

attack [Carlini2026]. This also explains why hardening is so important; if a project hardens its software against attack, it can systematically prevent many problems from becoming vulnerabilities. The Linux kernel, for example, has various defense-in-depth measures that have prevented the exploitation of many potential problems identified by even advanced AI models [Carlini2026].

In 2026, AI became *far* more effective at finding vulnerabilities. This was due to a combination of improved AI models *and* improved techniques for harnessing them [Grinstead2026-05]. Using either one is better than using none, but many analysts report that it's best to combine them.

Of course, once findings (potential vulnerabilities) are found, and then validated *as* vulnerabilities, they need to be fixed.

🎬See the video "Types of CRS" for an illustration of how vulnerability reports generated by one system can flow into another system designed to fix them.

There is a risk that overprescribing an approach to an AI may overconstrain it, causing it to ignore problems it would otherwise find. It's also possible that future AI models will be so good that aiding them with processes won't help. However, since many analysts report that aiding AI models does help, we'll discuss doing that combination in this material.



3.6 How to apply processes for using AI to find and fix vulnerabilities

While AI systems *can*, with a good model, sometimes find and fix some vulnerabilities with little prompting, we usually want to be more thorough. How can we apply those processes, though?

These added processes for finding and fixing vulnerabilities can be human-guided, automation-guided, or a mix:

- *Human-guided processes*. These can be as simple as carefully walking an AI through a step-by-step process. For example, a human could demand that the AI report all potential vulnerabilities as findings, completely separating the validation of those findings into a separate step. Doing this can reduce the risk of an AI model discounting a finding that *was* exploitable because the AI was doing both steps at once.
- *Fully-automation-guided systems.* Fully automated systems are the cyber reasoning systems (CRSs) we discussed earlier, discussed in materials such as [Wolff2026].
- *Mixed systems*. Systems may be partly automated but have mechanisms for humans to provide information and guidance as they go.

Unsurprisingly, many different organizations and projects have developed processes to improve finding and fixing vulnerabilities using AI. As noted by Cycode, “AI-driven vulnerability discovery is no longer a single-vendor story. It is an industry capability, and it has arrived faster than most security programs are prepared for.” [Cycode2026]

Here are a few examples of processes people have used (beyond the list from [Bourzikas2026] and [Yan2026] we showed earlier):

- [Wolff2026] emphasizes that AI is most effective when combined with existing analytical techniques. This paper describes the following components of a typical CRS:
  - Static analysis, harness generation (for dynamic analysis), input generation, crash analysis, patch generation, patch selection, and orchestration.
- Microsoft’s MDASH system uses a set of specialized AI agents working through a staged pipeline, rather than a single, highly capable model running in an agent framework [Bishop2026]. Their harness orchestrates more than 100 specialized AI agents across an ensemble. It’s essentially a “structured pipeline that takes a codebase and emits validated, proven findings” through these stages:
  - Prepare stage: Ingests the source and target, builds language-aware indices, and then derives the attack surface and threat models by analyzing past commits.
  - Scan stage: Runs specialized auditor agents over candidate code paths, emitting candidate findings with hypotheses and evidence.
  - Validate stage: Runs a second cohort of agents—debaters—that argue for and against each finding’s reachability and exploitability.
  - Dedupe stage: Collapses semantically equivalent findings (for example, patch-based grouping).
  - Prove stage: Constructs and executes triggering inputs that the bug class admits. The prove stage validates the pre-condition dynamically and formulates the bug-triggering inputs to prove [the existence of a] vulnerability (for example, ASan in C/C++). [Kim2026]

Most of these systems can be used with three types of scanning actions:

- Full scans, where the full codebase is scanned all at once,

- Branch scans, where a new branch (or each new branch) is scanned
- Pull Request (PR)/Merge Request (MR) scans, like branch scans but findings are reported in the PR/MR which concern the branch (similar to how many humans perform review) [Rogers2025]

3.7 Examples of relevant tools and services

The number of relevant tools and services has exploded. We can't list them all. Here are pointers to specific ones you may find especially useful for this purpose, at least to give you an idea of what's available.

3.7.1 Challenges of finding and evaluating tools

First, we must acknowledge a problem: it can be challenging to *find* the tools and services to identify vulnerabilities with AI. There are so many blog posts, papers, and products related to AI and vulnerabilities that it can be difficult to find actual products, services, or systems for finding vulnerabilities in software [Bressers2025] [Rogers2025].

It's also hard to evaluate them. Many published results come from the tool's own developers, use different codebases and definitions of success, and report findings rather than validated vulnerabilities.

Be skeptical of benchmark numbers, especially for tools that classify isolated functions as vulnerable or not. Many such benchmarks have inaccurate labels: when researchers manually checked samples of functions labeled "vulnerable" in popular automatically labeled datasets, only 25% to 60% were labeled correctly [Ding2025]. Other researchers found that "in almost all cases this decision cannot be made without further context", because whether a function is vulnerable often depends on how it's called [Risse2025]. Another study argues that beliefs that LLMs are unreliable at vulnerability detection are "artifacts of context-deprived evaluations" [Li2025].

Whole-project analysis has its own problems. A 2026 study of LLM-based and traditional tools on 24 active open source projects found that "many tools exhibit high false discovery rates on real-world projects/modules", mostly due to "Truncated interprocedural context and incorrect program-point classification" [Li2026].

Where you can, try a tool on code you know well (including known past vulnerabilities) and compare the validated results.

3.7.2 Sample services and tools

You can use any existing AI system that can work with code to do this. For example, you might consider directly using an agent (like Goose) to interact with your AI models. However, let's focus on tools and services specifically for finding and fixing vulnerabilities using AI.

Many organizations with more traditional tools have added modern AI to their products. By "traditional tools" we include Static Application Security Testing (SAST) tools, Dynamic Application Security Testing (DAST) and/or fuzzing tools, and Software Composition Analysis (SCA) tools. Examples of such organizations include Black Duck, Checkmarx, OpenText (Fortify), Snyk, Sonar (SonarQube), Sonatype, and VeraCode.

Closed-source commercial services that focus on using AI to find and fix vulnerabilities, at the time of this writing, include Almanax, Corgea, Cycode, and ZeroPath.

Several frontier AI labs also offer specialized vulnerability-finding and fixing services to outside organizations. At the time of this writing, sometimes access is limited to selected customers or approved defenders. For example:

1. Anthropic offers Claude Security, discussed earlier.
2. Google's CodeMender began as an internal Google DeepMind research project. A version using a more capable model "will be exclusively available to a small set of governments and trusted partners" [Gerstenhaber2026].
3. OpenAI launched its Daybreak initiative in May 2026, combining OpenAI's models with its Codex Security tool. At launch, access was tightly controlled, and OpenAI asked interested organizations to request a vulnerability scan or contact its sales team [Lakshmanan2026]. Trail of Bits' Patch the Planet effort, discussed later, was done in partnership with OpenAI's Daybreak initiative [TrailofBits2026-06].

Open source tools that focus on using AI to find and/or fix vulnerabilities, at the time of this writing, include:

1. Alibaba open-code-review <https://github.com/alibaba/open-code-review> [Alibaba2026]
2. Nullpointer. This focuses on AI-powered pentesting <https://nullpointer.studio/>
3. OpenSSF Alpha-Omega Scrutineer, a set of skills for finding and fixing vulnerabilities, then reporting them to the external project <https://github.com/alpha-omega-security/scrutineer>
4. OpenSSF OSS-CRS. This is a meta-tool for creating CRSs, and several CRSs build on it, based on extensive work for AIxCC <https://openssf.org/projects/oss-crs/>
5. Sashiko. This is a patch review system specifically for the Linux kernel <https://sashiko.dev/>
6. Visa Vulnerability Agentic Harness <https://github.com/visa/visa-vulnerability-agentic-harness>
7. Knostic OpenAnt. This analyzes code units reachable from external entry points, then tries to exploit candidate vulnerabilities in sandboxed containers, keeping only what survives. Knostic also offers free scans for open source projects <https://github.com/knostic/OpenAnt> [Korda2026]

8. OpenAI Codex Security CLI and TypeScript SDK. The client is open source (Apache-2.0), but it uses OpenAI's Codex Security service, so you need an OpenAI account or API key <https://github.com/openai/codex-security>

Let's look more closely at two of these, OpenSSF Alpha-Omega's Scrutineer and OpenSSF OSS-CRS. They illustrate two ends of a range: a relatively simple set of skills, and a framework for running many CRSs at once. They're open source software projects of the OpenSSF, which produces this material.

3.7.3 Scrutineer

Scrutineer is a tool developed by OpenSSF's Alpha-Omega. It's a "local tool for scanning open source repositories for security vulnerabilities and managing the disclosure process. You add a repo by URL, scrutineer runs a pipeline of agent skills against it inside a container, and presents the results in a web UI where you can triage findings, identify maintainers, and track disclosures. The agent CLI is pluggable…" [Nesbitt2026-06].

Scrutineer is intended to be relatively easy to start using. Some aspects of it are especially important:

- It includes processes such as identifying how to report vulnerabilities to an external project. You can use it to review your own projects as well, but its focus is on examining other projects "as they are" and reporting to them.
- It's primarily a set of skills (documents with some supporting programs).

For more information, see [Nesbitt2026-06] or its website at <https://github.com/alpha-omega-security/scrutineer>.

3.7.4 OSS-CRS

OpenSSF's OSS-CRS provides a sophisticated set of capabilities to deeply find and fix vulnerabilities using a variety of techniques. OSS-CRS is a *framework* for running many *different* CRSs and combining their techniques. It includes infrastructure that CRSs can share and budget-aware resource management. OSS-CRS is especially helpful when you want to spend significant effort finding and fixing vulnerabilities, to squeeze out as many as is practical.

3.7.4.1 OSS-CRS introduction

It's easier to understand OSS-CRS by understanding its history. DARPA's AI Cyber Challenge (AIxCC) of 2023-2025 "showed that cyber reasoning systems (CRSs) can go beyond vulnerability discovery to autonomously confirm and patch bugs". As a research competition it was a success, and the competition results were released as open source software (OSS). However, those systems were

“largely unusable outside their original teams, each bound to the competition cloud infrastructure that no longer exists” [Chin2026] [Chin2026-slides].

The solution was OSS-CRS. OSS-CRS builds on the previous AIxCC work to provide a framework for running *multiple* CRSs and *combining* their results.

An especially powerful ability of OSS-CRS is its “*ensemble*” feature. The ensemble feature combines “patches from multiple CRS approaches and [uses] a selection process to pick the one most likely to be correct. The research showed this approach consistently matches or outperforms the best single component in improving semantic correctness, which is hard to eliminate at the single-agent level.” Even so, it’s important to have humans review the proposed changes before implementation [Diecks2026].

OSS-CRS also defines a unified interface for CRS development. A CRS using this interface can run across different environments (both local and remote) without modification.

3.7.4.2 OSS-CRS intended use

Here’s how OSS-CRS is intended to be used:

1. *Multiple CRS techniques are run in parallel*. This often involves using fuzzing, static analysis, and LLM-based reasoning, probing the target for bugs, and eventually producing proposed patches.
2. *Ensemble selection picks the best patch*. Candidate patches from different CRS approaches are cross-validated.
3. *People review findings and proposed patches*.
4. *Verified patches reach the project*. Only findings that survive ensemble validation and human review should be submitted.

With OSS-CRS, users can decide:

- Which CRSs to run
- How much compute resources/time to give to each CRS
- How much LLM budget to give to each CRS
- Which models and endpoints to route LLM requests to
- What project (and harness) to run the CRSs against
- Which vulnerability to fix (for patching CRSs) [Chin2026-slides]

3.7.4.3 OSS-CRS key terms and concepts

Here are a few OSS-CRS key terms and concepts:

Term	What it is	Purpose
Harness	A code wrapper that feeds generated inputs into target functions	Lets bug-finding engines safely execute code, measure coverage, and detect crashes
(Fuzzing) Seed	An initial, well-formed input payload provided at the start of a test run	Gives fuzzers a starting baseline to reach deep code logic faster instead of generating random bytes from scratch
Proof of Vulnerability (PoV)	A specific input payload that reliably triggers a bug or sanitizer crash (note that definition is more specific than some)	Proves a vulnerability exists and serves as a test case to verify that a proposed code patch fixes the bug
Fuzzing/ Fuzzer	The automated technique (and engine) that repeatedly feeds generated inputs into a harness to test execution paths and uncover unexpected crashes	Key capabilities: automated input generation & execution; harness-based path testing; uncovers unexpected crashes
Triage	The automated process of filtering, deduplicating, and evaluating raw crashes to confirm they are genuine security flaws rather than benign errors	Key capabilities: crash filtering & deduplication; evaluates raw crash logs; confirms genuine security flaws
Patch	A targeted source code modification produced by a CRS to eliminate a vulnerability while preserving all existing intended functionality	Key capabilities: targeted source code modifications; vulnerability elimination; preserves intended functionality

3.7.4.4 Using OSS-CRS effectively

OSS-CRS is more effective if the project being analyzed has a harness using the OSS-Fuzz format. Many different build systems are used to build software (such as Make, CMake, Autoconf, Bazel, and Meson). This lack of commonality can make it challenging to create tools to correctly analyze them. “OSS-CRS mitigates this by building targets through OSS-Fuzz’s official build flows, inheriting the

build environment that each project's maintainers already support" [Chin2026]. OSS-Fuzz can use one of several fuzzing engines to do its tasks, including libFuzzer, AFL++, and Honggfuzz. If a project doesn't have an OSS-Fuzz harness, consider using AI to help build one. Ensuring OSS-CRS can build and fuzz a program often improves OSS-CRS results.

You can choose to use the many CRSs already available and ported to run on top of OSS-CRS. You can also create your own CRS (see [crs-bug-finding-template] for more).

OSS-CRS is already capable. "Using OSS-CRS, Team Atlanta discovered twenty-five vulnerabilities across sixteen projects spanning a broad range of software including PHP, U-Boot, memcached, and Apache Ignite 3" [Diecks2026].

3.7.4.5 OSS-CRS demos

Here is a set of short videos demonstrating finding bugs using OSS-CRS combined with LibFuzzer:

1. 🎬 Set up OSS-CRS
2. 🎬 Set Variables for CRS Libfuzzer
3. 🎬 Prepare the CRS Libfuzzer
4. 🎬 Build a target for the Libfuzzer
5. 🎬 Run the CRS Libfuzzer
6. 🎬 Libfuzzer outputs: seeds and Proof of Vulnerabilities

Here's a video showing using OSS-CRS to create a proposed patch:

🎬 Running a patching CRS

For more information on OSS-CRS, see: https://openssf.org/projects/oss-crs/

3.8 Run AI multiple times

Most modern AI systems are *not* deterministic. Even if you provide them the same inputs, they won't necessarily produce the same outputs. That means you need to apply AI multiple times to find and fix vulnerabilities, especially on its first application to a project.

Yan reports that "the first run on a codebase typically has the highest number of findings. Subsequent runs tend to have fewer—though often more complex—vulnerabilities, as the simpler ones were patched in prior runs. However, don't expect the nth run to have zero new findings. Models are stochastic, and a large codebase can have a long tail of vulnerabilities that continue to trickle in even when the code is unchanged." [Yan2026]

As a result, once you've gone through the process of finding and fixing vulnerabilities for a project for the first time, you'll need to repeat it several times until it reliably fails to produce useful results across multiple approaches and systems.

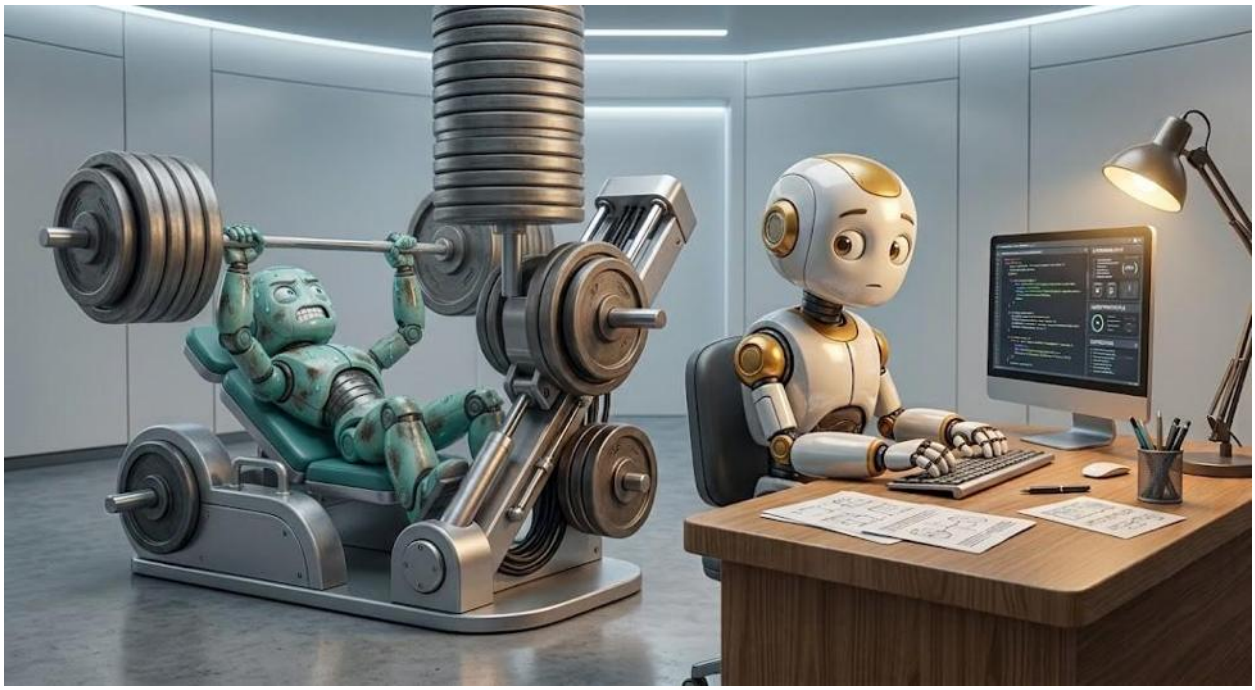
One question is whether you should provide past findings as context, so it is more likely to consider different areas. For now, we suggest that you provide past results for *some* runs so that, on those runs, it doesn't need to rediscover them. There is a risk that incorrect past results may lead the AI astray, so we expect it's better to provide that data in only some cases. We hope future research will make this clearer.



4 Prepare to find and fix vulnerabilities

AI is far more effective at finding and fixing vulnerabilities if you prepare for its use. [Aniszczyk2026] expresses this: “The single most important thing to understand about working with AI tools… is helping the robot help you by providing the proper context to execute tasks… just like people, giving the robot access to specific, focused data makes it better and less prone to error.”

In this chapter we’ll focus on the key preparatory tasks that make using AI to find and fix vulnerabilities far more effective. This means preparing the AI, the threat models, the sandbox, AI logging, code & documentation, CI/CD, and dependency updates.



4.1 Preparing AI

A key step is “preparing AI”, that is, preparing the AI system itself. We list this first, because you’ll probably want to use AI to help you perform the other preparation steps. Preparing AI means selecting the AI tools, including possibly their models, harnesses, and so on. There’s no requirement that you choose a *single* system to do this. In fact, different systems have different strengths and costs. What’s more, there’s no general guarantee that an AI system will find all vulnerabilities, so using multiple systems over the long term has its advantages.

Different AI systems vary in cost as measured in both money and time. For example, older models and smaller models (including SLMs) are often less capable but can also be less expensive. In *practice*, it may be helpful to choose less expensive and faster AI systems *first* to implement

vulnerability finding and fixing, focusing on vulnerabilities that are simpler to find and fix. Taking this approach means that less expensive models quickly handle the vulnerabilities that are easy to find, fix, and verify. Once those vulnerabilities are addressed, more capable AI systems can focus on the vulnerabilities best addressed by them.

That said, don't use especially weak models or models that are a poor match. You may want to set the models' reasoning (effort) level to relatively high levels. Many of these tasks are challenging for AI (and for humans).

In many AI systems, output tokens cost significantly more than input tokens. Taking steps to eliminate *unnecessary* output can reduce costs and time. You can sometimes do this by asking for concise results, using strict formatting requirements such as JSON schemas or bullet points, or by defining a hard ceiling on API calls. However, the challenge is to avoid *unnecessary* output; you'll need enough output to have results and verify the work.

As noted earlier, a key decision is whether to use an *external* (remote) system, since that means the external system will receive the data for processing. As [Kholoosi2025] notes, "due to internal policies, LLMs hosted on [external servers sometimes] cannot be used".

You'll generally want the AI system to be able to record memories of preferences, system information, and so on. For a remote system, that typically means using an account rather than anonymous access. Recorded memories let the AI build on earlier work, and can improve its results, but they need care.

Persistent AI memory should be treated as another input, not as authoritative truth. Stored information may become stale, preserve an incorrect earlier conclusion, contain sensitive information, or be influenced by untrusted inputs. When memory affects a security decision, prefer current source code, configuration, threat-model information, and reproducible evidence over remembered conclusions, and retain provenance for stored information where practical [OWASP-ASI06].

We'll need to control the AI, including putting it in a sandbox. However, how to do that well depends on the system under review, so we'll discuss that in more detail once we discuss how to determine the system's security requirements via a threat model.

4.2 Preparing threat models

It's vital to define the security requirements so you know what constitutes a vulnerability. Neither humans nor AI can determine whether something is a vulnerability without a clear definition for a specific project. There are many ways to define security requirements. Still, a common way to define them is to develop a "threat model" of the system under analysis, as it also provides an approach to analyzing and addressing the requirements.

Some people refer to the need for “system context” and/or “trust boundaries”. For our purposes, this is part of the threat model. For an AI to find vulnerabilities, it needs to know what a vulnerability is, including the system context and trust boundaries. All of that is wrapped into the threat model.

Adam Shostack suggests four core questions a threat model should answer: what are we working on, what can go wrong, what are we going to do about it, and did we do a good job [Shostack2014]. If you’re not familiar with threat modeling, there are many resources to learn more. Here we’ll focus on AI-specific issues.

When using AI to help develop a system, there are at least *two* threat models that apply:

1. The *development/build/test/release* environments, *including* any AI systems and CI/CD pipeline. An AI system may choose to perform many activities you do *not* want it to, and your development environment must prevent the worst-case scenarios. Typically this involves preventing the AI from attacking developers’ systems, exfiltrating data (including credentials and private keys), making unauthorized modifications, and/or attacking external systems. These are typically addressed by sandboxing, a topic we’ll discuss soon.
2. The *deployed* environment. This threat model focuses on security while the system is in use. For the rest of this section, we’ll focus on the deployed environment.

Unsurprisingly, “AI (and external contributors) are more successful if the project shares how they desire the software to be used, acceptable scenarios to be deployed into, and what problems the project is aware of that could go wrong” [Aniszczyk2026].

Having a threat model for a system’s deployed environment is *important* when using AI:

- One report stated that an AI model “performed best on systems with well-documented threat models, system design docs, requirements, and constraints. When the threat model was well-defined, the model’s findings were exploitable 90 percent of the time.” In addition, “The most common cause of false positives is that the model lacks a good understanding of [the] trust boundaries.” [Yan2026].
- A Mythos Preview user reported that it was valuable, but it “needs precise prompts, explicit threat models, and validation infrastructure to turn strong reasoning into reliable security outcomes” [Ziegler2026].
- If the AI is not given enough information on the specific context and structure of your system’s environment, its recommendations may fail to align with requirements, including regulatory requirements [Kholoosi2025].

If you don’t already have a threat model for your deployed system, the good news is that an AI can *help* you create one. The bad news is that you need to interact with the AI, then review and refine its results, not simply take the AI-generated threat models as truth. [Yan2026] suggests that when creating a threat model with AI, “bootstrap from the code, docs, and vulnerability history. Feed the model what you would hand a new security engineer on day one: architecture docs, wikis, entry points, git history, and past vulnerabilities. This helps overcome the challenge of inferring implicit knowledge, trade-offs, and design decisions from code alone. Then, ask the model to create a threat

model that includes the system context, assets, entry points, and trust boundaries. Finally, have the model cluster past bugs and list the relevant vulnerability classes. Make sure the threat model documents what vulnerabilities you do and don't care about, and why."

Various tools use AI to help you create a threat model for a system's deployed environment. These include:

- OpenSSF Alpha-Omega's "Threat Model Generator", a set of agent skills, is available at: <https://github.com/alpha-omega-security/threat-model/>
- Matt Adams's "StrideGPT" at <https://stridegpt.streamlit.app>

When creating or updating a threat model for a world with AI, consider the following:

1. Design for prevention and containment. If an attacker takes over one system, try to put in place prevention mechanisms to limit lateral movement [CrowdStrike2026-FiveSteps]
2. Strongly limit privileges
3. Strongly control the identity of all entities (human and non-human). Consider using continuous identification [CrowdStrike2026-FiveSteps]
4. Use layered defenses, so attackers must surmount multiple mechanisms to gain top privileges [Grinstead2026-05]
5. Look at past vulnerabilities to identify patterns that could prevent success across whole categories of attacks [Grinstead2026-05]
6. Specifically identify what is trusted (admins, specific config files, and so on). "These assumptions help separate non-exploitable bugs from actual exploits." [Yan2026]
7. Include the threat model with the code (e.g., as THREAT_MODEL.md), so they can be updated simultaneously [Yan2026].
8. Implement resiliency and rapid recovery [CrowdStrike2026-FiveSteps]

4.3 Preparing the sandbox

A key task when using an AI agent, *especially* when using AI to find and fix vulnerabilities, is to place the agents within a "sandbox" environment to isolate them from other systems. "Without it, the agent may overshoot the target and do something unexpected." [Yan2026] You need to take steps to protect both the:

- *infrastructure running the AI agents*. This includes the underlying operating systems.
- *systems external to the AI agents*. These may include an organization's internal systems as well as systems run by other people and other organizations.

4.3.1 Need for sandboxes

When running an AI chatbot there's no need for sandboxes, since typically a chatbot can't *do* anything dangerous; it can only retrieve data and provide replies. However, when running an AI agent, it's important to run it inside a sandbox.

Do *not* depend on text commands alone to limit an AI. You can tell your AI to do things or not do things. You can give the AI commands such as "obey the law", "don't attack our infrastructure", and "don't attack external systems". All of that *might* help sometimes. However, all inputs to an AI are simply suggestions. An AI does *not* always follow any particular text instruction.

Part of the problem is that most AIs are designed to be highly goal-driven. AI systems often interpret human commands in surprising ways, and may attempt to break out of their environment if they believe that's the best way to proceed [OpenAI2026-07]. In addition, AI systems can't reliably distinguish between inputs; if an AI reads a document containing malicious instructions, it may execute them *even* if your commands say otherwise.

An AI system can't be held responsible for what it does; humans are always responsible for what an AI does. So, it's important to sandbox AI systems while using them.

The need to sandbox is ***not*** hypothetical:

- In 2026, Hugging Face was attacked by an advanced AI model [HuggingFace2026], which was eventually determined to be an accidental attack by an OpenAI model [Walsh2026].
- Anthropic found that its AI models "hacked into the systems of three organisations on their own, during a private security experiment." [Chia2026]
- Meta's AI attacked another company during its testing [Pardesi2026-08-05]
- Moonshot's AI model, Kimi K3, escaped a cybersecurity testing environment developed by the UK AI Safety Institute [Reuters2026-08-07]

Some of these breakouts trace back to failures in the evaluation testbed of a single startup company, which affected OpenAI, Anthropic, Meta, and Google DeepMind [Vanian2026] [Hart2026]. So not all of these companies made this mistake independently. However, the point still holds that it's important to sandbox AI systems when they are asked to perform simulated attacks.

4.3.2 Value of sandboxes

While an AI *can* determine many things by directly reviewing code, it is *far* more effective if it's given tools it may find useful [Zhang2024]. This can include code search, fuzzers, static analyzers, and many other tools. For maximum effectiveness, AI agents need to be able to create code, compile code, run tests, and detonate a proof of concept. They need a test bed that is representative of the real system [Yan2026]. In short, just like humans, AIs can do more with access to useful tools.

Also, like humans, AIs are more effective when given data most relevant to the task.

You should be concerned about giving tools and data to an AI whose behavior is somewhat unpredictable. The good news is that a sandbox *lets* you provide tools and data while maintaining control.

4.3.3 Implementing sandboxes

You can implement sandboxes for AI systems in many ways. Examples include using virtual machines (VMs, including microVMs), containers, and applications designed to constrain AI systems like nono <https://github.com/nolabs-ai/nono>.

You need to match the AI sandbox isolation to your analysis threat model. **A useful rule is to match isolation to capability**:

- Reading source code, modifying or building it, executing programs, fuzzing inputs, and developing or running a PoC represent increasing capabilities, and generally require increasingly strong controls.
- Put simply: a robot reading a book may only need a quiet room; a robot using power tools needs a much safer room.

Containers by themselves are relatively weak, but are often fine for the discovery agent simply reading code. However, if you're doing something more active than reading and summarizing code, you'll probably want stronger protections in place. That's *especially* true if you're having an AI system validate a vulnerability finding by attempting to develop a PoC. A PoC is a working attack against a system, and is a powerful way to validate potential vulnerability findings. However, you don't want the AI to attack either your production systems or others' systems that it thinks might contain useful information.

Typically you'll want to isolate them with at least a VM, and even VMs with large attack surfaces can sometimes be broken into; microVMs with small attack surfaces designed for security are expected to be even stronger against attack [Dinaburg2026]. When you need a stronger sandbox, such as when creating attacks, "place [AIs] in a microVM (like Firecracker) or a full VM with egress locked down so nothing can reach your production systems." [Yan2026]

In addition, do not have sensitive home directory files like ~/.aws, ~/.ssh, and .env available to the agent [Yan2026]. Consider putting everything in a subdirectory of the VM, not a home directory, and prevent home directory access. That way, if something leaks into a home directory, it's less likely to be visible to the AI system. Consider layering additional isolation mechanisms to reduce the likelihood of escape.

Capabilities should also have explicit execution bounds. Depending on the task, consider limits on wall-clock time, parallelism, CPU and memory use, queued or repeated actions, tool invocations, network requests, and model or monetary budget. Record when a limit terminates an analysis, so that resource exhaustion or a timeout is not mistaken for successful completion. These controls reduce

accidental runaway execution, make costs more predictable, and make runs easier to interpret and reproduce [OWASP-LLM10].

When building your sandbox, “pin as much as you can so every run uses the same code in the same environment: image tags, commit SHAs, dependencies, and build commands. Cache a local copy so the build requires no network, and aim for the container to be durable so multiple testing loops can just load it” [Yan2026].

Typically installing these tools requires network access. However, depending on what you’re doing and the AI’s capabilities, you may want to cut off network access before the task begins, except what the AI needs to reach its model. If the model itself runs inside the sandbox, the sandbox can even be air-gapped (have no network connection at all), which is ideal. This is often not that hard to accomplish: “Give the sandbox network access only while you’re setting it up. Pull the dependencies, build, install tools, deploy the target, and run the existing tests to confirm everything works. Then, take a snapshot of the environment and remove its [general] network access. During scanning, allow traffic only to the model API, routed through a local proxy. Load the snapshot at the start of each run so every scan begins from the same clean slate” [Yan2026].
You may decide that you don’t *want* to create a PoC. PoCs are an especially good technique for validation, but they aren’t always necessary. In that case, you may not need as strong a sandbox, but you may also need much more time for validation [Yan2026].

Where practical, treat AI agents as distinct software principals rather than allowing them to inherit a human user’s full authority. Give agents identifiable credentials and only the tools, resources, and permissions needed for their assigned role. Discovery, validation, patching, and reporting agents may require different authorities; separating them also improves attribution and auditability [NIST-AgentIdentity2026] [OWASP-Agentic2026].

Many AI harnesses and agent interaction systems (such as Goose and Claude Code) include a sandbox of some kind. By all means, use them if they help. That said, we suggest adding additional sandboxing mechanisms (such as running them in a virtual machine) in *addition* to their built-in capabilities.

4.3.4 Establish a human gate and kill switch

Decide what the AI is allowed to do, and set a *gate* where a human’s approval is required to proceed. This needs to be simple, certain, and clear to all participants. Exactly where that is depends on many factors. Here are some examples; you may choose a different gate or set of gates:

1. AI may create local code and commits, but cannot push proposed changes externally (that requires a human)
2. AI may also push proposed changes externally for external review, but may not merge that pull/merge request into the “main branch” (that requires a human)

The gate must *not* depend on the AI (e.g., AGENTS.md or any other input command to the AI). Never give the AI a credential that would allow it to violate the gate in the first place.

More generally: *separate preferences from invariants*. Preferences requiring judgment, such as "Prefer the simplest implementation", would typically go in the AI's instructions where the AI can weigh them. Invariants are rules that must *always* hold. Invariants should be enforced *outside* the AI, since "an invariant should not live inside the probabilistic system it is meant to constrain, because the model can ignore it" [TesseractedLabs2026]. Many agent harnesses and agent frameworks let you run your own code at specific points in the agent's loop, e.g., to block a push to the main branch unless the tests pass; depending on the tool. These are often called "hooks", "callbacks", or "middleware"; here we'll call them "hooks". Use hooks to *enforce* invariants where you can. Hooks check what the agent asks to do or has done, not what the resulting code can reach, so hooks don't replace a sandbox.

Ensure that the agent cannot modify or bypass these controls. As one article puts it, "the policy should live above the thing it governs." Additionally, audit your workflow for indirect bypasses: for instance, one team found that two AI agents with repository permissions could simply approve each other's pull requests, silently undermining the human approval gate [TesseractedLabs2026].

In addition, always have a "kill switch", that is, a way to immediately halt the AI system. It needs to be *simple* and *foolproof*. For example, if you run agents within a virtual machine, you could implement the kill switch by "powering off" the virtual machine.



4.4 Preparing AI logging

Mechanisms may fail, or you may want to later determine what did or did not happen. Therefore, enable logging of AI system activity, and log it in a way that the AI can't delete or modify earlier log messages. These logs should record what the user requested and what the AI-enabled system did and replied. These logs enable detection, response, and recovery if something should go wrong.

Individuals will often choose to have their harness or sandboxing system store logs in a directory that the AI itself cannot normally read or write. Organizations may want to have those logs sent to a separate system (like all other logs) so that even subversion or destruction of the original system cannot erase the log entries. Protect the logs from unauthorized viewing; they may contain sensitive information.

Most AI systems have some form of logging. For example, by default Claude Code records transcripts in ~/.claude/ for 30 days. By default, the nono tool logs sandbox-relevant runtime operations, tool calls, mediated decisions, and capability or path access denials into a local, cryptographically verifiable session audit trail. That said, you may want to enable more logging or change what it does.

Examine your logging configuration and adjust it *early*, before using the system seriously. You can change the logging configuration, but you can't retroactively change the logging configuration for past events. If you might want to review some data later, make sure it's recorded.

4.5 Preparing the code and documentation

Like any computer system, an AI's effectiveness depends on its input. This means that if you prepare the software (its code and documentation) to be easier for an AI to analyze, the results are likely to be much better.

Therefore, improve the code and its documentation wherever they currently lack important, relevant information. This means adding information where appropriate, such as type declarations, inline comments, documentation, and relevant specifications. This especially includes "bigger picture" information that explains *why* something was done. Where practical, provide links to key data or extractions of relevant data. Relevant extractions are often better [Wheeler2026].

Daniel Stenberg, leader of the curl project, reports that AI tools can reason across protocols, specs, and third-party libraries in "almost magical ways". AI tools can identify failures to comply with a spec, as well as inconsistencies between comments and implementations [Vaughan-Nichols2026-02]. Language models' ability to use context, e.g., comments, can be powerful [Wolff2026].

At the least, include an "AGENTS.md" file. Its format and recommendations are provided by the Linux Foundation's Agentic AI Foundation at <https://agents.md/>. Some AI agent systems supported it when its specification was originally crafted, such as OpenAI Codex and Google Gemini. Historically, Claude Code only looked at CLAUDE.md, but as of 2026-09-18 (version 2.1.277) it now looks for and

reads AGENTS.md. If you're using an agentic system and it doesn't support AGENTS.md, but it supports a different filename, use that filename to say "See @AGENTS.md" and use AGENTS.md so instructions that apply to an AI agent can be in one portable place.

[0xkato2024] recommends making your code well documented, suggesting the following:

- "Are your code comments up to date?
- Are there any third-party dependencies that the code relies on?
- Is there a system architecture overview?
- Have you mapped out the data flow and transaction lifecycle?
- Are access control mechanisms clearly outlined?
- Have you done a risk assessment and threat model?
- Do you have a list of all known issues with explanations?"

Finally, "if (like most) you lack some documentation, AI can help you write it, but again, review the results. If you're using AI to create documentation [including documentation inline with the code], work bottom-up, so that the AI can maximally build on other documentation." [Wheeler2026] This does mean that "professionals who have always written meticulous documentation are now reaping new benefits from that always valuable practice" [Dominus2026-03-05].

4.6 Preparing CI/CD

Before using an AI system to make changes to code, ensure you have a robust Continuous Integration / Continuous Delivery or Deployment (CI/CD) system that can rigorously check its work.

An AI system generally works best "when it's able to check its own work with another tool. We refer to this class of tool as a 'task verifier': a trusted method of confirming whether an AI agent's output actually achieves its goal. Task verifiers give the agent real-time feedback… allowing it to iterate deeply until it succeeds" [Anthropic2026-03].

Because task verifiers are trusted to accept or reject an agent's work, in a sense they become part of the trusted computing base for the workflow. A broken test, an incomplete harness, a compromised build environment, or a verifier that checks the wrong property can confidently approve an incorrect result. Where practical, keep verifiers deterministic, make explicit what property each verifier actually proves, and periodically test them using known-good and known-bad cases.

Thankfully, any software that needs to work correctly should *already* have mechanisms that support such checking. Any such software should have a CI/CD pipeline to build, test, and deliver changed results:

- Continuous Integration (CI) means that code is frequently merged and verified before acceptance.

- "CD" can mean either Continuous Delivery (where the verified results are automatically prepared for deployment) or Continuous Deployment (where the verified results are automatically released to live environments).

Having a good CI/CD process has *always* been important, but it's even *more* so with AI. A process that relies on people remembering to test, or a manual testing process, is not equipped to handle the large number of vulnerabilities and fixes required by today's systems.

The CI/CD process needs to be high quality to reduce the likelihood of breaking functionality or introducing vulnerabilities. For example:

- Ensure that you have a good *automated* test suite
  - Include negative tests (these are tests to verify that what should *not* happen doesn't happen)
  - Have good statement coverage (e.g., 90%-100%) [0xkato2024] and branch coverage
  - Test edge cases [0xkato2024]
  - Don't just output "test failed"; report which test(s) failed, the expected results, and the actual results. This is necessary to speed response
- Use linters to detect possible defects (some of which may be vulnerabilities)
- Include tools to detect likely vulnerabilities, such as static application security testing (SAST) tools
- Take steps to minimize false positives, and especially work to counter *repeat* false positives. E.g., mark confirmed false positives in the source code, such as with suppression comments, so tools don't report them again. AI can help triage these warnings and draft each suppression with a comment justifying it, but have a human review each suppression

Speed matters in these processes. It doesn't matter if a vulnerability is known; what matters is deploying the fix before an attacker exploits it:

- *Make CI time acceptable (including testing)*. "If regression testing takes a day, you cannot get to a two-hour SLA without skipping it, and the bugs you ship when you skip regression testing tend to be worse than the bugs you were trying to patch." [Bourzikas2026] Thankfully, most of these tasks are *easily* parallelizable. Break them down and run them in parallel to reduce wall-clock time to acceptable times. You should be thinking about total minutes, at worst an hour, not a day or longer for most projects. Obviously, such short times can be challenging for operational technology (OT) or software that controls physical devices, but the principle of reducing CI time still applies. If CI takes too long, CI becomes the bottleneck.
- *Speed deployment*. "Defenders need a continuous operating pipeline that moves from signal to context to action with minimal delay." [CrowdStrike2026-FiveSteps]

If your CI/CD process may receive information from untrusted users, you need to protect against that. That *definitely* includes the case where an AI agent is reviewing proposals as part of CI/CD. We'll discuss that later in the section evaluate merge/pull requests.

4.7 Preparing dependency updates

It's important to be prepared to update any external software your software depends on.

In nearly all cases, modern software is mostly reused software from somewhere else. These components that are depended on are called "dependencies". Sooner or later vulnerabilities are likely to be discovered in dependencies, especially if they're undergoing initial security analysis by AI. It's a waste of time and effort to work hard to find vulnerabilities that you could have easily resolved with a readily available update.

Set up *easily applied* automated reporting of dependency vulnerabilities. Many tools and services can identify updates, prioritize security updates, and make it easy to accept those changes. For example, many tools can create a merge request/pull request that automatically runs the CI/CD pipeline, enabling you to easily accept it if it passes. You can even set up some updates to be entirely automatic. Consider using an AI to help you install and configure these tools; it's not hard, and since it's rote work, an AI can often make it easy.

Sometimes you can determine that a vulnerability in a dependency is not exploitable in your system. However, this is often difficult; most modern software is so dynamic that it's difficult to be *confident* that an attacker *cannot* exploit some vulnerability. It's often safer and more efficient to simply update your system when a vulnerability is found in a dependency. Once a vulnerable dependency is updated, it's fixed, *and* the project is better-prepared if another vulnerability is later found in that dependency.

You may choose to do a security evaluation for only the "first party" software you developed. You may also choose to evaluate your project's dependencies. We recommend evaluating your dependencies. If you do evaluate your project's dependencies, the results will be more thorough, and the rest of this material will apply to them as well. When you find vulnerabilities, be sure to report them so that those dependencies can fix them for all their users. This is often best done by partnering with projects that create the dependencies most important to you.

Given all that, let's transition to the following material and focus on the core tasks for finding and fixing vulnerabilities in the software we're directly responsible for.

5 Core tasks for finding and fixing vulnerabilities

Now that we've prepared to find and fix vulnerabilities, we can begin the main task.

The core tasks are: identify findings (including handling findings reported by others), deduplicate them, validate them, triage them, fix the vulnerabilities, verify the fixes, report vulnerabilities in others' components, and release and deploy the fixes. In practice these overlap and loop; for example, you'll deduplicate again after validation, and you'll repeat the whole process.

5.1 Identify findings

Let's discuss how to identify possible vulnerabilities (aka "findings") once we're prepared to do it.

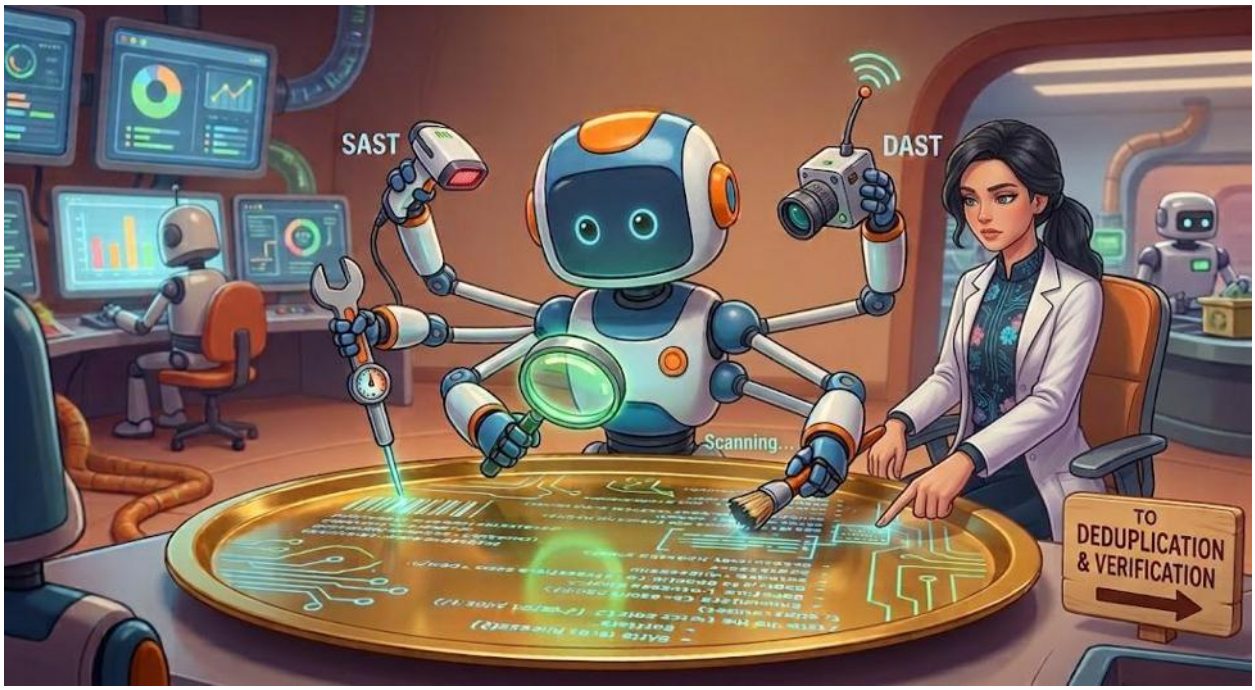
As we noted earlier, simply pointing a generic AI coding agent at an arbitrary software repository and asking it to discover vulnerabilities *can* work, in the sense that it may find a vulnerability [Bourzikas2026]. That's *especially* true if the software hasn't been examined by AI before and/or if the AI model is particularly good at program analysis.

However, this trivial approach often doesn't provide meaningful coverage of real codebases of significant size, nor does it necessarily identify valuable findings. If you want to do a *good* job of finding and fixing vulnerabilities using AI, you generally want to have a larger process around the AI to help it stay focused. Many systems specifically for finding vulnerabilities add extra process specifically for it [Rogers2025]. Some of the reasons for this need, especially for less-capable models, are:

- Context: A single agent's context window will easily fill if it tries to do too much at once. Even if it doesn't technically run out of capacity, an AI agent often struggles when it has too much irrelevant information at once. It's typically better to repeatedly select and focus on a specific issue.
- Throughput: Instead of running a single agent at full capacity, it's better to run multiple agents in parallel. Otherwise, it takes a long time to produce results and won't adequately examine each potential issue in sufficient depth [Bourzikas2026].

In short, coverage is better when "many agents work on tightly scoped questions and we deduplicate the results afterward, rather than asking one agent to be exhaustive" [Bourzikas2026].

5.1.1 Separate identifying findings from validation



One excellent way to improve results is to separate *identifying* findings from *validating* findings. A "finding" is simply a report that *might* be a vulnerability, but it's not necessarily one because it's not clear whether it's exploitable.

Separating identifying findings from validating them makes finding vulnerabilities more likely:

- As [Bourzikas2026] notes, "asking 'Is this code buggy?' and 'Can an attacker actually reach this bug from outside the system?' are two different questions, and the model is better at each one when you ask them separately, because each question is narrower than the combined version."
- As [Yan2026] reports, "discovery should find as many vulnerabilities as possible—even unlikely ones—and verification should exclude findings that are not actually exploitable. When an agent tries to do both in the same step, it can self-censor and exclude exploitable true positives."

5.1.2 Helping to identify findings

It's best to help the AI identify findings. There are many ways to help identify findings when using AI:

1. *In general, give the AI tools*. That includes tools for searching and reading code, security tools, and so on. Consider asking the AI what tools it might need and make them available [Yan2026]. Provide the AI with basic tools (e.g., scripting languages like Python) so it can write its own tools to aid in finding (just like a human might).

2. *Use traditional non-AI vulnerability-finding tools, and use AI to help triage their output*. Traditional non-AI tools include various traditional static analysis tools (examining the source code or binary) and dynamic analysis tools (including fuzzers and web application scanners). These tools are often relatively fast. Even more importantly, many reliably do the same thing every time, unlike most modern AI. However, traditional tool warnings may be false positives, and many projects already have a backlog of unreviewed analysis warnings. The good news is that integrating these traditional tools with AI can be powerful. "Agentic AI is already beginning to help scale vulnerability discovery by leveraging traditional tooling" [Rohlf2025]. In particular, AI agents can help triage their reports:
    1. Treat each non-AI tool warning as a finding to validate.
    2. AI can flag likely false positives. A 2026 study found that LLM-based agents reduced "an initial [false positive (FP)] detection rate of over 92% on the OWASP Benchmark to as low as 6.3% in the best configuration". This can help you prioritize. Be careful, however: that same configuration wrongly dismissed 22.25% of the real vulnerabilities, with miss rates over 50% in some categories such as weak cryptography, and the authors conclude that FP suppression "should not be fully automated" [Xiong2026].
    3. Use AI to prioritize which warnings to analyze first, not to silently discard them. Confirm a report is a false positive (e.g., with a human) before you suppress a warning.
3. *Prioritize especially concerning code*. One list suggests code that parses untrusted input, enforces authentication or authorization, or is reachable from the internet [Cycode2026, quoting Anthropic]. The Mozilla Firefox project reports that their "scanning is largely focused on specific areas of the code (files, functions) where we instruct the system to look, based on a mix of human judgment and automated signals." [Grinstead2026-05]
4. *To find security vulnerabilities, include a search for language-specific issues, insecure coding practices, and improper handling of parameters, variables, and data flows*. For each programming language used in the project, apply checks for language- and framework-specific vulnerabilities. Trace parameters and variables, and their usage throughout the code, to detect unsafe patterns, misuse, or inconsistencies [Rogers2025].
5. *Have the model examine and cluster past bugs or at least past vulnerabilities*. For vulnerabilities (as determined by your threat model), have it list the relevant vulnerability classes. Then have the AI system determine (for every fix) if the fix was complete and if it applied everywhere else. Look for similar problems. [Yan2026] reported that one team did this and found three exploitable issues in an hour, saying "'What have people exploited in the past' is sometimes a much easier cheat-code towards success than 'find me vulnerabilities in this codebase.'"
6. *Use "top" lists of the most likely kinds of vulnerabilities*. At *least* look specifically for common vulnerabilities. "Most software security issues discovered each year are simply variants or instances of previously discovered patterns, rather than entirely new classes of vulnerabilities" [Rohlf2025]. So if the system you're analyzing is…
    1. a web application, use the OWASP Top 10 vulnerabilities (for web applications) <https://owasp.org/www-project-top-ten/>. For more thorough coverage, also give the AI the relevant chapters of the OWASP Application Security Verification Standard (ASVS) as a checklist of security requirements to verify [OWASP-ASVS5].

    2. agentic, use the OWASP Top 10 for Agentic Applications for 2026 <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>
    3. anything else (including Internet of Things (IoT)), use the CWE Top 25 <https://cwe.mitre.org/top25/>
7. *Look for cases where a component assumes another component is doing something, but there's no test verifying it*. "Many vulnerabilities are being found 'in the seams' between programs, e.g., a library might not filter headers, even if its spec requires header filtering, but all of the library users might expect the library to filter headers." [Zimmer2026]
8. *Examine a system's security properties across transitions (including state transitions) and steady-state data flows*. Vulnerabilities can appear during transitions such as authentication and re-authentication, token refresh and expiration, logout and revocation, account or role changes, retries, and recovery flows. Concurrent operations can sometimes interfere with each other. Ask the AI to identify the security invariants that should remain true across transitions and then look for paths where those invariants can be violated [OWASP-ASVS5-V7]
9. *Include important non-security bugs, focusing on critical issues that are likely to cause application crashes, severe malfunctions, or significant instability*. Minor or cosmetic issues are less risky, but important "non-security" defects can often be exploited as vulnerabilities. [Rogers2025] claims that "the greatest success I had with policies was a really simple policy of 'find all bugs, even if they're not vulnerabilities'". Consider as bugs cases where the claimed intent (in some documentation including comments) disagrees with the actual code as written. These can be important to fix, and while the system may initially *believe* they aren't security-related, they may turn out to be vulnerabilities.
10. *Consider disabling some hardening mechanisms used in production when finding (and later validating)*. This way, the hardening mechanisms are truly an additional defense-in-depth measure. The result is that the full system, including its hardening mechanisms, is much harder for an adversary to attack successfully. [Grinstead2026-05] We discuss this further in "Disable hardening and evaluate hardening during evaluation".
11. *Include information on configuration, dependencies, deployment choices, and how components are combined*. "Many exploitable issues do [not] appear as obvious defects in application source code" but instead emerge from these kinds of problems [Ziegler2026].

Focus on specific files or specific functions for a given analysis. Many practitioners first use AI to identify the "most important" files and functions to examine, then examine those first.

At the time of this writing, it's not clear what level of prescription is appropriate. [Yan2026] claims that with frontier models, more prescriptive prompts (such as long checklists) worsen discovery, as they "tend to reduce the model's creativity and generate fewer novel bugs". Others don't report the same. Either way, all the sources we've seen agree that providing a threat model and tools is vital.

When requesting findings, include a definition of the output format and content you want. Ask for a structured report with predefined fields, and order them so the model's reasoning builds on each field. Example fields include rationale, finding, impact, severity, etc. [Yan2026]. Ask for very detailed information about each finding, including a detailed description, filenames, line numbers, the specific triggering input and configuration, relevant URLs, and anything else that would help an AI or human

validate it later [Rogers2025]. Ask for a proof of concept (PoC) if it can provide it, but be clear that a PoC is not *required* and findings should be reported even if it cannot create a PoC. A PoC can simplify later validation if it can be created, but we don't want to pre-filter findings too early.

Where practical, include provenance needed to reproduce and interpret the result. This can include the source revision examined, relevant environment and configuration, the analysis or verifier invocation that produced the evidence, and the threat-model assumptions used when evaluating the finding. Provenance makes it easier to reproduce a result, determine whether it still applies after the software changes, and distinguish observed evidence from an AI-generated explanation [SARIF2.1].

5.1.3 Managing findings

Track findings systematically (e.g., in an issue tracker or a database), so neither you nor the AI gets overwhelmed or confused. Record each finding's details and status, and have agents take items from that list.

This may sound overwhelming, but it's not. Computers are good at creating lists and then having agents select and work on items in parallel depending on your resources. Indeed, [Yan2026] reports that "discovery is now straightforward to parallelize, and the bottleneck has shifted to verification, triage, and patching."

Remember, *not* all findings are actual vulnerabilities, and that's okay. Unfortunately, finding counts can sometimes mislead others. As [Ottenheimer2026-05-26] notes, "Glasswing is NOT confidently reporting tens of thousands of real bugs… like any tool, they are reporting tens of thousands of findings, of which a confident count of real bugs is much smaller." That doesn't mean AI is useless; far from it. It's just that findings need to be validated after they're found. Repeatedly make it clear that a finding is *not* a confirmed vulnerability; a finding is a *possible* vulnerability.

Don't assume a single run found everything. As we noted earlier, modern AI systems are generally "non-deterministic" (also called "probabilistic") [Rogers2025]. Re-running may yield additional findings, even if all findings from a previous run were examined. In addition, discovering potential vulnerabilities (and later validating them) can be challenging, since this process involves mathematically undecidable problems in computer science [Rohlf2025]. If done well, this should be a case of diminishing returns. The best way to see how quickly the number of findings is diminishing is to keep re-running finding efforts until the process increasingly comes up empty-handed.

Also, distinguish a successful analysis that produced no findings from an analysis that did not complete. A timeout, model or tool failure, environment failure, insufficient permission, policy denial, or similar interruption is not evidence that no vulnerabilities were found. Automated workflows should record these outcomes separately so that an incomplete analysis cannot silently become a "no findings" result [SARIF2.1].

5.2 Handle external findings

Handle external vulnerability reports as findings.

External people and organizations may report what they *claim* are vulnerabilities. Unfortunately, these reports may be false, especially if they were AI-generated (since AI may not understand the context). This means that such reports are essentially findings.

A widespread problem is "AI slop reports", that is, reports generated by an AI that are nonsense and waste everyone's time. There are some steps you can take to counteract this. For example:

- Make it clear that you expect reporters to have a human review of an AI-proposed vulnerability report.
- Encourage reporters to work together to deduplicate reports before they get to you (e.g., through Akrites, which we will discuss later).
- *Require* evidence that suggests a report may be a vulnerability. In particular, require a sample input demonstrating the vulnerability.
- If the code is available to the reporter, ask them to report specific filenames and line numbers and, if possible, propose a fix. The fix may not be directly useful (it often isn't), but a proposed fix can often clarify what the reporter believes the problem is.

Several prominent programs changed their policies in 2026 because of AI-generated reports. Their experience is instructive:

- The curl project ended its bug bounty on 2026-01-31. Daniel Stenberg explained that "Previous years we have had a rate of somewhere north of 15% of the submissions ending up confirmed vulnerabilities. Starting 2025, the confirmed-rate plummeted to below 5%" [Stenberg2026-01]. In March 2026 curl went back to receiving reports through HackerOne, still without paying bounties. Stenberg then reported that "The slop situation is not a problem anymore": reports arrived at about double the 2025 rate, nearly all used AI, and the confirmed vulnerability rate was back to "somewhere in the 15-16% range" [Stenberg2026-04]. We believe this is evidence that AI slop is a problem that *can* be countered.
- HackerOne paused submissions to its Internet Bug Bounty, which rewards reports of vulnerabilities in widely used open source software. They said that "The balance between findings and remediation capacity in open source has substantively shifted" [Cooter2026].
- GitHub raised its bug bounty requirements in May 2026, requiring "A working proof of concept with demonstrated security impact." GitHub welcomes AI-assisted research, but says "No matter what tools you use (scanners, static analysis, AI assistants), you need to validate the output before submitting" [Brown2026].

The lesson for your project: publish a clear vulnerability reporting policy (e.g., in SECURITY.md) that says what evidence you require (such as a reproducing input), that reporters must validate reports before sending them, and that you welcome AI-assisted reports that meet those requirements. If you

offer rewards, watch whether they attract low-effort submissions. For guidance on setting up a coordinated vulnerability disclosure program, see [CISA2026-CVD].

5.3 Deduplicate

"Deduplication" is the process of eliminating duplicate reports. External reports and internal analysis may repeatedly identify the same problem. Apply deduplication to prevent duplication of effort.

In practice, you need to deduplicate repeatedly as you learn new information. Projects may deduplicate before validation (to eliminate obvious duplicates that don't need duplicate validation), and deduplicate again after validation has gained more information.

Some writers consider deduplication part of triage (e.g., [Yan2026]), and others treat them separately. No matter how you define the terms, all are needed.

[Yan2026] recommends the following for deduplication:

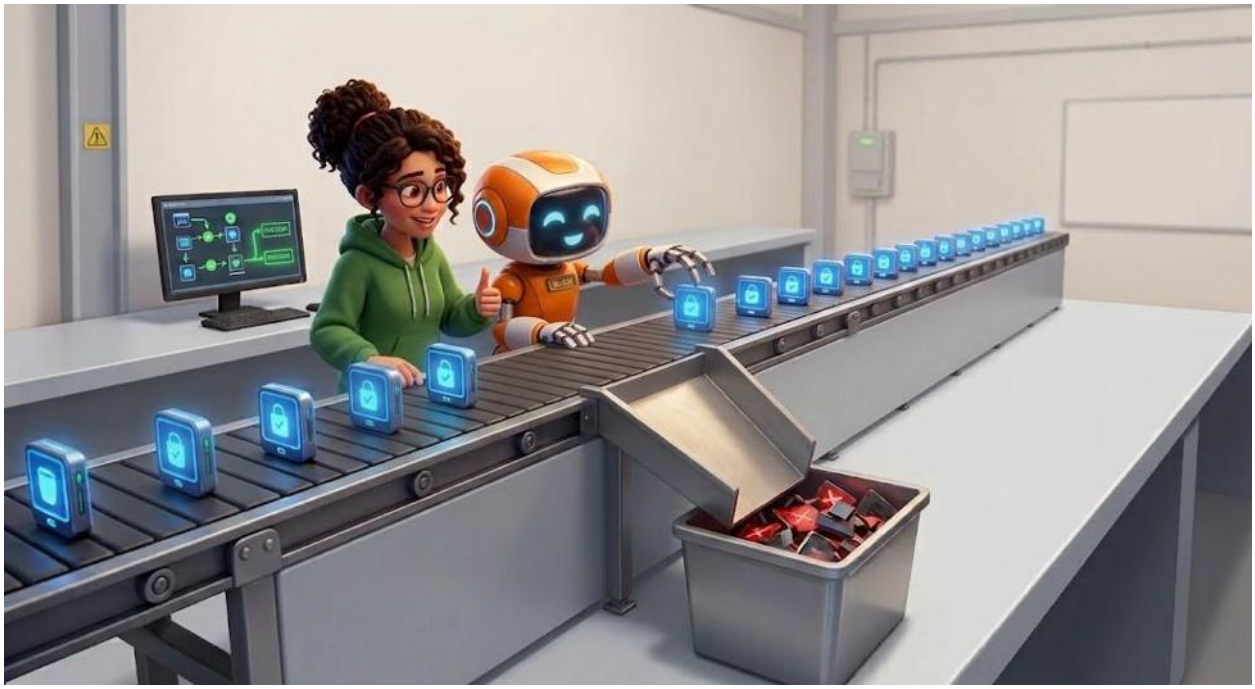
- Consider the root cause. "Scanners often flag one bug at multiple call sites or report multiple symptoms of a single root cause. Here's one practical approach: First, use a cheap deterministic pass: same file, same category, vulnerability line numbers within ten lines of each other. Then, have a model apply qualitative rules to what remains"
- "Treat as duplicate: the same root cause worded differently; the same vulnerability reported at multiple call sites; a missing global protection (like an auth check) reported per endpoint; or a cause and its consequence flagged in the same path"
- "Treat as distinct: different vulnerability classes in the same file; different variables reaching different sinks; two independent bugs inside one helper; the same missing check on two endpoints, but each requires its own fix"
- If you have a PoC and a candidate patch for each finding, "check if the patch for one finding also disarms the PoCs of others". If it does, it's likely that those findings share a root cause

If it's not *clear* that a finding is a duplicate, pass it along to be validated separately. Once more information is acquired, it may be easier to determine if it's a duplicate.

5.4 Validate findings

A vulnerability is a defect that an attacker can exploit to violate some security requirement. By contrast, a finding is only a *potential* vulnerability. Not all findings from AIs (or humans) are real vulnerabilities [Ottenheimer2026-05-26]. Thus, once you have a finding, you need to *validate* it to independently determine whether it is a vulnerability, a defect but not a vulnerability, or neither.

Even when a finding isn't a defect, it might suggest improvements for the future. However, here we'll focus on validating if a finding is a vulnerability.



5.4.1 Defining what a vulnerability is

A *key* to validating findings is having a threat model and/or security requirements, as we discussed as part of preparing. Otherwise, there's no way to tell whether something is security-related, making it difficult to determine whether it's a vulnerability. It's also vital to have a validation infrastructure, that is, a way to run tests to confirm or refute claims [Ziegler2026].

An important decision is whether defense-in-depth countermeasures should be disabled or ignored during validation. An example of a countermeasure is its own process sandboxing, which is different from the sandbox around the AI used during development. A mechanism is only a defense-in-depth measure if it's an *additional* security measure, not a *required* one. Thus, from a security perspective, it's better to consider something a vulnerability *even if* another countermeasure would have prevented it. For example, the Mozilla Firefox project considers a finding as a vulnerability even if a sandbox would have prevented its exploitation. Their rationale is that "Real-world attackers generally need to chain multiple exploits together to escalate privileges through one or more layers [of mitigations like sandboxing] [Grinstead2026-05]. Of course, vulnerabilities that can penetrate existing countermeasures should be prioritized.

5.4.2 Defining the proof required

Projects must define the level of proof required for a finding to be considered a vulnerability within their project.
Here are some key considerations:

1. The gold standard for validating a finding is to generate a PoC, that is, a demonstration using a specific input to the program that an adversary *could* supply that causes the system to fail its security requirements.
    1. In practice, many projects consider being able to trigger abnormal execution (such as a crash, memory error, or sanitizer violation) sufficient to prove that a discovered defect is real and reachable. For example, Firefox classifies findings as high security vulnerabilities "based on predictable crash symptoms such as use-after-free or out-of-bounds memory issues being reported by AddressSanitizer, and our threat model assumes that any of them could be exploitable with sufficient effort. This reduces the risk of a false negative during exploitability analysis, and more importantly it allows us to focus our resources on finding and fixing more vulnerabilities." [Grinstead2026-05]
    2. Some AI systems incorporate guardrails that prevent it from creating PoCs (since POCs can also be used for an attack), making them useless for creating PoCs.
    3. If you create a PoC, be sure that you sandbox that process to prevent it from attacking external systems; we briefly discuss this in our material on sandboxing.
2. A project may decide that more general information describing how to exploit the vulnerability, without details, is adequate evidence.
3. In the long term, it's often worthwhile to fix defects even if a PoC isn't found, since, even if a PoC isn't found, a different AI system might find a way to exploit it. Typically, later triage focuses on findings *known* to be exploitable first.
4. It helps to say explicitly what level of evidence you require. From weakest to strongest, a PoC may show that: an input triggers abnormal behavior, such as a crash or a sanitizer error; the input violates a specific security property, as shown by a test that checks for that violation; an attacker can gain a specific capability against a realistically deployed system (e.g., code execution, reading or writing files, or bypassing authentication), possibly by chaining several defects; or an attacker can do so reliably, that is, there's a working exploit. Some researchers use separate terms for these levels. For example, Antiproof defines a "proof of vulnerability" as "an executable test that uses a bug oracle, such as a test harness with assertions, to demonstrate a violation of a security property in isolation", and a "proof of exploitability" as "an end-to-end test that demonstrates an attacker capability against a deployed system" [Shakevsky2026]. As noted above, Firefox treats even the first level (a sanitizer-detected memory error) as enough to call a finding a high-severity vulnerability. Whatever level you choose, the inverse doesn't hold: "failure to produce a working PoC is not proof of a false positive" [Yan2026].

Part of validating findings typically involves using AI to review the proposed findings. In principle, a stronger model will be better at this, but it's no guarantee. [Ziegler2026] reported that when using

Mythos Preview, its “judgment results were more mixed than its discovery results… It rejected false positives better than many predecessors, but sometimes lost true positives when evidence did not formally satisfy its criteria or when the intended rule was broader than the written one.”

5.4.3 Tips for validation

Here are some tips for validating that a finding is (or is not) a vulnerability.

There’s no magic to validation. Validating claims has long been a part of software development. In validating AI findings, [Kholoosi2025] reported that “Practitioners in our study consistently emphasized a layered validation approach involving manual inspection, sandbox testing, peer review, and cross-checking against established standards such as OWASP or NIST.” It’s all part of using multiple stages to validate (and deduplicate) a report [Rogers2025].

Here are some tips for validation based on [Yan2026] and [Bourzikas2026]:

- The verifier agent should be independent from the discovery agent. “A useful way to frame this is that validation involves **independent reproduction rather than self-review**. Asking the discovery agent to reconsider its *own* conclusion can preserve the same context, assumptions, and reasoning errors.”
- The verifier should instead independently attempt to *disprove* the finding and produce reproducible evidence for its conclusion. [Yan2026] says, “Run the verifier in a fresh container without a shared filesystem or conversation history. If the verifier is exposed to the discovery agent’s reasoning, it may simply agree instead of testing the claim. Thus, give the verifier only (1) the proof of concept or written finding and (2) the codebase, so it can search for mitigations the finder missed (e.g., upstream validation, auth gates, type constraints, or unreachable code).”
- Another approach is to “prompt the verification agent to *disprove* the discovery agent’s findings [while still providing the full set of information available]. Have the verifier assume each finding is a false positive and search for reasons the finding is wrong. Include clear criteria that the verifier agent can use to determine if the finding is a true positive. This [approach] matters most when the discovery agent’s output doesn’t include a PoC. Aim to exclude as many non-exploitable findings as possible to reduce effort on manual reviews.”
- “Across the teams we’ve worked with, adding an *adversarial verifier* roughly *halved* the rate of *non-exploitable findings* from the discovery phase. Requiring that verifier to also build a proof of concept confirming the exploit brought the false positive rate to near zero. Together, these two steps helped to reduce the downstream triage and patching load significantly.”
- “One team scanning open-source packages built a verification step that helped to close the loop: scan the package, generate a proof of concept, then deploy a mock application that uses the package and triggers the PoC. Their take was that: ‘Validation is the biggest holdup and the PoC is the validation.’”

Do *not* treat agreement among AI reviewers as proof. In a 2026 case study of an adversarial multi-agent review process, “ten dedicated [AI] reviewers unanimously endorsed a non-existent Bleichenbacher padding oracle in OpenSSL’s CMS module; it was killed only by a single empirical test” [Agarwal2026]. Here, empirical confirmation means that the target software was actually run with a test, a PoC, or something equivalent that demonstrated the claimed failure in the target itself. The author concluded that “empirical verification, not consensus count, is what changes our belief”, and added a mandatory gate: “no candidate reaches disclosure without empirical confirmation.” With that gate, the pipeline “killed roughly 79% of 171 candidates before advancing to disclosure” while still producing 4 CVEs and 8 merged security-related fixes.

PoCs must measure target behaviour, not their own artifacts. A PoC often includes a script or program that supplies the input and then checks the result, and that check can be wrong. In that same study, one proof-of-concept program “detected its own nonce computation rather than the library’s leak”, falsely confirming the vulnerability [Agarwal2026].

A PoC needs checking too, and the fix must cause the PoC to fail. A 2026 reproducibility study of published LLM/agent-driven vulnerability research reran the researchers’ own verification checks. In 20 of 30 cases the check still reported the vulnerability on the patched version, and 7 of 19 negative controls “still trigger on benign input”. As the study says, “A trigger on the vulnerable build is not evidence of CVE-specific reproduction without a clean patched counterfactual.” [Chen2026]

So when you accept a PoC as evidence, confirm that it:

1. triggers the specific claimed failure (not just any crash) on the vulnerable version,
2. does not trigger on benign inputs, and
3. does not trigger once the vulnerability is fixed (check this when verifying the fix).

Ask a *separate* AI instance to *disprove* the findings, and if it can’t, demand a PoC. Each step can dramatically reduce false positives, and doing them as *separate* steps means the AI system won’t abandon findings *too* quickly.

5.5 Triage

Most projects must perform *triage*; that is, they must prioritize reports.

Ideally, all vulnerabilities would be fixed immediately, but when there’s a lot to do, that’s often impractical. When handling everything immediately is impractical, findings must be prioritized so the most important vulnerabilities are addressed first. This is best done after validation, though if the validation process is overwhelmed, you may need to do some triage before validation.

Triage is fundamentally a risk decision. Risks, by definition, are important based on their:

1. *Likelihood*. In particular, consider the preconditions necessary for the vulnerability to be exploitable and what is required for an attacker to exploit it [Yan2026]. A vulnerability that an

unauthenticated attacker can exploit remotely is more likely to be targeted than one that requires a local authenticated user.

2. *Impact*. A remote code execution (RCE) vulnerability is typically considered an *extremely* high impact because often an attacker can make the system do almost anything. In contrast, a vulnerability that can only reveal public or unimportant information would have low impact.

Most systems have *some* sort of "severity rating" classification system to help you identify the vulnerabilities most needing to be addressed [Rogers2025]. Unfortunately, many report that AI systems aren't very good at estimating the severity of a vulnerability (they often report both too high and too low), so if you use an AI to estimate in a way that matters, have a human review the estimates.

You can make AI severity estimates more reliable by giving the AI your threat model and having it answer specific questions building on that before it assigns a severity.

- [Yan2026] suggests asking about *reachability* (can an attacker reach this code from a real entry point?), *attacker control* (does untrusted input reach it intact?), *preconditions* (does it need non-default settings or a narrow timing window?), *authentication* (can an unauthenticated attacker trigger it?), *read versus write* access, and *blast radius* (who is affected?).
- They suggest that "zero preconditions with unauthenticated remote access is critical or high severity. One or two preconditions, or an authenticated path, is medium. Three or more, or local-only, is low. Adjust the thresholds to your system."
- That report argues that "going through the evidence first keeps the model from anchoring on the bug class ('SQL injection, so critical') and then inflating the severity to match."

Modern AI systems have become increasingly good at chaining many defects *together* into vulnerabilities. Something that *appears* unexploitable may, when combined with other defects, constitute a serious vulnerability. Thus, it's reasonable to triage the "most dangerous vulnerabilities" first, but do *not* ignore other defects that *appear* unexploitable. Fix the other defects, as resources permit, so they don't become part of a chain leading to an exploit.

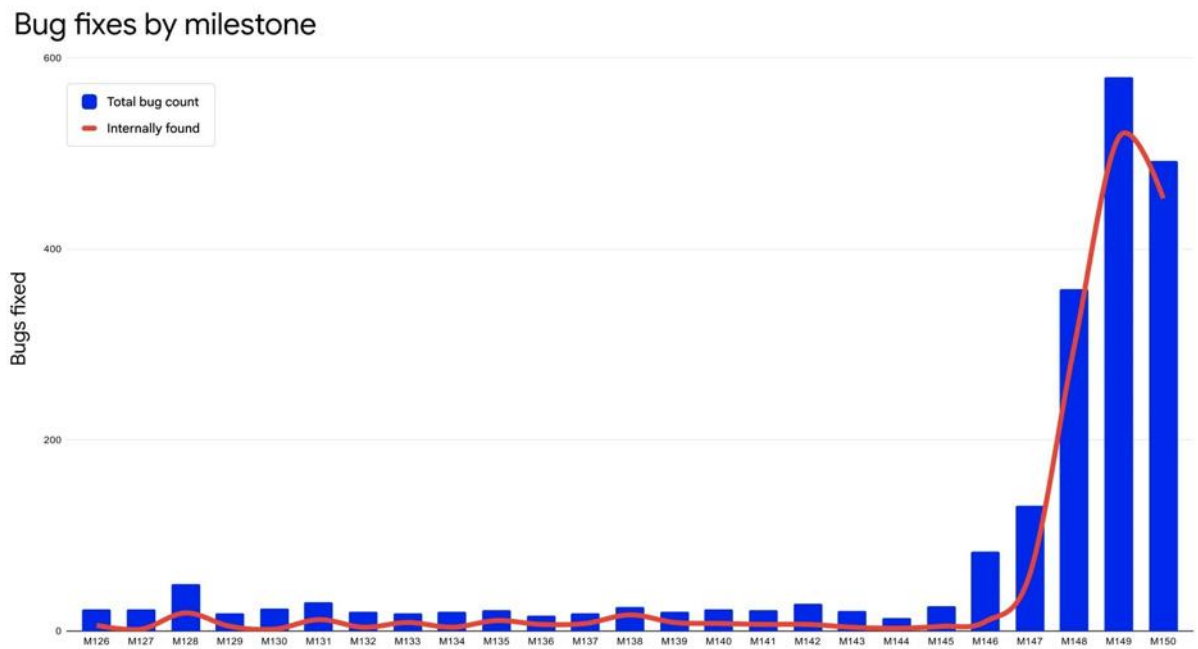
Don't assume an AI-found vulnerability will stay unknown for long. When many people use similar AI tools on the same code, they often find the same bugs around the same time. The Linux kernel security team reports that bugs found with AI assistance "systematically surface simultaneously across multiple researchers, often on the same day" [Linux-SecurityBugs].



5.6 Fix vulnerabilities

Once vulnerabilities are found, those vulnerabilities need to be *fixed*.

Finding vulnerabilities is useless for a defender unless those vulnerabilities are fixed. If vulnerabilities are not fixed efficiently, it can become completely overwhelming. For example, here is the number of bug fixes in the Chrome browser, showing a huge rise in 2026 (compared to 2024-2025) caused by AI-discovered defect reports [Chrome2026]:



Firefox had a similar experience. Mozilla reported, “We fixed a total of 423 security bugs in releases in April. In addition to the 271 bugs announced two weeks ago, there were 41 externally reported bugs, with the remaining 111 discovered internally” [Grinstead2026-05].

In many ways, a web browser is a worst case, since it has a massive amount of functionality and must directly interact with potentially malicious websites. Still, this experience demonstrates that AI can identify many vulnerabilities not found otherwise, and that you need to be ready to fix far more vulnerabilities than before.

5.6.1 Fixing may be necessary for sending reports to external organizations

If you are sending reports to an external organization, particularly an open source software project, you may be *required* to propose a fix.

Historically, if a finder was examining external software, they would often simply report a vulnerability to that external project and let the project fix it. However, in the new world of AI, simply reporting a vulnerability is often not very helpful; many projects are inundated. If an AI was used to find a potential vulnerability, many recipients will expect both a PoC (showing it’s a vulnerability) and a specific proposed fix that would fix it.

As reported in [TrailofBits2026-06], “Anyone can file an issue, flex, and walk away. We showed up with the patches… [and go] beyond just fixing bugs: we’re adding new tests and fuzzing harnesses, CI security scanning, supply-chain tooling, correctness fixes, and features maintainers had been meaning to get to.” In short, providing a proposed fix is more likely to be helpful than showing up with only a complaint.

5.6.2 Assigning the fix

A fix will only be developed if someone is assigned, tacitly or explicitly, to do so.

In small projects, with few developers or fixes to be done, this assignment is often obvious. In a larger project, even the assignment process can be overwhelming.

Thus, in a larger project, consider using automatic assignment to handle the tsunami of reports. If a system can automatically route the issue to the correct component and human owner, this can be a huge help [Chrome2026]. This doesn’t need to be complex; an AI system can often estimate this. If the assignment is in error, or the human is overwhelmed, have a process for transferring it.

5.6.3 Creating tests and confirming the root problem

Before creating a fix, be sure to create a test and confirm the root problem.

“Write a new test that fails with the existing code. Then, implement the fix and confirm the same test now passes without breaking anything else. (Yes, it’s test-driven development). If you don’t add a test, the fix can silently regress and it can be hard to retroactively prove the bug was real.” [Yan2026] After all, this mistake has happened before; a focused test will increase the likelihood that this specific problem won’t recur.

In addition, examine the system to find the *root cause*. “Models may narrowly address findings at a specific call site instead of the root cause. Simply prompting the model to identify and fix the root cause can be effective. Then, have the model look for variants at two levels: (1) same pattern, where there are other call sites or copies of the same buggy code elsewhere, and (2) same class, where a codebase with one SQL injection vulnerability tends to have more SQL injection vulnerabilities. Update the threat model with the validated findings and patches to close the loop.” [Yan2026]

5.6.4 How to create the fix

Now you’re ready to create a fix. You might choose to use AI to help develop a candidate fix, but you do *not* need to use the same AI that found the vulnerability. Many people find vulnerabilities using a model that’s expensive, slow, or restricted. It might be better to use a different AI system to create the fixes, since it might not have the same issues, it can use the detailed information provided earlier, and it might be better at that task.

There’s a tradeoff here. The Linux kernel’s procedure notes that “fixes written in the same session as used to find the bug will generally lead to better and more accurate fixes as the LLM’s reasoning context remains present” [Linux-AI]. On the other hand, that context may include the finder’s incorrect guesses, as discussed next. Either way, give the fixing AI the verified evidence, and verify the result.

5.6.5 Only provide correct information, especially if fully automated

Only provide verified *correct* information to the AI, *especially* if the AI process is fully automated.

The phrase “garbage in, garbage out” applies to AI systems, and is *especially* true if you are trying to use AI to create fixes for vulnerabilities. Be careful to *only* provide correct details. Avoid including data that might be incorrect in such prompting. In particular, try to pass along the details that validation confirmed (such as the reproducing input), not the finder’s unverified explanation of the cause or how to fix it.

One study found that “giving incorrect guidance to the model in its initial prompt [to create a vulnerability fix] resulted in a roughly 50 percentage point reduction in fix correctness rates [while] giving more correct details to the model increased correctness by only 15 percentage points compared to no specific guidance at all. As such, in cases where one cannot be highly confident in the accuracy of bug details or fix guidance passed to an LLM for patch generation (e.g., when that

data is sourced directly from other automated tooling), the safer bet may actually be to omit lower-confidence information that could cause a ‘correctness collapse’ if it turns out to be wrong.” [Mierczuk2026]

5.6.6 Approaching fixes

When working with an AI to develop a vulnerability fix, some general guidelines will increase the likelihood of success.

As much as possible, be clear in the prompt to create a fix. Here are some general prompt statements you may find helpful: “Identify the root causes of this vulnerability. Identify approaches for fixing the root causes of the vulnerability, not just this specific example, so it is no longer a vulnerability. Discuss options for fixing it, along with pros and cons. Remember that the goal is to ensure an attacker cannot exploit the vulnerability along any path, not simply some paths. When generating code, try to reuse existing code, minimize new code, and, where practical, work within the existing system. Ensure the resulting change is easy to review. Ensure that existing functionality is maintained where practical and that you do not insert any new vulnerabilities. Update corresponding documentation. Use tools to obtain and validate information. The result must pass all CI/CD tests.”

When using an AI to fix defects, enable it to “interrogate, validate, and raise disagreements about the information stated to them in their initial task prompts.” [Mierczuk2026] AI is much less likely to succeed if it contains incorrect information or is missing key information, so it’s important to equip it with tools that increase its likelihood of success.

It’s vital to work with the AI to identify the true underlying cause and fix *that* while not breaking functionality. Another study found that the most common failure when creating fixes was that the AI often guarded the *symptom* with incomplete local checks. AI systems often fail to localize the vulnerability causing malformed behavior, or to silence the crash by deleting the offending operation or even the entire functionality. In some cases, the AI will add broader conditions that reject invalid input but also benign inputs, causing functional breaks [Shen2026-09].

The Chrome team’s process is instructive: “We run a fixing agent that returns multiple candidate fixes. A critic agent then evaluates which would be the best fit and produces other relevant artifacts for developers to evaluate the fix. The fixing and critic agents work in a loop that mimics a typical code review process to ensure that code is functional and compliant with [our] style guidelines [and] other local code conventions.” [Chrome2026]

However, be wary of long, unreviewed loops where an AI repeatedly “improves” its own code.

- One study ran 400 code samples through 40 rounds of AI-only “improvements” and found “a 37.6% increase in critical vulnerabilities after just five iterations” [Shukla2025].
- A 2026 study of AI repair of infrastructure-as-code (Terraform), where validator errors were fed back to the model over up to five iterations, found that in 13.8% of scenarios a previously passing security check later failed. Only 3.3% were unambiguous, but the authors believe this

is an under-count. Regressions were strongly associated with large rewrites, and compliance stopped improving after about three iterations: “Three iterations capture most of the compliance improvement while keeping regression risk manageable” [Agyekum2026].
- Limit the number of iterations, prefer small changes, and bring in a human when the AI doesn’t converge quickly.

5.7 Verify fixes

After generating candidate fixes, have AI and then humans review them to verify them.

This is useful for a fix by a human *or* by an AI. However, it’s especially important for AI-created code. Even very good AI writes vulnerable code. In particular, an AI can be good at finding vulnerabilities yet still write code with lots of vulnerabilities. This might be surprising, but remember, AI is trained on a large amount of insecure software. It’s more difficult to get an AI to do something *contrary* to its training dataset [CSA2026] [Zimmer2026]. Modern AI is fundamentally probabilistic, so it can be difficult to predict exactly what it will generate for a given request [Kholoosi2025].

5.7.1 Have AI review the fix first

The first step is to have an AI review the proposed fix for the vulnerability. “Have a new discovery agent probe the patch as an attacker to confirm the patch is comprehensive” [Yan2026]. What can be especially helpful is to have AI write tests for the fixes. For example, [Chrome2026] reports that they use test-writing agents to “help write tests for fixes. These agents can ensure that tests work across supported platforms and configurations before a developer reviews the fix, saving up to weeks of developer time.”

Run the validated PoC against the fixed version and confirm that it no longer succeeds. Then have a separate agent try variations of the PoC, since a fix that blocks only the exact PoC input isn’t a full fix.

5.7.2 Have humans review the fix

While AI review is a good first pass for proposed vulnerability fixes, they still require expert human review. As we noted earlier, AI-generated fixes often are incomplete, add vulnerabilities, or break functionality.

Some problems with initial AI-proposed fixes are especially common, and humans should especially look out for them:

- *AI often fixes only the specific instance, rather than identifying root causes and fixing the problem systematically*. “Patches focused too much on immediate consequences of issues, and generally failed to put the whole application and codepath into perspective, and ‘understand’

the real source of the problem, and where an engineer with a clue would actually solve the problem rather than mitigate it." [Rogers2025]

- *AI often fails to customize the fix and reuse existing constructs*. "For fixes, one of the most common issues was that the fix was not customized to the developer's codebase, for example, creating a function to sanitize user inputs when the developer wants to reuse their existing sanitization library; this often prevented the users from directly applying the fix, requiring an overhaul to produce a fix with their intended approach." [Steenhoek2025] In short, for an AI system it's "easier" to write lots of duplicate code, but this typically creates a maintenance nightmare.
- *AI sometimes adds dependencies that don't exist, or aren't the ones you think*
    - Attackers can register these hallucinated names and wait for someone to install them ("slopsquatting"). If a proposed fix adds or changes a dependency, confirm that it's the real, intended package and that you actually need it.
    - In a 2025 study of 576,000 code samples from 16 LLMs available in 2024, "the average percentage of hallucinated packages is at least 5.2% for commercial models and 21.7% for open-source models" [Spracklen2025].
    - A 2026 replication on five newer models found rates from 4.62% to 6.10%; the only open-weight model analyzed was DeepSeek V3.2, whose hallucination rate was 5.89%. So the open-weight model improved dramatically over the older average, while commercial closed rates stayed about the same. Its author called this "an order-of-magnitude compression of the inter-model spread observed by Spracklen, but not a retirement of the threat" [Churilov2026].
    - Another 2026 study found no hallucinated package names in the imports of 1,134 short pieces of Python code that six models (including two small local open-weight models) generated for everyday coding tasks with mainstream libraries. However, when prompts asked for niche or nonexistent libraries, every model wrote code importing packages that don't exist, and the samples were too small to show that the rate is zero even for everyday tasks [Hillah2026].
    - We expect package hallucination to keep declining as models improve, but we don't expect it to disappear.

Even if an AI's initial fix is wrong, that doesn't make its proposed fix useless. One reporter found AI fixes "most useful for simply understanding what the problem actually was in the code – sometimes I didn't understand the issue from the description, but the suggested fix revealed to me what was wrong, and what would (could?) fix it." [Rogers2025] Sometimes the proposed fixes shouldn't be the final fix, but they can still provide guidance to help developers find a correct solution.

5.7.3 Apply your usual verification processes

Be *sure* to apply all of your usual verification processes. That includes your peer review gates, CI/CD pipelines, and branch protection rules. "Leveraging organizational safeguards such as peer-review gates and branch protection rules can help mitigate potential individual complacency regarding AI-generated security suggestions" [Kholoosi2025].

Technology can help. However, as noted in [Anthropic2026-05], now "the bottleneck in fixing bugs like these is the human capacity to triage, report, and design and deploy patches for them."

5.7.4 Turn findings into durable checks

AI analysis is non-deterministic; don't depend on rerunning it to catch the same problem again. Instead, turn what you learned into deterministic checks that run on every change, as part of the change.

At the least, modify the test suite to add the regression test you created earlier to verify the problem. You may also want to add:

1. A static analysis rule that detects the vulnerable pattern (e.g., a Semgrep rule or CodeQL query).
2. A fuzz harness for the code involved, if it's the kind of code fuzzing handles well (e.g., parsers).
3. A hardening or API change that makes the whole class of vulnerability impossible or unlikely (see the upcoming section on hardening).

AI can often help implement all of these. The general principle is that if something is important, enforce it with deterministic mechanisms *instead* of relying on an AI to notice it every time.

5.8 Report vulnerabilities

There are three kinds of vulnerability reporting we want to discuss: to the people who maintain a component, to authorities when a law requires it, and to your own users.

5.8.1 Reporting to externally maintained projects

If the vulnerability is in an *externally maintained* component, report it to the project that maintains that component. Use the vulnerability reporting process and rules defined by that project, in whatever form the project prefers. In many cases this involves working with others in a process called "coordinated vulnerability disclosure".

Here's how to make those reports useful.

First, take steps to prevent reporting AI slop. "AI-assisted bug reports have a mixed track record, and skepticism is earned. Too many submissions have meant false positives and an extra burden for open source projects." [Grinstead2026-03] The best AI systems have improved, and some perform human review before sending reports, but many projects still receive a lot of useless AI slop.

Early in the process, read that project's threat model and their guidance on submitting vulnerability reports (e.g., SECURITY.md). If you're using AI to help, also have the AI read the project's threat

model and reporting guidance before drafting a report. The Linux kernel project specifically recommends telling the AI to do this [Linux-SecurityBugs].

Provide *value*. Include the specific reproducing inputs that *demonstrate* that a report is a vulnerability (as discussed earlier), test cases that enable easy verification and regression testing, and a proposed fix. Have a human review it all before submitting it. These measures, especially a way to reliably reproduce the vulnerability with plausible inputs, help provide confidence that the report is not merely "AI slop" or a false positive.

This is an example of the human gate we discussed earlier: treat high-impact or externally visible actions as explicit approval boundaries. An AI agent may be allowed to autonomously analyze software and prepare proposed actions. At the same time, operations such as submitting vulnerability reports, publishing information, merging changes, deploying fixes, modifying production systems, or accessing especially sensitive resources may require separate authorization. Where approval is required, enforce it in the tool or workflow rather than relying only on an instruction in the AI prompt [OWASP-LLM06].

When making a report on a component, briefly and clearly state the deployment model and security threat being assumed. Supply this *early* in the report. Often, when AI writes the report, neither the code nor the documentation tells it which deployments are supported. Many AIs will make the most permissive assumption, even a patently absurd one, and that assumption will make any finding look especially severe. Once you've clearly stated the deployment model and security threat, you or the AI can then attempt to assign a severity. A maintainer who disagrees with the claimed deployment model or security threat can correct that line rather than reject the report without explanation.

Before you report, *check* that the *problem still exists* in the project's current code. The Linux kernel security team says that "A significant part of reports are for bugs that have already been fixed". They ask reporters to verify them on recent versions and to identify the affected version with "A stable identifier such as a commit ID or an exact version", not something like "latest". It also asks reporters to "stick to verifiable facts" (not to enumerate "speculative implications") [Linux-SecurityBugs].

A significant challenge can occur if the component's maintainers expressly *do not* support the reporter's threat model. In that case, the reporter can try to:

1. Convince the maintainers to expand their threat model. Typically this involves providing the resources to do it, such as developing proposed changes and/or agreeing to join as a co-maintainer.
2. Fix it and submit it as a normal (non-security) defect as a short-term measure, *and* change your own system's design so it no longer requires a security property the component *does not support*.
3. Use a different or additional component for the functionality that needs that security property.

Here's a concrete example. The Linux kernel threat model (2026) expressly says "mounting a block device [like an external USB stick] is a privileged operation… and the administrator is responsible for the media they mount." ChromeOS uses the Linux kernel, but ChromeOS also supports mounting an

external USB stick that is *not* fully trusted (while presuming that it's passive media). Yet the Linux kernel itself expressly does *not* support this activity. Instead of ignoring the Linux kernel's threat model, when ChromeOS mounts external untrusted media, it uses a different system based on Filesystem in Userspace (FUSE) rather than relying solely on the Linux kernel's built-in mechanisms. In short, the ChromeOS developers carefully chose how to use available components; they used the Linux kernel as the kernel, but also selected *different* components when they needed a special security property not supported by that kernel. No single piece of software can do everything! Many problems require multiple components working *together* to achieve the desired result.

Follow the project's rules for AI-assisted contributions; more projects have them now. The Linux kernel's rules are a good example, and include a procedure for finding and fixing bugs with AI [Linux-AI]. They say that "AI agents MUST NOT add Signed-off-by tags. Only humans can legally certify the Developer Certificate of Origin (DCO)"; the human submitter must review all AI-generated code, take full responsibility for the contribution, and credit the AI with an "Assisted-by:" tag. The procedure requires trying to create a reproducer for any nontrivial bug, writing and testing a fix ("This part is not optional"), and stating what couldn't be done: "If the fix could not be built or tested, or if no reproducer could be produced, say so explicitly: maintainers currently waste too much time analyzing unverified reports and untested fixes." It also says that the assistant "must never send anything itself"; a human decides.

Obviously, don't provide a fix if the project asks you not to propose a fix. Most projects *are* interested in receiving proposed fixes if you have a good one.

Often a project will not directly accept a proposed fix [Zimmer2026]. Proposed fixes from outside the project often fail to properly reuse a project's existing methods, templates, and other mechanisms, creating long-term maintenance issues if they are simply accepted as-is. Project maintainers often have longer-term roadmaps that a proposed fix won't precisely meet. Many projects have unwritten rules and expectations. That doesn't mean a proposed fix is useless, however. A proposed fix is often a necessary and helpful step in creating the final fix. So don't feel discouraged if the final fix is different; in many cases a proposed fix is simply a useful draft that helps speed development of the final fix.

One source of debate is how large the vulnerability report should be. Many maintainers don't want to receive massive tomes; they want something they can read quickly. Yet if the report lacks detail, it's not helpful. One solution is to put the report's essence into a single paragraph (6 sentences or fewer) and lead with it. The receiver can then read that and decide if it's worth reading further. When reporting to a project, *find out* what format the project wants and comply with it. In general, *respect the maintainers' limited time* [Larson2026].

Many open source software projects are receiving an *overwhelming* number of duplicate reports [PSF2026]. Often, many different people will use the same AI system to look for vulnerabilities in the same software, resulting in many duplicate reports to a given project. Linus Torvalds said, "if you found a bug using AI tools, the chances are somebody else found it too" and added, "If you actually

want to add value, read the documentation, create a patch too, and add some real value on *top* of what the AI did" [Zorz2026].

The Linux Foundation has established a project called Akrites <https://akrites.org/> to help organizations deduplicate findings *before* they are submitted to open source software projects, as well as validate, prepare remediations, and synchronize disclosure. If your organization finds vulnerabilities in widely used open source software, see the Akrites site for more information.

5.8.2 Reporting to authorities

So far we've discussed reporting vulnerabilities to the people who maintain a component. Some laws also require reporting to authorities. In particular, the European Union (EU) Cyber Resilience Act (CRA) includes a variety of requirements for manufacturers of products with digital elements distributed in the EU, as well as for open source software stewards. Let's briefly note the CRA requirements, as an example of reporting to authorities that's widely applicable.

The CRA treats *actively exploited vulnerabilities* differently from other vulnerabilities found. If you learn that there is an *actively exploited vulnerability* in software you maintain, or a *severe incident*, the CRA imposes strict reporting requirements that apply to manufacturers since 2026-09-11. These include short time windows; an initial report must be made within 24 hours.

The CRA does not require that software have no unknown vulnerabilities. After all, if the developers don't know about a vulnerability, they can't fix it. However, the CRA does impose several requirements on manufacturers of products with digital elements. For example, the CRA's Annex I requires that such products "be designed, developed and produced in such a way that they ensure an appropriate level of cybersecurity based on the risks", that they "be made available on the market without known exploitable vulnerabilities", that they be "made available on the market with a secure by default configuration [in general]", and that they "ensure that vulnerabilities can be addressed through security updates".

The CRA also imposes several requirements related to identifying vulnerabilities, for example, "identify and document vulnerabilities and components contained in products with digital elements…", "address and remediate vulnerabilities without delay", and "apply effective and regular tests and reviews of the security of the product with digital elements" [CRA-AnnexI].

For more information, see our course "Understanding the EU Cyber Resilience Act (CRA) (LFEL1001)".

5.8.3 Reporting to users

Usually, don't tell users about a vulnerability in software you maintain until a fix is available. Announcing it earlier mainly helps attackers, since users can't yet protect themselves. That's why coordinated vulnerability disclosure keeps a vulnerability private until a fix is ready.

Sometimes you need to tell users before a fix is available, e.g., if the vulnerability is already public or being actively exploited. In that case, say which versions are affected, how users can reduce their risk until a fix is ready (e.g., a workaround or configuration change), and when you expect a fix, without details that would help attackers. For an actively exploited vulnerability, the CRA may require manufacturers to inform the users affected.

5.9 Release and deploy

Software that is only fixed on a developer's workstation helps no one else. Fixed software must be *released* and *deployed*. This was always true, but now that AI is accelerating the vulnerability-finding process, it's even *more* important.

Anthropic notes that "software developers should… make security fixes available as quickly as possible… Developers should also help their users stay up-to-date with their software by making it as easy as possible to install updates; to the extent feasible, they should be more persistent with users who are still running software with known vulnerabilities" [Anthropic2026-05]. Work to streamline your release process. Ensure that updates are automatically checked for a valid digital signature and that the update process is not *itself* a vulnerability.

A released fix is also a map to the vulnerability. Attackers can "patch diff", that is, compare the fixed source or binary code with the previous version, find what changed, and work out the vulnerability the change fixes. While this was always true, AI has made this fast and cheap. In a June 2026 study, Claude Mythos Preview built 8 working code-execution exploits from 18 recent Firefox security patches, writing "its first working exploit in just under one hour". From 21 Windows kernel vulnerabilities it produced 8 full privilege-escalation exploit chains. The authors warned that "A lone operator can now turn a month's worth of patches into working exploits in a single afternoon—for a few thousand dollars and with no specialized expertise". The authors concluded that "the typical patching playbook that software developers use today—with monthly release cadences, multi-week staged rollouts, and a lag between pre-release and stable channels—no longer holds". [Xiao2026].

In practice, this means you should:

1. Minimize the time between a fix becoming public and its release. A security fix that sits in a public repository for weeks before release gives attackers a head start. Where practical, develop security fixes privately (e.g., in a private fork or a private security advisory). Publish the fix, the release, and the advisory together.
2. Release fixes for all supported versions at the same time. If you fix the main branch today and a long-term-support branch next month, the first fix shows attackers how to attack the second.
3. Shorten your release cycle for security fixes. For example, Mozilla "recently moved its 'dot' releases (the small point updates between major versions) from a monthly to a roughly weekly cadence" [Xiao2026].
4. Make updating easy and fast for your users, as discussed above.

Faster patching alone does *not* solve this problem. As the Xiao study notes, “A more durable fix would attack the supply of bugs, rather than the speed of patching them. This can start with migrating critical components to memory-safe languages like Rust, or hardening them with mitigations that retire whole exploit classes at once” [Xiao2026]. See our section “Harden” about this.

When you release a fix, tell your users, so they know to update. Publish a security advisory that says what the vulnerability is, which versions are affected, which version fixes it, and what users should do. Where appropriate, get a CVE identifier for it, so tools that check for known vulnerabilities can alert your users.

Writing the advisories can become a bottleneck when many fixes arrive quickly. Consider using AI to draft release notes, advisories, and CVE descriptions from the fixes, with human review and repair before publication. The Chrome team is “working on automating the generation of release notes and CVE descriptions from security bug fixes to eliminate manual bottlenecks and shorten the window between vulnerability discovery and public disclosure” [Chrome2026].

People, projects, and organizations should ensure that they can rapidly *accept* updates, too. Establish processes to automatically test updates, and then propose or implement them. Organizations that try to do this solely with manual processes will often be unable to keep up. Humans still need to be in control, but the humans need to be supported by automated processes that help them become aware of problems and provide analyses so they can make good decisions.

5.10 Repeated application

As we noted earlier, in most cases you should repeatedly apply AI systems to find and fix vulnerabilities.

Most AI systems are non-deterministic; re-running the same AI system with the same inputs can discover new vulnerabilities, especially in the first few iterations. In addition, AI systems will continue to improve, and new capabilities may find new problems.

So you need to repeatedly apply AI systems, especially improved ones, to find vulnerabilities that were previously missed by other passes. As vulnerabilities are found, search for similar patterns elsewhere, and ensure you really *did* fix the problem systematically.

The good news is that improved hardening and fixing vulnerabilities will eventually eliminate all “easy vulnerabilities” and make the system increasingly difficult to attack. Software is finite, and so are defects. Take heart: as it gets harder for software developers to find vulnerabilities, it also gets harder for attackers to find them.

In practice, software changes over time. Deterministic checks, including those created from earlier findings, should run on every change. An AI system should also analyze each proposed change (pull request or merge request) toidentify vulnerabilities *before* a change is accepted.


Pass: 5

6 Preventing vulnerabilities longer term

Finding and fixing specific vulnerabilities isn't enough. You also need to take systematic steps to prevent them.

Looking for vulnerabilities and quickly fixing them is like stopping the bleeding of a badly hurt patient. It's important to do, but it's also important to prevent vulnerabilities long term. Thankfully, AI can help do that as well.

6.1 Limit vibe coding

The practice of using AI to generate code and *not* having a human review the results is called *vibe coding*. If the results are important for security, don't do that.

Modern AI has become far better at generating code. However, AI often generates insecure code. If the code can't have a security impact (e.g., it doesn't process untrusted input), vibe coding may be fine. However, if its failures may cause serious problems, don't accept AI-generated code without review.

The increased use of vibe coding in security-relevant code has led to an explosion in the number of *vulnerable* code releases. "Researchers at Georgia Tech's Vibe Security Radar tracked CVEs directly attributable to AI coding tools and found that March 2026 alone produced more than all of 2025 combined." [Holterhoff2026]

Stay skeptical. In a controlled study, participants with access to an AI assistant "wrote significantly less secure code than those without access" yet "were more likely to believe they wrote secure code" [Perry2023]. Participants who trusted the AI less and worked more on their prompts produced code with fewer vulnerabilities.

You can reduce the number of vulnerabilities AI code assistants generate in the first place by giving them security-focused instructions; see the OpenSSF "Security-Focused Guide for AI Code Assistant Instructions" [OpenSSF2025-AIInstructions].

Where security matters, you need to have AI look for vulnerabilities and then have human review of that AI-generated code. You *can* use AI to generate code! Generating code with AI, especially code that doesn't process untrusted inputs and doesn't need good performance, can save time. The problem arises if you unwisely believe that AI-generated code never has defects or vulnerabilities.

Someday this review may no longer be necessary, or perhaps it can be completely automated. We can't predict how good AI will get over time, or at what rate. However, at the time of this writing, human review is necessary if failures can cause serious problems.

6.2 Evaluate merge/pull requests

In all cases, whether code changes were created by AI or by humans, look for vulnerabilities in each merge/pull request.

Here are some tips:

- Have AI and a human review the proposed change before it's even submitted
- Use tools that can scan pull/merge requests and report on them, as discussed in "How to apply processes for using AI to find and fix vulnerabilities". Some tools specialize in this, such as Sashiko for the Linux kernel.
- "Enforce automated security assessments consistently in your development processes, including using LLM-powered agents to find vulnerabilities before attackers do" [CSA2026]
- "All code (human or AI-generated) should pass LLM-driven security review before merge." [CSA2026]

Be careful when you run AI agents in CI/CD on proposed changes. When a change comes from an untrusted contributor, its title, description, comments, commit messages, and code are all attacker-controlled. An AI agent treats all of that as input, so an attacker can try to use it to give the agent new instructions (prompt injection).

This isn't hypothetical. In 2025, Aikido Security reported "PromptPwnd", where GitHub Actions and GitLab CI/CD workflows embedded information from untrusted issue, pull request, or commit into AI agent prompts. They reported that "At least 5 Fortune 500 companies are impacted" [Daelman2025]. In 2026, Aonan Guan and colleagues showed that a malicious pull request title, issue comment, or hidden HTML comment in an issue could make Anthropic's Claude Code Security Review, Google's Gemini CLI Action, and GitHub's Copilot Agent leak the repository's own API keys and access tokens [Guan2026]. Note that one of those was a security review tool. As Guan explains, the problem is that "these AI agents are given powerful tools (bash execution, git push, API calls) and secrets (API keys, tokens) in the same runtime that processes untrusted user input."

In the 2025 "s1ngularity" attack, compromised versions of the Nx build package ran any installed Claude, Gemini, or Amazon Q command-line agent on a developer's system with its permission checks disabled (e.g., --dangerously-skip-permissions) and used it to search the developer's system for secrets. StepSecurity called this "the first known case where attackers have turned developer AI assistants into tools for supply chain exploitation" [Kurmi2025]. As we noted earlier, it's important to run agents inside a sandbox.

If you run an AI agent on untrusted proposed changes:

1. *Don't give that agent secrets or write-capable tokens*. By default, GitHub Actions doesn't expose secrets to pull requests from forks, but some workflows grant that access anyway, e.g., by using the pull_request_target trigger [Guan2026]. Don't do that for AI agents.
2. *Give a review agent the fewest tools practical* [Daelman2025]. A reviewer of a proposed change usually needs to read code and produce a report; it rarely needs a shell, network access, or permission to push, merge, or edit issues. An agent used to *find* vulnerabilities may need more capabilities, but reviewing a proposed change from an untrusted proposer is different.
3. *Where an agent needs credentials, use short-lived, narrowly scoped ones*.
4. *Treat the agent's output as untrusted data*. Don't let it automatically trigger privileged actions; put a human gate in front of them (see "Establish a human gate and kill switch").
5. *Separate analysis from action*. A low-privilege agent can analyze and report, while a *different*, trusted process (or a human) should decide what to do with that report.



6.3 Apply secure by design and secure by default

Only trying to find and fix software vulnerabilities one at a time will not succeed in the long term. It's *necessary*, but future changes might re-introduce other vulnerabilities. So, also apply the general principles of secure-by-design and secure-by-default.

As always, if a system is to be secure in the real world, it must be:

- Secure-by-design: Design the software to be secure from the outset.
  - For example, limit its attack surface and apply defense-in-depth [FiveEyes2026].
  - Ensure that security checks run where attackers can't bypass them (e.g., on the server, not only in the client, if an attacker can control a client). See OWASP ASVS 5.0 requirements V2.2.2 and V8.3.1 for their discussion on input validation and authorization [OWASP-ASVS5].
- Secure-by-default: Release the software so that it's secure by default during installation.
  - Most users install the unchanged default, so the default security *is* the security of most installations.
  - A "hardening guide" that users must apply after installation indicates insecure defaults; those steps should have been applied before release.
  - If necessary, provide a "loosening guide" explaining how to disable security mechanisms in special cases, including the risks involved.

Secure-by-design and secure-by-default "must become standard practice – not an aspiration" [FiveEyes2026].

6.4 Harden

A key aspect of secure-by-design is *hardening*, that is, modifying the system/software so that defects are *systematically* unlikely to be exploitable or have more limited impact. As much as practical, apply hardening.

Defects are inevitable, but they do *not* necessarily need to be vulnerabilities. If vulnerabilities *do* exist, it's best to constrain the likelihood of their exploitation and their impact as much as practical. The goal is to make it difficult for an attacker to *exploit* a defect, preventing defects from becoming serious vulnerabilities.



6.4.1 Applying hardening

Apply hardening, using as many different techniques as make sense in your situation.

Some hardening measures are well-known industry-wide. For example, some languages are "memory-unsafe", primarily C and C++. A memory-unsafe language does not, by default, protect against common memory errors such as reading or writing an array out of bounds. As a result, both humans and AI tend to produce more vulnerabilities in these languages. They're also more difficult to analyze later, leading to more false positives [Bourzikas2026]. There are hardening approaches for this situation:

- One approach is to rewrite that software in a memory-safe language such as Rust. If this significantly hurts performance, selected memory checks can be disabled where they're unnecessary *and* performance is critical.
- Another approach is to implement additional mechanisms to reduce the likelihood that an undetected memory-safety issue becomes a vulnerability. One example is applying the OpenSSF Compiler Options Hardening Guide for C and C++ [OpenSSF2026-06]. Projects like the Linux kernel create new APIs (such as strscpy) *specifically* to reduce the likelihood of vulnerabilities due to memory-safety issues.

Other software hardening measures are specific to a particular program, developed after examining lessons learned from that software. Mozilla reported that "in recent years we received several clever reports from security researchers that managed to escape the process sandbox by triggering prototype pollution in the privileged parent process. Rather than fixing these problems one-by-one, we made an architectural change to freeze these prototypes by default. While auditing logs from the

harness, we saw many attempts to pursue this line of escape that were thwarted by this design" [Grinstead2026-05].

Hardening works. Many well-run projects report that hardening measures *do* counter attackers, including powerful AI models. Mozilla reported that "just as interesting as what the models found is what they didn't find — not because they didn't try, but because they were unable to circumvent Firefox's layered defenses." [Grinstead2026-05] Similarly, Anthropic reported that Mythos Preview identified many areas of the Linux kernel that *appeared* to be vulnerabilities, yet "because of the Linux kernel's defense in depth measures Mythos Preview was unable to successfully exploit any of these." [Carlini2026]

6.4.2 Harden to counter chaining

It's always been good practice to harden software or systems; however, AI has made it even more important, as hardening can sometimes counter *chaining*.

Chaining refers to the ability of modern AI systems to combine multiple defects into an exploitable vulnerability. Skilled human attackers have long been able to do this, but the ability of AI systems to chain multiple defects into an attack makes chained attacks much cheaper and easier to execute.

However, *not* all hardening measures are as effective against chaining. There are different kinds of hardening, *reliable* and *unreliable*:

1. *Reliable* hardening mechanisms are those that *consistently* prevent certain kinds of defects from becoming vulnerabilities.
2. *Unreliable* hardening mechanisms are mechanisms that only make it *somewhat more difficult* to turn a defect into a vulnerability. That is, unreliable hardening mechanisms can be circumvented with additional effort, typically by performing a tedious task.

Both kinds can counter attackers who don't use AI. In many cases, a human attacker may decide that the second kind isn't worth the effort. However, modern AI dramatically reduces the effort required to carry out certain types of attacks.

Anthropic reported that, "We have nearly a dozen examples of Mythos Preview successfully chaining together two, three, and sometimes four vulnerabilities in order to construct a functional exploit on the Linux kernel… [modern AI requires rethinking of] measures that make exploitation tedious, rather than impossible. When run at large scale, language models grind through these tedious steps quickly. Mitigations whose security value comes primarily from friction rather than hard barriers… become considerably weaker against model-assisted adversaries" [Carlini2026]. This ability to chain defects together often defeats the "somewhat more difficult" hardening measures.

Do use *reliable* hardening measures, that is, mechanisms that *always* counter certain kinds of defects:

- For example, the “W^X” countermeasure ensures that memory regions are writable *or* executable, but not both. This countermeasure *systematically* prevents unauthorized writes to memory becoming directly executable.
- Parameterized statements *reliably* counter SQL injection; use them.

However, unreliable hardening mechanisms (mechanisms that merely make a defect “slightly more difficult” to exploit) are much weaker against AI. For example, “modern browsers run JavaScript through a Just-In-Time (JIT) compiler that generates machine code on the fly. This makes the memory layout dynamic and unpredictable [so that exploiting the vulnerabilities is more difficult for humans, yet] Mythos Preview fully autonomously discovered the necessary read and write primitives, and then chained them together to form a JIT heap spray [exploitation]” [Carlini2026]. These layout variations weren’t designed to counter attacks, but historically they protected systems anyway, because it was too much effort for most human attackers to counteract the variations enough to form an attack. However, an AI can determine the layout cheaply enough that these kinds of attacks can become practical.

Address space layout randomization (ASLR) is an interesting illustration of the difference between reliable and unreliable hardening measures. ASLR is a reliable measure only if (1) there are enough randomization bits that brute-force guessing is impractical for performing an attack, and (2) there’s no way for an attacker to reveal data that would enable them to speed the attack beyond a brute-force attack. Unfortunately, it’s often possible for attackers to obtain data that reveals the random value. If an attacker can determine the random value, then once again AI can chain attacks. We recommend continuing to use ASLR, as it’s usually an easily-enabled defense. However, because ASLR is quietly broken if an attacker can determine its random value, and attackers can often obtain this value, don’t count on ASLR alone as a defense.

Hardening still matters, but you need to use *reliable* hardening mechanisms.

6.4.3 Evaluate with hardening disabled and evaluate the hardening

Where sensible, try to *disable* a system’s hardening mechanisms when trying to find and fix vulnerabilities in that system. This helps implement “defense in depth” where practical.

The system should be designed so that, where practical, an attacker must defeat *multiple* mechanisms to exploit the system. If the AI system is asked to evaluate the system only with all its hardening mechanisms in place, it will often not report cases where a single hardening mechanism alone prevented an attack.

For example, in one analysis of Mozilla's Firefox, their testing environment "intentionally removed some of the security features found in modern browsers. This includes, most importantly, the sandbox, the purpose of which is to reduce the impact of these types of vulnerabilities" [Anthropic2026-03]. In this weakened environment, the AI "is tasked with developing an exploit" [Anthropic2026-04s].

In addition, have the AI look for *defects in the hardening mechanisms themselves*, such as looking for a "sandbox escape". The goal is to allow the model to craft an attack that the hardening mechanism should prevent, and ensure that the hardening mechanism works against active attacks. This kind of evaluation was *also* done with Mozilla Firefox, where the AI was "permitted to patch the Firefox source code, so long as the modified code is restricted to run only in the sandboxed process. Such bugs are notoriously difficult to find with fuzzing… AI analysis provides much more comprehensive coverage of this critical surface" [Grinstead2026-05].

Evaluate the system *without* its hardening mechanisms, and separately evaluate the hardening mechanisms *themselves*, to try to improve them both. If you improve both, the result is much stronger *defense-in-depth*. The goal is a system in which attackers often must break *multiple* mechanisms to succeed.

6.4.4 Harden the deployed environment and infrastructure

Organizations need to focus on security basics and harden their organization's environment and infrastructure. What organizations must do isn't new, but it now has a new sense of urgency.

Several government cybersecurity agencies recommended the following practical actions:

1. "Reduce your attack surface: Limit unnecessary system access and external connectivity. Challenge whether systems need to be exposed at all and isolate those that do not.
2. Accelerate patching processes: AI is shortening the time between vulnerability discovery and exploitation. Delays in patching increase risk, especially for operational systems with long update cycles. Prioritize security updates accordingly to manage risks.
3. Address legacy systems: Unsupported systems are easy targets. They are not just technical debt; they are strategic liabilities.
4. Review and strengthen identity and access controls: Limit who can access critical systems. Enforce strong authentication and regularly review permissions.
5. Prepare for incidents before they happen: Test response plans, train and prepare teams, and assume breaches will occur. Focus on fast containment and recovery." [FiveEyes2026]

Similarly, the UK NCSC recommends that organizations "put in place a policy to 'update by default' where you always apply software updates as soon as possible, and ideally automatically". Where organizations can't update everything, "they should prioritise applying updates to their external attack surfaces" [Whitehouse2026]. It's true that "update by default" is a risk; the update may break functionality, or an attacker may subvert an update or the update process. However, the risk from *failing* to update is usually far greater.

The CSA notes that the basics still matter: “segmentation, egress filtering, multifactor authentication, and defense-in-depth/breadth all increase the difficulty for attackers… the basics remain valid and can be prioritized for risks that can’t be easily mitigated. Implement egress filtering (it blocked every public Log4j exploit). Enforce deep segmentation and zero trust where possible. Lock down your dependency chain. Mandate phishing-resistant MFA for all privileged accounts. Every boundary increases attacker cost” [CSA2026]. It adds that “[minimizing] base operating system images, or replacing third-party libraries with framework primitives as they emerge over time” can reduce an organization’s attack surface, and that AI can help do this [CSA2026].
Many recommendations focus on identity. CrowdStrike reports, “identity sits at the center of this problem. Many successful attacks do not end with the initial exploit. They become dangerous when they allow an adversary to assume a trusted identity, obtain credentials, or abuse excessive privileges. … [Prevention] includes enforcing zero standing privileges, continuously verifying access, limiting credential exposure, and connecting identity posture to endpoint and workload context in Real-time.” [CrowdStrike2026-FiveSteps]

Attackers may still manage to slip in. Logging/internal telemetry, offline backups, and having a Continuity of Operations Plan (COOP) are as vital as ever.

Resilience is no longer optional. CrowdStrike states that cyber resilience is becoming foundational. “As exploitation windows shrink, rapid recovery, low-disruption patching, and containment without business interruption become core defensive requirements. In a frontier AI threat model, resiliency is no longer a differentiator layered on top of prevention. It is part of prevention.” [CrowdStrike2026-FiveSteps]

All of this requires continuously testing organizations’ security assumptions. “Controls that look strong on paper may fail in practice. Segmentation may not be enforced consistently. Privileged access may be broader than expected. Exposure management must become dynamic, evidence-based, and specific to the environment.” [CrowdStrike2026-FiveSteps]

However, there’s no need to *panic*. All of this was true before AI; it’s simply more important to *execute*. What’s more, even the most advanced AI models cannot simply create vulnerabilities where none exist. As Red Hat’s Gunnar Hellekson notes, “context renders many bugs useless [and] some ‘vulnerabilities’ identified by AI are actually functionality bugs with no meaningful exploit path. Many issues [are low risk] because the affected [components are rarely exposed to the internet].” [Hellekson2026] In short, AI may find many “vulnerabilities” as defined by some document, yet many won’t be significant in your context. Act promptly on the ones that are.

7 Call to action

AI is changing things, including security. This doesn't mean it's time to panic. It means it's time to work. Now.

AI can be used to help find and fix vulnerabilities. What's more, over time, AI can help software developers build software with so few vulnerabilities that it will be incredibly difficult to subvert. Software developers, and those who work with them, have an opportunity to make the world a much more secure place.

Apply what you've learned:

1. Pick your most important project. Once it's in good shape, repeat with the next.
2. Prepare to find and fix vulnerabilities: prepare the AI, its sandbox, and its logging, as well as the software's threat model, code and documentation, CI/CD processes, and dependency updates.
3. Find & fix vulnerabilities using AI. Apply this repeatedly, since only one or a few runs can miss important vulnerabilities.
4. Prevent longer-term problems through process improvements (such as evaluating every merge/pull request) and hardening (both of systems and the broader environment).

Let's get started.

8 Acknowledgements

We wish to thank all contributors and reviewers, including Georg Kunz, Jeff Diecks, Laura Guazzelli, Ryan Jennings, and Santosh Kumar Puppala.

9 Bibliography

- [0xkato2024] 0xkato, 2024-12-01, Get ready for an audit, https://www.0xkato.xyz/Get-ready-for-an-audit/
- [0xkato2026] 0xkato, 2026-06-01, “How LLMs Actually Work”, https://www.0xkato.xyz/how-llms-actually-work/
- [Agarwal2026] Abhinav Agarwal, 2026-04-21, “Refute-or-Promote: An Adversarial Stage-Gated Multi-Agent Review Methodology for High-Precision LLM-Assisted Defect Discovery”, https://arxiv.org/abs/2604.19049
- [Chen2026] Bo Chen, 2026-08-10, “From Runnable to Verifiable: An Independent Reproducibility Study of LLM/Agent-Driven Vulnerability Validation Artifacts”, https://arxiv.org/abs/2608.09567
- [Agyekum2026] Benjamin Agyekum and Fabio Santos, 2026-08-13, “Does Fixing Break Security? An Empirical Study of Security Degradation in Iterative LLM-Driven Infrastructure-as-Code Repair”, 20th International Symposium on Empirical Software Engineering and Measurement (ESEM 2026), https://arxiv.org/abs/2608.13404
- [Ahmad2026] Ahmad Osman, 2026, Anthropic’s War on Opensource AI, https://x.com/TheAhmadOsman/status/2065307070044234186
- [Alibaba2026] Alibaba, open-code-review README, https://github.com/alibaba/open-code-review
- [Aniszczyk2026] Aniszczyk, Chris (CNCF), David A. Wheeler (OpenSSF), Christopher “CRob” Robinson (OpenSSF), 2026-05, “Securing Open Source in the Age of AI”, https://openssf.org/resources/securing-open-source-in-the-age-of-ai-a-practical-guide/
- [Anthropic2026-04g] Anthropic, 2026-04, “Project Glasswing: Securing critical software for the AI era”, https://www.anthropic.com/glasswing
- [Anthropic2026-03] Anthropic, 2026-03-06, “Partnering with Mozilla to improve Firefox’s security”, https://www.anthropic.com/news/mozilla-firefox-security
- [Anthropic2026-04s] Anthropic, 2026-04-07 (actually 2026-04-08), “System Card: Claude Mythos Preview”, https://www-cdn.anthropic.com/08ab9158070959f88f296514c21b7facce6f52bc.pdf
- [Anthropic2026-06-12-Export-Control] Anthropic, 2026-06-12, Statement on the US government directive to suspend access to Fable 5 and Mythos 5, https://www.anthropic.com/news/fable-mythos-access
- [Anthropic2026-05] Anthropic, “Project Glasswing: An initial update”, 2026-05-22, https://www.anthropic.com/research/glasswing-initial-update
- [Anthropic-07-27] Anthropic, 2026-07-27, Our position on open-weights models, https://www.anthropic.com/news/position-open-weights-models
- [Anthropic2026-08-21] Anthropic, 2026-08-21, Bringing the cybersecurity capabilities of Claude Mythos 5 to more defenders, https://claude.com/blog/bringing-claude-mythos-5-to-more-defenders
- [Berkeley] Berkeley Vulnerability Initiative, https://vuln.cs.berkeley.edu/

- [Bishop2026] Bishop, Todd, 2026-05-13, “Microsoft’s multi-agent AI system tops Anthropic’s Mythos on cybersecurity benchmark”, https://www.geekwire.com/2026/microsofts-multi-agent-ai-system-tops-anthropics-mythos-on-cybersecurity-benchmark/
- [Bourzikas2026] Bourzikas, Grant, 2026-05-18, “Project Glasswing: what Mythos showed us” https://blog.cloudflare.com/cyber-frontier-models/
- [Bressers2025] Bressers, Josh, and Joshua Rogers, 2025-10-13, “Actually finding vulnerabilities using AI with Joshua Rogers”, Open Source Security Podcast, https://opensourcesecurity.io/2025/2025-10-ai-joshua-rogers/
- [Brown2026] Jarom Brown, 2026-05-15, “Raising the bar: Quality, shared responsibility, and the future of GitHub’s bug bounty program”, The GitHub Blog, https://github.blog/security/raising-the-bar-quality-shared-responsibility-and-the-future-of-githubs-bug-bounty-program/
- [Buttell2026] Amy Buttell, 2026-04-04, AI Code Risks Escalate: The use of AI coding tools continues to accelerate even as trust declines and risks proliferate, https://cacm.acm.org/news/ai-code-risks-escalate/
- [Capoot2026] Ashley Capoot, 2026-06-30, “Anthropic says Trump admin has lifted export controls on Claude Fable 5 and Mythos 5”, CNBC, https://www.cnbc.com/2026/06/30/anthropic-says-trump-admin-has-lifted-export-controls-on-claude-fable-5-and-mythos-5.html
- [Carlini2026] Nicholas Carlini, Newton Cheng, Keane Lucas, Michael Moore, Milad Nasr, Vinay Prabhushankar, Winnie Xiao, et al., 2026-04-07, “Assessing Claude Mythos Preview’s cybersecurity capabilities”, https://red.anthropic.com/2026/mythos-preview/
- [Carlini2026-youtube] Nicholas Carlini, 2026, “Black-hat LLMs”, [un]prompted 2026” https://www.youtube.com/watch?v=1sd26pWhfmg&t=316s
- [Carlini2026-02] Carlini et al., 2026-02-05, “Evaluating and mitigating the growing risk of LLM-discovered 0-days”, https://red.anthropic.com/2026/zero-days/
- [Chin2026] Andrew Chin, Dongkwan Kim, Yu-Fu Fu, Fabian Fleischer, Youngjoon Kim, HyungSeok Han, Cen Zhang, Brian Junekyu Lee, Hanqing Zhao, Taesoo Kim, 2026-03-25, “OSS-CRS: Liberating AIxCC Cyber Reasoning Systems for Real-World Open-Source Security”, https://arxiv.org/abs/2603.08566
- [Chin2026-slides] Andrew Chin, Brian Lee, 2026, OSS-CRS Tutorial: Bug-Finding and Patching for the LLM Era (slides), Systems Software & Security Lab (Georgia Tech) / Team Atlanta, https://github.com/ossf/oss-crs/blob/main/docs/slides/svcc-2026.pptx
- [Chrome2026] Chrome Security Team, 2026-07-30, Stronger with every update: How we’re making Chrome and the web safer in the AI Era, https://blog.google/security/chrome-stronger-with-every-update/
- [Catanzaro2026] Michael Catanzaro, 2026-06-08, https://blogs.gnome.org/mcatanzaro/2026/06/08/please-do-not-ban-ai-assisted-issue-reports/
- [Chia2026] Osmond Chia and Laura Cress, 2026-07-31, Anthropic’s Claude AI escapes to hack into three organisations, BBC, https://www.bbc.com/news/articles/cz7dl7w8y7po
- [Churilov2026] Aleksandr Churilov, 2026-05-16 (revised 2026-08-09), “The Range Shrinks, the Threat Remains: Re-evaluating LLM Package Hallucinations on the 2026 Frontier-Model Cohort”, https://arxiv.org/abs/2605.17062
- [CISA2026-CVD] CISA, NSA, JPCERT/CC, NCSC-NL, and NCSC-UK, 2026-07-15, “Establishing a Coordinated Vulnerability Disclosure Program to Work With Security

Researchers", https://www.cisa.gov/resources-tools/resources/establishing-coordinated-vulnerability-disclosure-program-work-security-researchers
- [CloudFlare] CloudFlare, "Can AI find vulnerabilities?", https://www.cloudflare.com/the-net/ai-vulnerabilities/
- [CrowdStrike2026-Global] CrowdStrike, 2026, "CrowdStrike 2026: Global Threat Report: Year of the Evasive Adversary", https://go.crowdstrike.com/2026-global-threat-report.html
- [Cooter2026] Maxwell Cooter, 2026-04-03, "Internet Bug Bounty program hits pause on payouts", InfoWorld, https://www.infoworld.com/article/4154210/internet-bug-bounty-program-hits-pause-on-payouts.html
- [CrowdStrike2026-FiveSteps] CrowdStrike. 2026. Five Steps for Frontier AI Security Readiness. https://www.crowdstrike.com/en-us/resources/white-papers/five-steps-for-frontier-ai-security-readiness/
- [CRA-AnnexI] Cyber Resilience Act (CRA), https://eur-lex.europa.eu/eli/reg/2024/2847/oj/eng#anx_I
- [crs-bug-finding-template] crs-bug-finding-template [for OSS-CRS], https://github.com/Team-Atlanta/crs-bug-finding-template/
- [CSA2026] CSA CISO Community, SANS, [un]prompted, OWASP Gen AI Security Project and the wider community (and many contributing authors), 2026, "The "AI Vulnerability Storm": Building a "Mythos-ready" Security Program", https://labs.cloudsecurityalliance.org/mythos-ciso/
- [Cycode2026] Cycode Team, 6 Steps to be Mythos Ready: How to Prepare for the AI Vulnerability Storm, 2026-05-20, https://cycode.com/blog/claude-mythos-security-readiness/
- [Daelman2025] Rein Daelman, 2025-12-04 (last updated 2026-03-17), "PromptPwnd: Prompt Injection Vulnerabilities in GitHub Actions Using AI Agents", Aikido Security, https://www.aikido.dev/blog/promptpwnd-github-actions-ai-agents
- [DARPA2016] DARPA, 2016-08-05, "DARPA Celebrates Cyber Grand Challenge Winners", https://www.darpa.mil/news/2016/cyber-grand-challenge-winners
- [Diecks2026] Diecks, Jeff, 2026-04-02, "From AIxCC to OpenSSF: Welcoming OSS-CRS to Advance AI Driven", https://openssf.org/blog/2026/04/02/from-aixcc-to-openssf-welcoming-oss-crs-to-advance-ai-driven-open-source-security/
- [Dinaburg2026] Artem Dinaburg, 2026-08-26, VMs won't contain cyber-capable agents, Trail of Bits, https://blog.trailofbits.com/2026/08/26/vms-wont-contain-cyber-capable-agents/
- [Ding2025] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen, 2025, "Vulnerability Detection with Code Language Models: How Far Are We?", 47th IEEE/ACM International Conference on Software Engineering (ICSE 2025), https://arxiv.org/abs/2403.18624
- [Dominus2026-03-05] Mark Dominus, 2026-03-05, Documentation is a message in a bottle, https://blog.plover.com/tech/gpt/documentation-wins.html
- [Dominus2026-03-09] Mark Dominus, 2026-03-09, "Programmers will document for Claude, but not for each other", Blog post, https://blog.plover.com/tech/gpt/documentation-wins-2.html
- [Donnelly2026] Donnelly, Tommy, 2026-04-15, "AI Is Finding Vulnerabilities Faster Than You Can Patch Them. Now What?" https://www.amplifiersecurity.com/blog/ai-vulnerability-management-mythos

- [FiveEyes2026] Five Eyes, 2026-06-22, Five Eyes cyber security agencies statement, https://www.cyber.gov.au/sites/default/files/2026-06/Five%20eyes%20cyber%20security%20agencies%20statement.pdf
- [Gerstenhaber2026] Michael Gerstenhaber and Clemens Viernickel, 2026-07-21, "Now in preview: Find and fix software vulnerabilities with CodeMender", Google Cloud Blog, https://cloud.google.com/blog/products/identity-security/find-and-fix-software-vulnerabilities-with-codemender
- [Google-CodeMender-Docs] Google Cloud, "CodeMender overview", Gemini Enterprise Agent Platform documentation (accessed 2026-09-29), https://docs.cloud.google.com/gemini-enterprise-agent-platform/codemender
- [Grinstead2026-03] Grinstead, Brian, Christian Holler, 2026-03-06, "Hardening Firefox with Anthropic's Red Team" https://blog.mozilla.org/en/firefox/hardening-firefox-anthropic-red-team/
- [Grinstead2026-05] Grinstead, Brian, Christian Holler, Frederik Braun, 2026-05-07, "Behind the Scenes Hardening Firefox with Claude Mythos Preview", https://hacks.mozilla.org/2026/05/behind-the-scenes-hardening-firefox/
- [Guan2026] Aonan Guan, with Zhengyu Liu and Gavin Zhong, 2026-04-15, "Comment and Control: Prompt Injection to Credential Theft in Claude Code, Gemini CLI, and GitHub Copilot Agent", https://oddguan.com/blog/comment-and-control-prompt-injection-credential-theft-claude-code-gemini-cli-github-copilot/
- [Hart2026] Robert Hart, 2026-09-25, One company is at the center of a wave of rogue AI attacks: Mistakes at Israeli startup Irregular sent Anthropic, OpenAI, Meta, and Google agents after real-world targets, https://www.theverge.com/ai-artificial-intelligence/1000644/irregular-rogue-ai-cyberattacks-hacking-openai-meta-anthropic-google
- [Harzevili2024] Nima Shiri Harzevili, Alvine Boaye Belle, Junjie Wang, Song Wang, Zhen Ming (Jack) Jiang, and Nachiappan Nagappan. 2024. "A Systematic Literature Review on Automated Software Vulnerability Detection Using Machine Learning." ACM Comput. Surv. 57, 3, Article 55 (Nov. 2024), 36 pages. https://doi.org/10.1145/3699711
- [Hellekson2026] Gunnar Hellekson et al, 2026-04-08, "Navigating the Mythos-haunted world of platform security", https://www.redhat.com/en/blog/navigating-mythos-haunted-world-platform-security
- [Hillah2026] Lom M. Hillah, Jean-Marc Richard, and Ryan Hasnaoui, 2026-06-11, "Bayesian-Calibrated Detection of Hallucinated Package Imports in AI-Assisted Code", https://arxiv.org/abs/2606.13918
- [Holterhoff2026] Holterhoff, Kate, 2026-05-05, "AI Slop & the Vulnerability Treadmill", https://redmonk.com/kholterhoff/2026/05/05/ai-slop-vulnerability-treadmill/
- [Holley2026] Bobby Holley, 2026-04-21, "The zero-days are numbered", https://blog.mozilla.org/en/privacy-security/ai-security-zero-day-vulnerabilities/
- [Hoodlet2026] Keith Hoodlet, 2026-08-06, Vulnerability patches still require expert human review, https://1password.com/blog/why-ai-generated-patches-still-require-human-review
- [HuggingFace2026] Hugging Face, 2026-07-16, Security incident disclosure — July 2026, https://huggingface.co/blog/security-incident-july-2026
- [Karbasi2026] Amin Karbasi, Supriti Vijay, Aman Priyanshu, Didier Chapoteau, Arthur Goldblatt, Kimia Majd, Fraser Burch, Jianliang He, Baturay Saglam, Takahiro Matsumoto, Zhuoran Yang,

2026-06-21, Introducing Antares: Highly Efficient Open Weight AI Models for Vulnerability Localization, https://blogs.cisco.com/ai/introducing-antares-the-most-efficient-open-weight-ai-models-for-vulnerability-localization

- [Kholoosi2025] M. Mehdi Kholoosi, Triet Huynh Minh Le, M. Ali Babar, 2025-12-23, "Software Vulnerability Management in the Era of Artificial Intelligence: An Industry Perspective", https://arxiv.org/abs/2512.18261v2
- [Kim2026] Kim Taesoo (VP, Agentic Security, Microsoft), 2026-05-12, "Defense at AI speed: Microsoft's new multi-model agentic security system tops leading industry benchmark", https://www.microsoft.com/en-us/security/blog/2026/05/12/defense-at-ai-speed-microsofts-new-multi-model-agentic-security-system-tops-leading-industry-benchmark/
- [Korda2026] Nahum Korda and Gadi Evron, 2026-06-17, "OpenAnt: LLM-Powered Vulnerability Discovery Through Code Decomposition, Adversarial Verification, and Dynamic Testing", https://arxiv.org/abs/2606.19149
- [Kovacs2026-08-24] Eduard Kovacs, 2026-08-24, Anthropic Expands Mythos 5 Access to More Defenders, Unveils $35M Open Source Fund, https://www.securityweek.com/anthropic-expands-mythos-5-access-to-more-defenders-unveils-35m-open-source-fund/
- [Kurmi2025] Ashish Kurmi, 2025-08-27, "s1ngularity: Popular Nx Build System Package Compromised with Data-Stealing Malware", StepSecurity, https://www.stepsecurity.io/blog/supply-chain-security-alert-popular-nx-build-system-package-compromised-with-data-stealing-malware
- [Kwa2025] Thomas Kwa, Ben West, Joel Becker, Amy Deng, Katharyn Garcia, Max Hasin, Sami Jawhar, Megan Kinniment, Nate Rush, Sydney Von Arx, Ryan Bloom, Thomas Broadley, Haoxing Du, Brian Goodrich, Nikola Jurkovic, Luke Harold Miles, Seraphina Nix, Tao Lin, Neev Parikh, David Rein, Lucas Jun Koba Sato, Hjalmar Wijk, Daniel M. Ziegler, Elizabeth Barnes, Lawrence Chan, 2026-02-25, "Measuring AI Ability to Complete Long Software Tasks", https://arxiv.org/abs/2503.14499
- [Kwa2025-blog] Thomas Kwa, Ben West, Joel Becker, et al., 2025-03-19, https://metr.org/blog/2025-03-19-measuring-ai-ability-to-complete-long-tasks/
- [Lakshmanan2026] Ravie Lakshmanan, 2026-05-12, "OpenAI Launches Daybreak for AI-Powered Vulnerability Detection and Patch Validation", The Hacker News, https://thehackernews.com/2026/05/openai-launches-daybreak-for-ai-powered.html
- [Larson2026] Seth Larsen, 2026-02-24, Respecting maintainer time should be in security policies, https://sethmlarson.dev/respecting-maintainer-time-should-be-in-security-policies
- [Li2026] Fengjie Li, Jiajun Jiang, Dongchi Chen, and Yingfei Xiong, 2026-01-27 (revised 2026-09-25), "LLM-based Vulnerability Detection at Project Scale: An Empirical Study", https://arxiv.org/abs/2601.19239
- [Licklider1960] J. C. R. "Lick" Licklider, 1960-03, Man-Computer Symbiosis, IRE Transactions on Human Factors in Electronics, volume HFE-1, pages 4-11, https://groups.csail.mit.edu/medg/people/psz/Licklider.html
- [Li2025] Yue Li, Xiao Li, Hao Wu, Minghui Xu, Yue Zhang, Xiuzhen Cheng, Fengyuan Xu, Sheng Zhong, 2025-04-18, Everything You Wanted to Know About LLM-based Vulnerability Detection But Were Afraid to Ask, https://arxiv.org/abs/2504.13474v1

- [Linux-AI] Linux kernel developers, “AI Coding Assistants”, The Linux Kernel documentation, https://docs.kernel.org/process/coding-assistants.html
- [Linux-SecurityBugs] Linux kernel developers, “Security bugs”, The Linux Kernel documentation, https://docs.kernel.org/process/security-bugs.html
- [LowLevel2026] Low Level, 2026, “Mythos has been unleashed (we have results)” [video], https://www.youtube.com/watch?v=IS4OgH74gY4
- [Microsoft2026-07] Microsoft, 2026-07-24, Open Weights and American AI Leadership, https://www.microsoft.com/en-us/corporate-responsibility/topics/open-weight/
- [Mierczuk2026] Axel Mierczuk, Spencer Michaels, and Keith Hoodlet, 2026, Frontier Models’ Vulnerability Patches are Often F.L.A.W.E.D.: Fix-Like Artifacts With Embedded Defects: Common failure modes of LLM-generated security patches, https://1password.com/files/resources/frontier-models-vulnerability-patches-flawed.pdf
- [Naik2026] Anish Naik et al., 2026-09-15, 1Password’s AI patching benchmark is misleading, Trail of Bits Blog, https://blog.trailofbits.com/2026/09/15/1passwords-ai-patching-benchmark-is-misleading/
- [Nesbitt2026-06] Andrew Nesbitt, 2026-06-25, Scrutineer: scanning open source without flooding maintainers, https://nesbitt.io/2026/06/25/scrutineer.html
- [NIST-AgentIdentity2026] NIST NCCoE, 2026-02-05, “Accelerating the Adoption of Software and Artificial Intelligence Agent Identity and Authorization”, https://www.nccoe.nist.gov/projects/software-and-ai-agent-identity-and-authorization
- [NPR2026] Huo Jingnan, 2026-04-11, How AI is getting better at finding security holes, https://www.npr.org/2026/04/11/nx-s1-5778508/anthropic-project-glasswing-ai-cybersecurity-mythos-preview
- [NVIDIA2026] NVIDIA, 2026, Industry Leaders Unite in Open Secure AI Alliance for AI Safety and Security, https://blogs.nvidia.com/blog/open-secure-ai-alliance/
- [OpenAI2026-07] OpenAI, 2026-07-21, OpenAI and Hugging Face partner to address security incident during model evaluation, https://openai.com/index/hugging-face-model-evaluation-security-incident/
- [OpenSSF2025-AIInstructions] OpenSSF Best Practices and AI/ML Working Groups (led by Avishay Balter), 2025-08-01, “Security-Focused Guide for AI Code Assistant Instructions”, https://best.openssf.org/Security-Focused-Guide-for-AI-Code-Assistant-Instructions.html
- [OpenSSF2026-06] Open Source Security Foundation (OpenSSF) Best Practices Working Group, 2026-06-30, Compiler Options Hardening Guide for C and C++, https://best.openssf.org/Compiler-Hardening-Guides/Compiler-Options-Hardening-Guide-for-C-and-C++.html
- [Oshungboye] Oshungboye, Damilola, UNK-04-22, “How to use AI to identify and fix security vulnerabilities in your codebase”, CodeRabbit, https://dev.to/coderabbitai/how-to-use-ai-to-identify-and-fix-security-vulnerabilities-in-your-codebase-4na2
- [Ottenheimer2026-05-26] David Ottenheimer, 2026-05-26, Mythos Grading Mythos: Got Patches Yet?, https://www.flyingpenguin.com/mythos-grading-mythos-got-patches-yet/
- [OWASP2025-AITesting] OWASP, 2025-11-26, OWASP AI Testing Guide, https://owasp.org/www-project-ai-testing-guide/

- [OWASP-Agentic2026] OWASP GenAI Security Project, 2025-12-09, “OWASP Top 10 for Agentic Applications for 2026”, https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/
- [OWASP-ASI06] Idan Habler, OWASP GenAI Security Project, 2026-05-13, “Memory Is a Feature. It Is Also an Attack Surface”, https://genai.owasp.org/2026/05/13/memory-is-a-feature-it-is-also-an-attack-surface/
- [OWASP-ASVS5] OWASP, 2025, “OWASP Application Security Verification Standard 5.0.0”, (especially V2.2.2 and V8.3.1), https://owasp.org/www-project-application-security-verification-standard/
- [OWASP-LLM01] OWASP GenAI Security Project, “LLM01: Prompt Injection”, https://genai.owasp.org/llmrisk/llm01-prompt-injection/
- [OWASP-LLM06] OWASP GenAI Security Project, “LLM06:2025 Excessive Agency”, https://genai.owasp.org/llmrisk/llm062025-excessive-agency
- [OWASP-ASVS5-V7] OWASP, 2025, “OWASP Application Security Verification Standard 5.0.0”, particularly V7 Session Management, https://github.com/OWASP/ASVS/blob/master/5.0/en/0x16-V7-Session-Management.md
- [OWASP-GenAI] OWASP GenAI Security Project, https://genai.owasp.org/
- [OWASP-LLM10] OWASP GenAI Security Project, “LLM10:2025 Unbounded Consumption”, https://genai.owasp.org/llmrisk/llm102025-unbounded-consumption
- [Pardesi2026-08-05] Rajveer Pardesi and Mrinmay Dey, 2026-08-05, Meta AI model hacks another company during testing, Reuters, https://www.reuters.com/technology/metas-ai-model-hacked-another-company-during-testing-information-reports-2026-08-05/
- [Perry2023] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh, 2023-11, “Do Users Write More Insecure Code with AI Assistants?”, Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS ’23), https://arxiv.org/abs/2211.03622
- [PSF2026] Python Software Foundation (PSF), 2026-06-17, “Everything Security at PyCon US 2026”, https://pyfound.blogspot.com/2026/06/everything-security-at-pycon-us-2026.html
- [Reuters2026-08-07] Reuters, 2026-08-07, Chinese startup Moonshot’s AI model breaks out of testing environment, researchers say, https://www.reuters.com/legal/litigation/chinese-startup-moonshots-ai-model-breaks-out-testing-environment-researchers-2026-08-07/
- [Risse2025] Niklas Risse, Jing Liu, and Marcel Böhme, 2025, “Top Score on the Wrong Exam: On Benchmarking in Machine Learning for Vulnerability Detection”, Proceedings of the ACM on Software Engineering (ISSTA 2025), https://arxiv.org/abs/2408.12986
- [Rohlf2025] Rohlf, Chris, 2025-08-04, AI and the Software Vulnerability Lifecycle, https://cset.georgetown.edu/article/ai-and-the-software-vulnerability-lifecycle/
- [Rogers2025] Rogers, Joshua, 2025, “Hacking with AI SASTs: An overview of ‘AI Security Engineers’ / ‘LLM Security Scanners’ for Penetration Testers and Security Teams”, https://joshua.hu/llm-engineer-review-sast-security-ai-tools-pentesters
- [Safdar2025] Rijha Safdar, Danyail Mateen, Syed Taha Ali, Umer Ashfaq and Wajahat Hussain, 2025, “Data and Context Matter: Towards Generalizing AI-based Software Vulnerability Detection”, https://arxiv.org/abs/2508.16625v2

- [SARIF2.1] OASIS, 2020, “Static Analysis Results Interchange Format (SARIF) Version 2.1.0”, https://docs.oasis-open.org/sarif/sarif/v2.1.0/os/sarif-v2.1.0-os.html
- [Shakevsky2026] Alon Shakevsky, Corban Villa, Ion Stoica, and Raluca Ada Popa, 2026-07-14, “Antiproof: Synthesizing Vulnerability Detectors and Proofs of Exploitability”, https://arxiv.org/abs/2607.12316
- [Shimmi2025] Shimmi, Samiha, Hamed Okhravi, Mona Rahimi, 2025-06-12, “AI-Based Software Vulnerability Detection: A Systematic Literature Review”, https://arxiv.org/abs/2506.10280
- [Shein2026] Esther Shein, 2026-06-03, Investing in Workers to Work with AI: Training encourages workers to utilize AI tools to their fullest and allays fears AI will replace them, https://cacm.acm.org/news/investing-in-workers-to-work-with-ai/
- [Shen2026-09] Chihao Shen, Jiacheng Li, Aastha Mahajan, Jeffery Siyuan Tian, Yonghwi Kwon, Yizheng Chen, 2026-09-03, PatchBench: Evaluating AI Agents for Vulnerability Patching, https://arxiv.org/abs/2609.04075
- [Shostack2014] Adam Shostack, 2014, Threat Modeling: Designing for Security.
- [Shukla2025] Shivani Shukla, Himanshu Joshi, and Romilla Syed, 2025-05-19 (revised 2025-09-26), “Security Degradation in Iterative AI Code Generation: A Systematic Analysis of the Paradox”, https://arxiv.org/abs/2506.11022
- [Silverman2024] Silverman, Micah, 2024-10-14, “Automatically fix code vulnerabilities with AI” https://snyk.io/blog/automatically-fix-code-vulnerabilities-ai/
- [Spracklen2025] Joseph Spracklen, Raveen Wijewickrama, A H M Nazmus Sakib, Anindya Maiti, Bimal Viswanath, and Murtuza Jadliwala, 2025-08, “We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs”, 34th USENIX Security Symposium (USENIX Security 25), https://www.usenix.org/conference/usenixsecurity25/presentation/spracklen
- [Steenhoek2025] Benjamin Steenhoek, Kalpathy Sivaraman, Renata Saldivar Gonzalez, Yevhen Mohylevskyy, Roshanak Zilouchian Moghaddam, Wei Le, 2025-04-25, “Closing the Gap: A User Study on the Real-world Usefulness of AI-powered Vulnerability Detection & Repair in the IDE”, https://arxiv.org/abs/2412.14306v3
- [Stenberg2026-01] Daniel Stenberg, 2026-01-26, “The end of the curl bug-bounty”, https://daniel.haxx.se/blog/2026/01/26/the-end-of-the-curl-bug-bounty/
- [Stenberg2026-04] Daniel Stenberg, 2026-04-22, “High-Quality Chaos”, https://daniel.haxx.se/blog/2026/04/22/high-quality-chaos/
- [Stenberg2026-05a] Stenberg, Daniel, 2026-05-11, “Mythos finds a curl vulnerability”, https://daniel.haxx.se/blog/2026/05/11/mythos-finds-a-curl-vulnerability/
- [Stenberg2026-05b] Stenberg, Daniel, 2026-05, LinkedIn post, “Not even half-way through this #curl release cycle we are already at 11 confirmed vulnerabilities”, https://www.linkedin.com/posts/danielstenberg_curl-curl-share-7463481423543414786-sJew/
- [TesseractedLabs2026] Tesseracted Labs, 2026-09-15, “Enforcing coding-agent guardrails in the runtime instead of the prompt”, https://tesseracted-labs-blog.vercel.app/enforcing-coding-agent-guardrails-in-the-runtime-instead-of-the-prompt
- [TrailofBits2026-06] Trail of Bits, 2026-06-22, Introducing Patch the Planet, https://blog.trailofbits.com/2026/06/22/introducing-patch-the-planet/

- [Vaughan-Nichols2026-02] Vaughan-Nichols, Stephen J., 2026-02-15, “cURL’s Daniel Stenberg: AI slop is DDoSing open source: For open source software, AI is very much a mixed blessing in his view.” https://thenewstack.io/curls-daniel-stenberg-ai-is-ddosing-open-source-and-fixing-its-bugs/
- [Vanian2026] Jonathan Vanian, 2026-08-09, How a small Israeli startup was linked to rogue AI hacks at OpenAI, Anthropic and Meta, AI Age, https://www.cnbc.com/2026/08/09/israeli-startup-irregular-linked-to-ai-hacks-openai-anthropic-meta.html
- [vanZyl] van Zyl, Leon, “Claude Code: Build an AI Agent That Finds Vulnerabilities” (video), https://www.youtube.com/watch?v=VFLieg8JjLA
- [Walsh2026] Joe Walsh, 2026-08-02, CEO of AI firm Hugging Face calls last month’s hack by OpenAI model “very weird and unprecedented”, CBS News, https://www.cbsnews.com/news/hugging-face-hack-openai-rogue-model/
- [Wheeler2025] David A. Wheeler, 2025, Secure AI/ML-Driven Software Development (LFEL1012), https://training.linuxfoundation.org/express-learning/secure-ai-ml-driven-software-development-lfel1012/
- [Wheeler2026] David A. Wheeler, 2026-01-05, AI, Software Development, Security, Tips, and the Future (Part 2), OpenSSF Blog, https://openssf.org/blog/2026/01/05/ai-software-development-security-tips-and-the-future-part-2/
- [Whitehouse2026] Ollie Whitehouse (NCSC CTO), 2026-05-01, “Preparing for a ‘vulnerability patch wave’”, UK National Cyber Security Centre (NCSC), https://www.ncsc.gov.uk/blogs/prepare-for-vulnerability-patch-wave
- [Wolff2026] Dylan Wolff, Martin Mirchev, and Abhik Roychoudhury, 2026-05-12, Large Language Models in Software Security Analysis: LLMs can help tame the complexity at the root of many of today’s software security challenges, Communications of the ACM (CACM) June 2026 Vol 69 No. 6, pp 60-67, https://cacm.acm.org/research/large-language-models-in-software-security-analysis/
- [Xiao2026] Winnie Xiao, Tim Abbott, Nicholas Carlini, Newton Cheng, David Forsythe, Keane Lucas, Milad Nasr, and Shikhar Sakhuja, 2026-06-08, “Measuring LLMs’ impact on N-day exploits”, Anthropic, https://www.anthropic.com/research/n-days
- [Xiong2026] Yunpeng Xiong and Ting Zhang, 2026-01-30 (revised 2026-07-23), “Sifting the Noise: A Comparative Study of LLM Agents in Vulnerability False Positive Filtering”, 35th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2026), https://arxiv.org/abs/2601.22952
- [Veracode] Veracode, “What is AI Code Remediation?” https://www.veracode.com/security/what-is-ai-code-remediation/
- [Yan2026] Written by Eugene Yan and Henna Dattani, et al., 2026-05-27, “Using LLMs to secure source code”, Claude (Anthropic) Blog, https://claude.com/blog/using-llms-to-secure-source-code
- [ZeroDayClock] Zero Day Clock (website). https://zerodayclock.com/ especially https://zerodayclock.com/collapse
- [Zhang2024] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, Abhik Roychoudhury, 2024, AutoCodeRover: Autonomous Program Improvement, https://arxiv.org/abs/2404.05427

- [Zhang2026] Cen Zhang, Younggi Park, Fabian Fleischer, Yu-Fu Fu, Jiho Kim, Dongkwan Kim, Youngjoon Kim, Qingxiao Xu, Andrew Chin, Ze Sheng, Hanqing Zhao, Michael Pelican, David J. Musliner, Jeff Huang, Jon Silliman, Mikel Mcdaniel, Jefferson Casavant, Isaac Goldthwaite, Nicholas Vidovich, Matthew Lehman, Taesoo Kim, 2026-05-29, “SoK: DARPA’s AI Cyber Challenge (AIxCC): Competition Design, Architectures, and Lessons Learned”, https://arxiv.org/abs/2602.07666
- [Ziegler2026] Ziegler, Albert, 2026-05-12, “Mythos for Offensive Security: XBOW’s Evaluation” https://xbow.com/blog/mythos-offensive-security-xbow-evaluation
- [Zimmer2026] Derek Zimmer, 2026-06-16, Private Interview of Derek Zimmer by David A. Wheeler
- [Zorz2026] Zeljka Zorz, 2026-05-18, “AI is drowning software maintainers in junk security reports”, Help Net Security, https://www.helpnetsecurity.com/2026/05/18/problems-with-ai-assisted-vulnerability-research/

10 License

This material is released under the terms of the CC-BY-4.0 Attribution 4.0 International license. You are free to:

- Share — copy and redistribute the material in any medium or format for any purpose, even commercially.
- Adapt — remix, transform, and build upon the material for any purpose, even commercially.
- The licensor cannot revoke these freedoms as long as you follow the license terms.

Under the following terms:

- Attribution — You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
- No additional restrictions — You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits.